



ถุงมือแปลภาษามือ (ข้างเดียว)
(SINGLE-HAND SIGN LANGUAGE TRANSLATOR GLOVE)

โดย
นาย ศุภโชค ชูใจ

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
วิทยาลัยวิศวกรรมศาสตร์ มหาวิทยาลัยรังสิต
ปีการศึกษา 2568

ปริญญานิพนธ์เรื่อง
ถุงมือแปลภาษามือ (ข้างเดียว)
(SINGLE-HAND SIGN LANGUAGE TRANSLATOR GLOVE)

โดย
นาย ศุภโชค ชูใจ 6505918

ได้รับการพิจารณาให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
วิทยาลัยวิศวกรรมศาสตร์ มหาวิทยาลัยรังสิต
ปีการศึกษา 2568

..... ประธานกรรมการ
(ดร. ชรินทร์ วงษ์ใหญ่)

..... กรรมการ
(รศ.ดร. รุ่ง ภูพวงไฟโรจน์)

..... กรรมการและอาจารย์ที่ปรึกษา
(ดร. อภิรักษ์ ภัคตวิงษ์)

(ดร. ชรินทร์ วงษ์ใหญ่)
หัวหน้าหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมคอมพิวเตอร์
วันที่.....เดือน.....พ.ศ.....



ถุงมือแปลภาษามือ (ข้างเดียว)
(SINGLE-HAND SIGN LANGUAGE TRANSLATOR GLOVE)

โดย
นาย ศุภโชค ชูใจ 6505918

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
วิทยาลัยวิศวกรรมศาสตร์ มหาวิทยาลัยรังสิต
ปีการศึกษา 2568

ชื่อ : นาย ศุภโชค ชูใจ รหัส : 6505918
 เรื่อง : ถู่มือแปลภาษามือ (ข้างเดียว)

อาจารย์ที่ปรึกษา : ดร. อภิรักษ์ ภักดีวงศ์, 134 หน้า
 คำสำคัญ : ถู่มือแปลภาษามือ, ปัญญาประดิษฐ์, โครงข่ายประสาทเทียม, อีเอสพี32, ภาษามืออเมริกัน,

บทคัดย่อ

การสื่อสารที่ยากลำบากระหว่างผู้พิการทางการได้ยิน ไม่สามารถสื่อสารกับบุคคลทั่วไปได้ส่งผลกระทบโดยตรงต่อการดำเนินชีวิตและการเข้าสังคม ปัญหานี้ฉบับนี้จึงเสนอการออกแบบและพัฒนา ถู่มืออัจฉริยะสำหรับแปลภาษามือแบบข้างเดียว ซึ่งเป็นอุปกรณ์อำนวยความสะดวกที่พัฒนาบนพื้นฐานของปัญญาประดิษฐ์ เพื่อมุ่งเน้นการใช้งานจริงและช่วยลดอุปสรรคด้านการสื่อสารให้แก่ผู้ใช้งานได้อย่างมีประสิทธิภาพ

ถู่มือแปลภาษามือ (ข้างเดียว) สามารถจำแนกท่าทางภาษามืออเมริกันตัวอักษร A ถึง Z รวมถึงคำศัพท์ที่ใช้บ่อยอย่าง Hello, Thank you, I love you, Yes และ No และแสดงผลเป็นข้อความบนเว็บแอปพลิเคชัน ระบบการทำงานของ ถู่มือแปลภาษามือข้างเดียว ใช้ไมโครคอนโทรลเลอร์ ESP32 อ่านค่าจากเซนเซอร์ตรวจจับการโค้งงอของนิ้วมือ ร่วมกับเซนเซอร์ MPU6050 ที่วัดความเร่งและองศาการหมุนบริเวณข้อมือ และส่งข้อมูลแบบไร้สายผ่านโปรโตคอล UDP ในรูปแบบ JSON ขนาด 60 เฟรมต่อท่าทาง ข้อมูลจะถูกประมวลผลด้วยโครงข่ายประสาทเทียมหลายชั้น (Multi-Layer Perceptron) ที่สร้างขึ้นด้วยไลบรารี Scikit-learn และส่งผลลัพธ์ไปแสดงผลผ่านเฟรมเวิร์ก Flask

จากการทดลองสามารถจำแนกท่าทางภาษามือได้จริง โดยทดสอบท่าทางแต่ละรูปแบบซ้ำรูปแบบละ 30 ครั้ง เพื่อนำผลลัพธ์การทดสอบทั้งหมดมาประมวลผลและคำนวณหาความแม่นยำเฉลี่ย ซึ่งพบว่าระบบมีความแม่นยำในการจำแนกท่าทางภาษามือเฉลี่ยคิดเป็นร้อยละ 96.02 อีกทั้งตัวระบบยังสามารถตอบสนองต่อการเคลื่อนไหวได้อย่างรวดเร็ว มีความหน่วงเวลาดำ และสามารถแปลสัญลักษณ์ภาษามือออกมาเป็นข้อความแสดงผลบนหน้าจอเว็บแอปพลิเคชันได้อย่างราบรื่นและถูกต้อง ซึ่งแสดงให้เห็นว่าระบบถู่มืออัจฉริยะที่พัฒนาขึ้น นี้มีความเสถียรและมีความพร้อมสูงในการนำไปประยุกต์ใช้งานจริงในเชิงสังคม เหมาะสำหรับการนำไปพัฒนาต่อยอดเพื่อใช้งานจริงต่อไป

Name : Mr. Supachok Chujai **ID** : 6505918
Title : Single-Hand Sign Language Translator Glove
Advisor : Apirak Pakdeewong, 134 Pages
Keywords : Sign Language Translation Glove, Artificial Intelligence, Neural Network, ESP32, American Sign Language

Abstract

The communication barrier faced by individuals with hearing impairments in interacting with the general public directly affects their daily life and social participation. This thesis therefore proposes the design and development of a Smart Glove for Single-Hand Sign Language Translation, an assistive device built on artificial intelligence, aimed at practical real-world application and effectively reducing communication barriers for users.

The single-hand sign language translation glove is capable of classifying American Sign Language gestures for letters A through Z, as well as frequently used words such as "Hello," "Thank you," "I love you," "Yes," and "No," and displaying the results as text on a web application. The system operates using an ESP32 microcontroller to read data from flex sensors that detect finger bending, together with an MPU6050 sensor that measures wrist acceleration and rotational angle. Data is transmitted wirelessly via the UDP protocol in JSON format, consisting of 60 frames per gesture. The collected data is then processed using a Multi-Layer Perceptron (MLP) neural network built with the Scikit-learn library, and the resulting output is displayed through the Flask framework.

Experimental results demonstrate that the system can accurately classify real sign language gestures. Each gesture was tested 30 times, and the results were aggregated to calculate the average accuracy, which was found to be 96.02%. Furthermore, the system responds to hand movements quickly with low latency and is able to translate sign language symbols into text displayed on the web application smoothly and accurately. These results indicate that the developed smart glove system is stable and ready for practical, real-world social application, and is well-suited for further development toward actual deployment.

กิตติกรรมประกาศ

ปริญญานิพนธ์ฉบับนี้สำเร็จลุล่วงได้ด้วยความช่วยเหลือ และการสนับสนุนจากบุคคลหลายฝ่าย คณะผู้จัดทำขอขอบคุณคณะกรรมการสอบปริญญานิพนธ์ทุกท่าน รวมถึงคณาจารย์ในภาควิชาวิศวกรรมคอมพิวเตอร์ที่ได้กรุณาให้แนวทาง คำแนะนำ และข้อเสนอแนะอันเป็นประโยชน์ในการปรับปรุงปริญญานิพนธ์ฉบับนี้ให้มีความสมบูรณ์ยิ่งขึ้น

ขอขอบคุณ ดร.อภิรักษ์ ภักดีวงศ์ อาจารย์ที่ปรึกษาปริญญานิพนธ์ ที่ได้กรุณาให้คำแนะนำ และข้อเสนอแนะอย่างใกล้ชิดตลอดระยะเวลาการดำเนินงาน ทั้งในด้านแนวคิดของโครงการ การออกแบบและพัฒนาระบบ รวมถึงการจัดทำรูปเล่มปริญญานิพนธ์ให้เป็นไปตามมาตรฐานที่กำหนด

ขอขอบคุณ รศ.ดร. ดวงอาทิตย์ ศรีมูล ที่ให้คำแนะนำด้านการต่อวงจร

ขอขอบคุณครอบครัวและญาติมิตรทุกท่าน ที่ได้ให้คำแนะนำต่างๆในการทำงาน

ขอขอบคุณ กลุ่มฐานแฟนคลับนายคิวะ ชมแสงสีเสียง เพื่อนผู้คอยให้กำลังใจและดูแลเอาใจใส่ตลอดระยะเวลาการจัดทำปริญญานิพนธ์ฉบับนี้

ศุภโชค ชูใจ
ผู้เขียน

สารบัญ

	หน้า
บทคัดย่อภาษาไทย	ข
บทคัดย่อภาษาอังกฤษ	ค
กิตติกรรมประกาศ	ง
สารบัญ.....	จ
สารบัญตาราง.....	ฉ
สารบัญรูป	ฐ
 บทที่ 1 บทนำ	 1
1.1 ที่มาและความสำคัญของปัญหา.....	1
1.2 วัตถุประสงค์โครงงาน.....	2
1.3 ขอบเขตโครงงาน.....	2
1.4 ประโยชน์และการนำไปใช้งาน	2
1.5 อุปกรณ์ที่ใช้งานและงบประมาณ	2
1.5.1 Hardware	2
1.5.2 Software.....	3
1.5.3 Budget sheet	4
1.6 แผนการดำเนินงาน	4
 บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	 7
2.1 ทฤษฎีเกี่ยวกับอุปสรรคการสื่อสารและภาษามือ	7
2.1.1 กระบวนการสื่อสาร (Communication Process)	7

สารบัญ (ต่อ)

บทที่ 2 (ต่อ)	หน้า
2.1.2 องค์ประกอบของภาษามือ (Parameters of Sign Language).....	7
2.1.3 การสะกดนิ้วมือ (Finger Spelling).....	8
2.1.4 การจำแนกประเภทท่าทางในโครงการ (Classification of Gestures)	8
2.1.5 ความรู้ทั่วไปเกี่ยวกับภาษามือ (Sign Language)	9
2.2 เทคโนโลยีการตรวจจับและแปลภาษามือที่เกี่ยวข้อง	15
2.2.1 แนวทางการตรวจจับด้วยคอมพิวเตอร์วิทัศน์.....	15
2.2.2 แนวทางการตรวจจับด้วยถุงมือติดเซนเซอร์	16
2.3 ไมโครคอนโทรลเลอร์ ESP32 (ESP32 Microcontroller)	16
2.3.1 คุณสมบัติทางเทคนิคและสถาปัตยกรรมหลัก	16
2.4 เซนเซอร์วัดความโค้งงอ (Flex Sensor)	18
2.4.1 โครงสร้างของ Flexsensor	18
2.4.2 วงจรแบ่งแรงดัน (Voltage Divider Circuit).....	19
2.5 เซนเซอร์วัดความเร่งและองศาการหมุน MPU6050	20
2.5.1 เซนเซอร์วัดความเร่ง (Accelerometer)	20
2.5.2 การหาค่ามุมเอียงของเซนเซอร์วัดความเร่ง.....	21
2.5.3 เซนเซอร์วัดความเร็วเชิงมุม (Gyroscope).....	21
2.5.4 กระบวนการแปลงการเบี่ยงเบนเป็นค่าดิบ.....	22
2.5.5 การแปลงค่าดิบเป็นหน่วยองศาต่อวินาที	22
2.6 โพรโตคอลการสื่อสารของระบบ	23
2.6.1 I2C (Inter-Integrated Circuit).....	23
2.6.2 โพรโตคอลการส่งข้อมูลแบบยูดีพี (UDP Protocol).....	23
2.6.3 โครงสร้างแพ็กเก็ตข้อมูล UDP	24

สารบัญ (ต่อ)

บทที่ 2 (ต่อ)	หน้า
2.6.4 ข้อดี	25
2.6.5 ความแตกต่างระหว่าง UDP และ TCP	25
2.7 ปัญญาประดิษฐ์ (Artificial Intelligence : AI).....	25
2.7.1 การใช้งานเทคโนโลยี AI.....	29
2.7.2 ประโยชน์ของปัญญาประดิษฐ์	30
2.8 การเรียนรู้ของเครื่อง (Machine Learning : ML).....	30
2.9 การเรียนรู้เชิงลึก (Deep Learning : DL).....	31
2.10 ระบบโครงข่ายประสาทเทียม (Artificial Neural Network : ANN).....	32
2.10.1 โครงข่ายประสาทเทียมแบบหลายชั้น	33
2.10.2 ฟังก์ชันกระตุ้น (Activation Functions).....	35
2.10.3 กระบวนการฝึกสอนแบบจำลองและการย้อนกลับ	36
2.10.4 การเตรียมชุดข้อมูลและการป้องกันสภาวะ Overfitting	37
2.11 Arduino IDE	38
2.12 วิธีการติดตั้งและใช้งาน Arduino IDE	38
2.12.1 วิธีการดาวน์โหลดและติดตั้ง	38
2.13 Visual Studio Code.....	41
2.13.1 วิธีการดาวน์โหลดและติดตั้ง	41
2.14 Python Programming Language	45
2.15 ไบเบรารีที่ใช้.....	46
2.15.1 NumPy	46
2.15.2 Flask	47
2.15.3 Scikit-learn.....	48

สารบัญ (ต่อ)

บทที่ 2 (ต่อ)	หน้า
2.15.4 Pandas	49
2.15.5 Socket Library.....	50
บทที่ 3 การวิเคราะห์และออกแบบระบบ.....	53
3.1 วิเคราะห์ระบบเดิม	53
3.1.1 การสื่อสารผ่านล่ามภาษามือ	53
3.1.2 ระบบแปลภาษามือด้วยกล้องและการประมวลผลภาพ	53
3.1.3 ระบบถุงมือแปลภาษามือแบบเดิม (Sensor-based Glove).....	54
3.2 วิเคราะห์ระบบใหม่	54
3.3 การออกแบบวงจรฮาร์ดแวร์ (Hardware Circuit Design)	55
3.4 Flowchart การทำงานของระบบ	56
3.5 ขั้นตอนการดำเนินงาน	58
3.5.1 การดำเนินงานในส่วน Hardware	58
3.5.2 การดำเนินงานในส่วนของ Software.....	61
3.6 การสร้าง Dataset	62
3.6.1 โครงสร้างของชุดข้อมูล.....	62
3.6.2 การเตรียมอุปกรณ์และซอฟต์แวร์.....	63
3.6.3 ขั้นตอนการเก็บรวบรวมชุดข้อมูล	63
3.6.4 การวิเคราะห์โครงสร้างโปรแกรมเก็บข้อมูล	64
3.6.5 รูปแบบการเก็บข้อมูล.....	66
3.7 Flowchart การเทรนโมเดล.....	71
3.8 การออกแบบโมเดล	73

สารบัญ (ต่อ)

บทที่ 3 (ต่อ)	หน้า
3.8.1 อธิบายโค้ดการเรนโมเดล.....	74
3.9 การพัฒนา Web Application.....	81
3.9.1 เทคโนโลยีที่ใช้ในการพัฒนา	81
3.9.2 อธิบายโค้ดส่วนการแสดงผล.....	81
3.10 สรุปท้ายบทที่ 3	85
บทที่ 4 ผลการดำเนินการ	86
4.1 ผลการดำเนินงาน.....	86
4.1.1 การเรนโมเดลและจัดเตรียมโมเดลปัญญาประดิษฐ์	86
4.1.2 กระบวนการประมวลผลข้อมูลแบบช่วงเวลา	86
4.1.3 การแสดงผลลัพธ์ผ่านเว็บแอปพลิเคชัน.....	87
4.2 ผลการประเมินประสิทธิภาพของโมเดลปัญญาประดิษฐ์	88
4.3 ผลการแสดงผลการจำแนกท่าทาง	109
4.4 สรุปผลการทดลอง	120
บทที่ 5 สรุปผลการดำเนินงานและข้อเสนอแนะ	121
5.1 สรุปผลการดำเนินงาน.....	121
5.2 ปัญหาและอุปสรรค.....	121
5.3 แนวทางในการนำไปพัฒนาต่อ	121

สารบัญ (ต่อ)

บรรณานุกรม.....	123
ภาคผนวก ก.โค้ดการแทนการแปลภาษามือ.....	124
ข.โค้ดการทำงานของแอปพลิเคชันทำนายผลภาษามือ.....	126
ค.โค้ดการแทนการส่งข้อมูล	128
ง.โค้ดการแทนการเก็บข้อมูล.....	132
ประวัติผู้เขียน	134

สารบัญตาราง

ตารางที่	หน้า
1.1 รายชื่ออุปกรณ์	4
1.2 แผนการดำเนินงานและระยะในการปฏิบัติงาน.....	5
2.1 ภาษามือและการสะกดนิ้วมือ	10
3.1 รายละเอียดของ Flex Sensor.....	56
3.2 รายละเอียดของ MPU 6050	56
3.3 โครงสร้างชุดข้อมูล	62
3.4 รูปแบบการเก็บข้อมูล Flex Sensor.....	66
3.5 รูปแบบการเก็บข้อมูลความเร่ง Accelerometer ของ MPU6050.....	68
3.6 รูปแบบการเก็บข้อมูลความเร็วเชิงมุม Gyroscope ของ MPU6050	69
4.1 ผลการทดสอบการแปลตัวอักษร A.....	88
4.2 ผลการทดสอบการแปลตัวอักษร B.....	89
4.3 ผลการทดสอบการแปลตัวอักษร C.....	89
4.4 ผลการทดสอบการแปลตัวอักษร D.....	90
4.5 ผลการทดสอบการแปลตัวอักษร E	91
4.6 ผลการทดสอบการแปลตัวอักษร F	91
4.7 ผลการทดสอบการแปลตัวอักษร G.....	92
4.8 ผลการทดสอบการแปลตัวอักษร H	93
4.9 ผลการทดสอบการแปลตัวอักษร I.....	93
4.10 ผลการทดสอบการแปลตัวอักษร J	94
4.11 ผลการทดสอบการแปลตัวอักษร K.....	95
4.12 ผลการทดสอบการแปลตัวอักษร L.....	95
4.13 ผลการทดสอบการแปลตัวอักษร M.....	96
4.14 ผลการทดสอบการแปลตัวอักษร N	97
4.15 ผลการทดสอบการแปลตัวอักษร O	97
4.16 ผลการทดสอบการแปลตัวอักษร P.....	98
4.17 ผลการทดสอบการแปลตัวอักษร Q	99

สารบัญตาราง (ต่อ)

ตารางที่	หน้า
4.18 ผลการทดสอบการแปลตัวอักษร R	99
4.19 ผลการทดสอบการแปลตัวอักษร S	100
4.20 ผลการทดสอบการแปลตัวอักษร T	101
4.21 ผลการทดสอบการแปลตัวอักษร U	101
4.22 ผลการทดสอบการแปลตัวอักษร V	102
4.23 ผลการทดสอบการแปลตัวอักษร W	103
4.24 ผลการทดสอบการแปลตัวอักษร X	103
4.25 ผลการทดสอบการแปลตัวอักษร Y	104
4.26 ผลการทดสอบการแปลตัวอักษร Z	105
4.27 ผลการทดสอบการแปลคำว่า Hello	105
4.28 ผลการทดสอบการแปลคำว่า Thank you	106
4.29 ผลการทดสอบการแปลตัวอักษร I love you	107
4.30 ผลการทดสอบการแปลคำว่า Yes	107
4.31 ผลการทดสอบการแปลคำว่า No	108
4.32 ผลการแสดงการจำแนกทำทาง	109

สารบัญรูป

รูปที่	หน้า
2.1 การสะกดนิ้วมือ	8
2.2 ท่าทางแบบหยุดนิ่ง	9
2.3 ท่าทางแบบเคลื่อนไหว	9
2.4 ตำแหน่ง PIN ESP32	17
2.5 โครงสร้างของ Flexsensor	18
2.6 ทิศทางการหมุนของ MPU6050	20
2.7 โครงสร้าง UDP	24
2.8 เปรียบเทียบ AI, Machine Learning, Deep Learning	26
2.9 การนำ AI มาใช้ในด้านอุตสาหกรรม	29
2.10 การเรียนรู้ของ Machine Learning	31
2.11 การเรียนรู้ของ Deep Learning	32
2.12 โครงข่ายประสาทเทียม	33
2.13 โครงสร้าง MLP	33
2.14 Backpropagation	34
2.15 การติดตั้ง Arduino IDE	38
2.16 การเลือกโฟลเดอร์ที่จะใช้เก็บไฟล์	39
2.17 การติดตั้ง Arduino IDE	39
2.18 การติดตั้ง Arduino IDE สมบูรณ์	40
2.19 หน้าต่าง Arduino IDE	40
2.20 การติดตั้ง Visual Studio Code	41
2.21 การเลือกโฟลเดอร์ที่จะใช้เก็บไฟล์	42
2.22 โฟลเดอร์เมนู Start สำหรับโค้ด Visual Studio	42
2.23 กำหนดค่าการติดตั้ง	43
2.24 ยืนยันการติดตั้ง	43
2.25 การติดตั้ง Visual Studio	44
2.26 การติดตั้ง Visual Studio สมบูรณ์	44

สารบัญรูป (ต่อ)

รูปที่	หน้า
2.27 หน้าต่าง Visual Studio Code.....	45
2.28 ตัวอย่างการใช้งาน Python	46
2.29 ติดตั้ง Numpy	46
2.30 เรียกใช้ Numpy.....	47
2.31 ตัวอย่างการใช้งาน Numpy	47
2.32 ติดตั้ง Flask	47
2.33 เรียกใช้งาน Flask.....	47
2.34 ตัวอย่างการใช้งาน Flask	48
2.35 ติดตั้ง Scikit learn	48
2.36 เรียกใช้ Scikit learn.....	48
2.37 ตัวอย่างการใช้งาน Scikit learn.....	48
2.38 ติดตั้ง Pandas	49
2.39 เรียกใช้ Pandas.....	49
2.40 ตัวอย่างการใช้งาน Pandas	50
2.41 ติดตั้ง Socket Library.....	50
2.42 เรียกใช้ Socket.....	50
2.43 ตัวอย่างการใช้งาน Socket	51
2.44 ตัวอย่างโค้ดของ Client	51
2.45 Server	52
2.46 Client.....	52
3.1 การออกแบบวงจร.....	55
3.2 Flowchart การทำงานของระบบ.....	57
3.3 Flex Sensor.....	58
3.4 MPU 6050	59
3.5 ESP32.....	59
3.6 การติดตั้ง Flex Sensor	60

สารบัญรูป (ต่อ)

รูปที่	หน้า
3.7 ติดตั้ง ESP32 และ MPU6050 ลงบนบอร์ด	60
3.8 บัดกรีอุปกรณ์ทั้งหมด	61
3.9 ขั้นตอนการเก็บข้อมูล	64
3.10 โค้ดตั้งค่า UDP	64
3.11 โค้ดล้าง Buffer	65
3.12 โค้ดรวบรวมข้อมูล	65
3.13 โค้ดการจัดเก็บข้อมูล	65
3.14 Flowchart การเทรนโมเดล	72
3.15 โครงสร้างโมเดล	73
3.16 โค้ดสำหรับนำเข้าไลบรารี	74
3.17 โค้ดการกำหนดข้อมูลนำเข้า	75
3.18 โค้ดจัดรูปแบบข้อมูลให้พร้อมสำหรับการเทรน	75
3.19 โค้ดการแบ่งข้อมูลเป็น Train และ Test	76
3.20 โค้ดสำหรับการปรับสเกล	77
3.21 โค้ดสำหรับการสร้างโมเดล	77
3.22 เป็นโค้ดสร้างแผนภาพ Confusion Matrix	78
3.23 กราฟประเมิน (Confusion Matrix)	79
3.24 โค้ดสำหรับการบันทึกโมเดล	80
3.25 ไฟล์ที่ได้จากการเทรน	80
3.26 โค้ดสำหรับนำเข้าไลบรารี	81
3.27 โค้ดสำหรับกำหนดตัวแปร	82
3.28 โค้ดสำหรับการโหลดโมเดล	82
3.29 โค้ดสำหรับกำหนดรูปแบบของ Template	83
3.30 โค้ดสำหรับสร้างช่องทางรับข้อมูลจาก ESP32	83
3.31 โค้ดสำหรับรับและแปลงข้อมูล	84
3.32 โค้ดสำหรับการทำนายผลลัพธ์	84

สารบัญรูป (ต่อ)

รูปที่	หน้า
4.1 ถู่มือต้นแบบที่พัฒนาขึ้น.....	87
4.2 หน้าต่าง Web Application	87

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของปัญหา

การสื่อสารเป็นปัจจัยพื้นฐานที่สำคัญในการดำรงชีวิตและการอยู่ร่วมกันในสังคม อย่างไรก็ตาม ผู้ที่มีความบกพร่องทางการได้ยินหรือการพูด (Deaf and Hard of Hearing Community) จะประสบปัญหาและอุปสรรคในการสื่อสารร่วมกับบุคคลในสังคมเนื่องจากความแตกต่างทางภาษา โดยภาษามือ (Sign Language) ซึ่งเป็นเครื่องมือหลักในการสื่อสารของพวกเขานั้น ไม่เป็นที่เข้าใจอย่างแพร่หลายในบุคคลทั่วไป ส่งผลให้เกิดช่องว่างในการสื่อสาร การเข้าถึงบริการสาธารณะ และโอกาสในการประกอบอาชีพ การพึ่งพาล่ามภาษามือนั้นมีข้อจำกัดทั้งในด้านค่าใช้จ่ายและความไม่สะดวกในการเรียกใช้บริการในสถานการณ์เร่งด่วน

ในปัจจุบัน เทคโนโลยีสิ่งอำนวยความสะดวก (Assistive Technology) ได้เข้ามามีบทบาทในการแก้ปัญหา งานวิจัยส่วนใหญ่นิยมใช้เทคโนโลยีคอมพิวเตอร์วิทัศน์ (Computer Vision) ผ่านกล้องเพื่อแปลภาษามือ แต่มีข้อจำกัดเรื่องสภาพแสง มุมกล้อง และความเป็นส่วนตัว ในขณะที่เทคโนโลยีอุปกรณ์สวมใส่ (Wearable Device) เช่น ถุงมืออัจฉริยะ (Smart Glove) สามารถตรวจจับการเคลื่อนไหวของนิ้วมือได้โดยตรงและมีความแม่นยำสูงกว่าในสภาพแวดล้อมที่หลากหลาย แต่ทว่าอุปกรณ์ที่มีอยู่ในท้องตลาดหรือในงานวิจัยเดิมจะมีการเชื่อมต่อแบบมีสาย (Wired) ซึ่งจำกัดระยะการเคลื่อนไหว หรือมีต้นทุนในการผลิตสูง

จากปัญหาดังกล่าว คณะผู้จัดทำจึงมีแนวคิดที่จะพัฒนา "ระบบแปลภาษามือเป็นข้อความและเสียงแบบไร้สายโดยใช้โครงข่ายประสาทเทียม" โดยประยุกต์ใช้ไมโครคอนโทรลเลอร์ ESP32 ที่มีราคาประหยัดและรองรับการเชื่อมต่อไร้สาย (Wi-Fi) ร่วมกับเซนเซอร์ตรวจจับการงอของนิ้ว (Flex Sensors) และเซนเซอร์ตรวจจับการเคลื่อนไหว (IMU) ข้อมูลจากถุงมือจะถูกส่งผ่านโปรโตคอล UDP ไปยังคอมพิวเตอร์เพื่อประมวลผลด้วยโมเดลปัญญาประดิษฐ์แบบโครงข่ายประสาทเทียมหลายชั้น (Multi-layer Perceptron) ที่พัฒนาด้วย Scikit-Learn และแสดงผลการแปลผ่านเว็บแอปพลิเคชัน เพื่อสร้างต้นแบบนวัตกรรมที่ช่วยลดช่องว่างในการสื่อสาร ใช้งานสะดวก และมีประสิทธิภาพ

1.2 วัตถุประสงค์โครงการ

1. เพื่อพัฒนา ถังมือแปลภาษามือ ที่สามารถตรวจจับการเคลื่อนไหวของมือและนิ้ว
2. เพื่อพัฒนาโมเดลปัญญาประดิษฐ์ (AI) ประเภทโครงข่ายประสาทเทียม (Multi-layer Perceptron Classifier) สำหรับจำแนกท่าทางภาษามือจากข้อมูลเซนเซอร์
3. เพื่อเป็นเครื่องมือช่วยสื่อสารระหว่างผู้พิการทางการได้ยินกับบุคคลทั่วไป ลดช่องว่างทางการสื่อสาร
4. เพื่อพัฒนาระบบแสดงผลผ่านเว็บแอปพลิเคชันที่สามารถแสดงผลการทำงานของแปลเป็นข้อความได้

1.3 ขอบเขตโครงการ

1. ใช้บอร์ดไมโครคอนโทรลเลอร์ ESP32 เป็นตัวส่งข้อมูล
2. ใช้ Flex Sensors จำนวน 5 ชิ้น เพื่อวัดค่าการงอของนิ้วมือทั้ง 5 นิ้ว
3. ใช้เซนเซอร์ MPU6050 (Accelerometer & Gyroscope) จำนวน 1 ชิ้น เพื่อวัดระนาบและการเคลื่อนไหวของข้อมือ
4. ใช้ Flask สำหรับพัฒนาเว็บและแสดงผลโดยมีตัวควบคุมในการกดแปล
5. รองรับเฉพาะท่าทางมือเดียว ได้แก่ A-Z และ คำศัพท์ (Hello ,Thank you, I love you, Yes ,no)
6. สามารถทำนายผลท่าทางภาษามือจากค่า Flex sensor และ MPU 6050 โดยใช้ MLP

1.4 ประโยชน์และการนำไปใช้งาน

1. ช่วยให้ผู้พิการทางการได้ยินและการพูดสามารถ สื่อสารกับบุคคลทั่วไปได้สะดวกและรวดเร็วขึ้น
2. ลดความเข้าใจผิดและอุปสรรคด้านการสื่อสาร ทำให้เกิด ความเท่าเทียมในสังคม
3. เป็นสื่อการเรียนรู้ภาษามือสำหรับบุคคลทั่วไปที่ต้องการเรียนรู้การสื่อสารกับผู้พิการ
4. ใช้ใน สถานศึกษา เพื่อช่วยนักเรียน/นักศึกษาที่มีความบกพร่องทางการได้ยินในการเรียนร่วมกับบุคคลทั่วไป
5. สามารถพัฒนาต่อยอดเป็น แอปพลิเคชันบนมือถือ ที่เชื่อมต่อกับถังมือ เพื่อเพิ่มความสะดวกและรองรับหลายภาษา

1.5 อุปกรณ์ที่ใช้งานและงบประมาณ

1.5.1 Hardware

- Notebook Asus TUF Gaming F17 / Intel Core i5-11400H / NVIDIA GeForce RTX 3050 / RAM 8GB DDR4 / SSD 512GB

- ESP 32 Microcontroller ใช้สำหรับประมวลผลข้อมูลของเซ็นเซอร์และสื่อสารกับคอมพิวเตอร์ด้วย WiFi

- Flex sensors 2.2" ใช้ตรวจวัดการงอของนิ้วมือ
- MPU6050 ใช้สำหรับกำหนดตำแหน่งของมือ
- ถังมือ
- สายจัมเปอร์
- ตัวต้านทาน 10 k Ω

1.5.2 Software

- Visual Studio Code ในการเขียนและออกแบบหน้า UI ของเว็บ ภาษา Python
- ภาษาโปรแกรม Python สำหรับเขียนชุดคำสั่งควบคุมระบบ และใช้พัฒนาแอปพลิเคชันร่วมกับระบบแสดงผล

- Pandas เป็นไลบรารีในภาษา Python ที่ใช้สำหรับการจัดการและวิเคราะห์ ข้อมูลในรูปแบบของตารางหรือ DataFrame

- NumPy ไลบรารีโอเพนซอร์สใน Python ที่ใช้สำหรับการคำนวณทางคณิตศาสตร์ และการจัดการข้อมูลเชิงตัวเลข ในการทำงานกับอาร์เรย์หลายมิติและเมทริกซ์

- Arduino IDE คือโปรแกรมพัฒนาและอัปโหลดโค้ดไปยัง microcontroller โดยรองรับภาษา C/C++ และออกแบบมาเพื่อใช้งานด้านไมโครคอนโทรลเลอร์

- CSS ในกำหนดรูปแบบการแสดงผลของหน้าเว็บ เช่น สี ตัวอักษร และการจัดวางองค์ประกอบ

- HTML สำหรับสร้างและจัดโครงสร้างเนื้อหาบนหน้าเว็บเพจ

- Scikit-learn เป็นไลบรารีของภาษา Python สำหรับการเรียนรู้ของเครื่อง ที่รวมเครื่องมือสำหรับการจำแนก การถดถอย การจัดกลุ่ม และการประเมินโมเดล

- Neural Network เป็นอัลกอริทึมการเรียนรู้ของเครื่องที่ใช้สำหรับการจำแนกข้อมูล โดยหาขอบเขตที่เหมาะสมที่สุดระหว่างกลุ่มข้อมูล

- Flask เป็นเครื่องมือของภาษา Python สำหรับสร้างและจัดการเว็บแอปพลิเคชันแบบง่ายและรวดเร็ว โดยใช้สถาปัตยกรรมแบบไมโครเฟรมเวิร์ก

- JavaScript ในการเขียนโปรแกรมที่ใช้สำหรับเพิ่มความโต้ตอบ

1.5.3 Budget sheet

การพัฒนาถุงมือแปลภาษามือข้างเดียวนี้ได้มีการจัดซื้ออุปกรณ์ขึ้นเพื่อให้การพัฒนาเป็นไปด้วยความสะดวก และตามขอบเขตของโครงการ มีรายละเอียดดังตารางที่ 1.1

ตารางที่ 1.1 รายชื่ออุปกรณ์

ลำดับ	รายการวัสดุ	จำนวน (หน่วย)	ค่าใช้จ่าย(บาท)	หมายเหตุ
Hardware				
1	Asus TUF Gaming F17	1	34,990	-
2	Flex sensors 2.2"	5	595	-
3	MPU6050	1	80	-
4	ถุงมือ	1	36	-
5	สายจัมเปอร์	20	22	-
Software				
1	Visual Studio Code 1.92	1	-	freeware
2	Python 3.14	1	-	freeware
3	Pandas 2.3.3	1	-	freeware
4	NumPy 2.3.2	1	-	freeware
5	Arduino IDE 2.0	1	-	freeware
6	CSS	1	-	freeware
7	HTML	1	-	freeware
8	Scikit-learn 1.7.2	1	-	freeware
9	Neural network	1	-	freeware
10	Flask 3.1	1	-	freeware
11	JavaScript	1	-	freeware

1.6 แผนการดำเนินงาน

การวางแผนดำเนินงานเริ่มต้นจากการศึกษาองค์ประกอบ ที่จำเป็นสำหรับการทำโครงการ ได้แก่ การศึกษาระบบการทำงานของโครงการ รวมไปถึงการศึกษาและเรียนรู้ภาษาต่างๆ หลังจากเสร็จขั้นตอนการศึกษาแล้ว จึงดำเนินการออกแบบโครงการและ พัฒนาระบบพร้อมทั้งปรับปรุงแก้ไข เมื่อ

ตารางที่ 1.2 แผนการดำเนินงานและระยะในการปฏิบัติงาน (ต่อ)

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎีเกี่ยวกับอุปสรรคการสื่อสารและภาษามือ

2.1.1 กระบวนการสื่อสาร (Communication Process)

ผู้ที่มีความบกพร่องทางการได้ยินสื่อสารกันด้วยภาษามือ (Sign Language) ซึ่งถ่ายทอดความหมายผ่านท่าทางแทนเสียงพูด แต่ละประเทศมีภาษามือของตนเองที่มีโครงสร้างทางไวยากรณ์และคำศัพท์แตกต่างกันไป เช่น ภาษามือไทย (TSL) และภาษามืออเมริกัน (ASL) โครงการนี้เลือกพัฒนาโดยอ้างอิงภาษามืออเมริกัน เนื่องจากเป็นภาษามือที่มีผู้ใช้งานแพร่หลายที่สุดและมีชุดข้อมูลสาธารณะรองรับการวิจัยจำนวนมาก ทำให้สามารถเปรียบเทียบผลการทดลองกับงานอื่นได้

2.1.2 องค์ประกอบของภาษามือ (Parameters of Sign Language)

ในทางภาษาศาสตร์ ภาษามือไม่ได้เกิดจากการขยับมือโดยสุ่ม แต่ประกอบขึ้นจากหน่วยย่อย 5 ส่วนที่เรียกว่า The Five Parameters of ASL ได้แก่ รูปร่างมือ (Handshape), ทิศทางฝ่ามือ (Palm Orientation), การเคลื่อนไหว (Movement), ตำแหน่งของมือเทียบกับร่างกาย (Location) และการแสดงออกทางสีหน้า (Non-manual markers)

องค์ประกอบทั้งห้านี้เป็นตัวกำหนดโดยตรงว่าถ้ามือในโครงการนี้ต้องติดตั้งเซนเซอร์ชนิดใดบ้างจึงจะครอบคลุมความหมายของท่าทาง กล่าวคือ รูปร่างมือและการเคลื่อนไหวสามารถวัดได้ด้วย Flex Sensor และ IMU ในขณะที่การแสดงออกทางสีหน้านั้นอยู่นอกขอบเขตของโครงการนี้เนื่องจากถ้ามือสามารถตรวจจับได้เฉพาะสัญญาณจากมือเท่านั้น

2.1.3 การสะกดนิ้วมือ (Finger Spelling)

การสะกดนิ้วมือ (Fingerspelling) คือการใช้รูปมือแทนตัวอักษรในภาษาพูด ซึ่งใช้ในกรณีที่คำนั้นไม่มีท่าภาษามือเฉพาะ หรือใช้สำหรับสะกดชื่อเฉพาะ ในภาษามืออเมริกัน การสะกดนิ้วมือใช้มือเพียงข้างเดียว ซึ่งสอดคล้องพอดีกับแนวทางของถุงมือข้างเดียวที่โครงการนี้พัฒนาขึ้น ดังรูปที่ 2.1



รูปที่ 2.1 การสะกดนิ้วมือ

2.1.4 การจำแนกประเภทท่าทางในโครงการ (Classification of Gestures)

ในการประมวลผลด้วยคอมพิวเตอร์ ท่าทางภาษามือในขอบเขต A-Z และ คำศัพท์ สามารถแบ่งออกเป็น 2 ประเภทหลัก คือ

1. ท่าทางแบบหยุดนิ่ง (Static Gestures) คือท่าทางที่ไม่มีการเคลื่อนไหวของข้อมือ ในขณะที่แสดงความหมาย เพียงแค่จัดระเบียบนิ้วมือและคางไว้ ดังรูปที่ 2.2



รูปที่ 2.2 ท่าทางแบบหยุดนิ่ง

2. ท่าทางแบบเคลื่อนไหว (Dynamic Gestures) คือท่าทางที่ต้องมีการเปลี่ยนตำแหน่งหรือทิศทางของมือในระหว่างการทำท่าทาง ดังรูปที่ 2.3

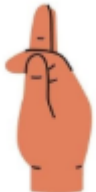


รูปที่ 2.3 ท่าทางแบบเคลื่อนไหว

2.1.5 ความรู้ทั่วไปเกี่ยวกับภาษามือ (Sign Language)

ภาษามือ คือ ภาษาศาสตร์ชาติที่ใช้ระบบการสื่อสารผ่านอวัจนภาษา แทนการใช้เสียง โดยเป็นเครื่องมือสำคัญในการสื่อสารของผู้บกพร่องทางการได้ยิน ตลอดจนบุคคลทั่วไปที่ต้องการติดต่อสื่อสารร่วมกัน ภาษามือไม่ได้เป็นเพียงแค่การทำท่าทางประกอบคำพูด แต่มีโครงสร้างทางไวยากรณ์ รูปประโยค และคลังคำศัพท์ที่เป็นเอกเทศและมีความสมบูรณ์ในตัวเอง ดังตารางที่ 2.1

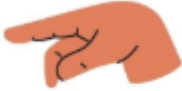
ตารางที่ 2.1 ภาษามือและการสะกดนิ้วมือ

รูปแบบ	ตัวอักษร	ลักษณะท่าทาง	การเคลื่อนไหว
 A	A	กำมือ นิ้วหัวแม่มือ เหยียดตรงแนบชิด ด้านข้างของนิ้วชี้	นิ่ง
 B	B	แบมือ นิ้วทั้งสี่เรียงชิด กัน นิ้วหัวแม่มือพับงอ มาแตะที่ฝ่ามือ	นิ่ง
 C	C	ทำมือเป็นรูปตัว C นิ้ว ทั้งสี่เรียงชิดและโค้งงอ นิ้วหัวแม่มือโค้งรับ	นิ่ง
 E	E	พับนิ้วทั้งสี่ลงมาแตะที่ อุ้งมือ นิ้วหัวแม่มือพับ เก็บไว้ใต้นิ้วทั้งสี่	นิ่ง
 D	d	ชูนิ้วขึ้นให้นิ้วหัวแตะ กับปลายนิ้วกลาง นาง และก้อยเป็นวงกลม	นิ่ง
 F	F	ปลายนิ้วชี้แตะปลาย นิ้วหัวแม่มือเป็น วงกลม นิ้วที่เหลือกาง ออก	นิ่ง

ตารางที่ 2.1 ภาษามือและการสะกดนิ้วมือ (ต่อ)

รูปแบบ	ตัวอักษร	ลักษณะท่าทาง	การเคลื่อนไหว
 G	G	ชูนิ้วชี้และนิ้วหัวแม่มือ ขนานกันชี้ไปด้านข้าง (คล้ายท่าจับของ) นิ้ว อื่นกำไว้	นิ่ง
 H	H	ชูนิ้วชี้และนิ้วกลางคู่ กันชี้ไปด้านข้าง นิ้วหัวแม่มือซ่อนไว้	นิ่ง
 I	I	ชูนิ้วก้อยขึ้นเพียงนิ้ว เดียว นิ้วอื่นกำไว้	นิ่ง
 J	J	ท่าท่าเหมือนตัว I	หมุนข้อมือวาด เป็นรูปตัว J ใน อากาศ
 K	K	ชูนิ้วชี้และนิ้วกลางเป็น รูปตัว V โดยมี นิ้วหัวแม่มือแทรกอยู่ ตรงกลาง	นิ่ง
 L	L	ชูนิ้วชี้และนิ้วหัวแม่มือ ตั้งฉากกันเป็นรูปตัว L	นิ่ง

ตารางที่ 2.1 ภาษามือและการสะกดนิ้วมือ (ต่อ)

รูปแบบ	ตัวอักษร	ลักษณะท่าทาง	การเคลื่อนไหว
 M	M	กำมือ โดยให้ 3 นิ้ว (ชี้, กลาง, นาง) พาดทับอยู่บนนิ้วหัวแม่มือ	นิ่ง
 N	N	กำมือ โดยให้ 2 นิ้ว (ชี้, กลาง) พาดทับอยู่บนนิ้วหัวแม่มือ	นิ่ง
 O	O	ปลายนิ้วทั้ง 5 มาชนกันเป็นวงกลม	นิ่ง
 P	P	ท่าท่าเหมือนตัว K แต่คว่ำมือลงให้นิ้วชี้ชี้ลงพื้น	นิ่ง
 Q	Q	ท่าท่าเหมือนตัว G แต่คว่ำมือลงให้นิ้วทั้งสองชี้ลงพื้น	นิ่ง
 R	R	ชูนิ้วชี้และนิ้วกลางไขว้กัน	นิ่ง

ตารางที่ 2.1 ภาษามือและการสะกดนิ้วมือ (ต่อ)

รูปแบบ	ตัวอักษร	ลักษณะท่าทาง	การเคลื่อนไหว
 S	S	กำมือแน่นนิ้วหัวแม่มือ พาดทับอยู่ด้านหน้า ของนิ้วชี้และนิ้วกลาง	นิ่ง
 T	T	กำมือ โดยสอด นิ้วหัวแม่มือให้อยู่ ระหว่างนิ้วชี้และ นิ้วกลาง (ชุนิ้วชี้ทับ โป้ง)	นิ่ง
 U	U	ชุนิ้วชี้และนิ้วกลางชิด กันขึ้นฟ้า	นิ่ง
 V	V	ชุนิ้วชี้และนิ้วกลาง แยกออกจากกันเป็น รูปตัว V	นิ่ง
 W	W	ชู 3 นิ้ว (ชี้, กลาง, นาง) แยกออกจากกัน	นิ่ง

ตารางที่ 2.1 ภาษามือและการสะกดนิ้วมือ (ต่อ)

รูปแบบ	ตัวอักษร	ลักษณะท่าทาง	การเคลื่อนไหว
 X	X	กำมือแล้วชูนิ้วชี้ขึ้น แต่งอปลายนิ้วลงมา เป็นตะขอ	นิ่ง
 Y	Y	กางนิ้วหัวแม่มือและ นิ้วก้อยออก นิ้วที่ เหลือพับเก็บ	นิ่ง
 Z	Z	ชูนิ้วชี้	วาดเป็นรูปตัว Z ในอากาศ
 HELLO	Hello (สวัสดี)	แบมือ ปลายนิ้วแตะ บริเวณหน้าผากหรือ ทางคิ้ว	ขยับมือออก จากหน้าผากไป ด้านข้างลำตัว เล็กน้อย
 THANK YOU	Thank you (ขอบคุณ)	แบมือ (Open B) ปลายนิ้วแตะที่คาง	ยื่นมือออกจาก คางไปด้านหน้า คู่สนทนาและ ลดระดับลง เล็กน้อย

ตารางที่ 2.1 ภาษามือและการสะกดนิ้วมือ (ต่อ)

รูปแบบ	ตัวอักษร	ลักษณะท่าทาง	การเคลื่อนไหว
	I love you (ฉันรักคุณ)	ชูนิ้วชี้และนิ้วกลางขึ้น 2 นิ้ว	นิ่ง
	Yes (ใช่)	กำนิ้วมือทั้งหมด หลวม ๆ (คล้ายตัวอักษร S) แล้วขยับข้อมือผงกลงซ้ำ ๆ	หักข้อมือขึ้นและลงซ้ำ ๆ
	No (ไม่ใช่)	ชูนิ้วโป้ง นิ้วชี้ และนิ้วกลางขึ้น แล้วกดนิ้วชี้และนิ้วกลางลง มาแตะกับนิ้วโป้งซ้ำ ๆ	นำนิ้วชี้และนิ้วกลางลงมาแตะกับนิ้วหัวแม่มือซ้ำ ๆ

2.2 เทคโนโลยีการตรวจจับและแปลภาษามือที่เกี่ยวข้อง

งานวิจัยด้านการตรวจจับและแปลภาษามือในช่วงหลายปีที่ผ่านมาแบ่งออกได้เป็นสองแนวทางหลัก คือ การตรวจจับด้วยคอมพิวเตอร์วิทัศน์ (Computer Vision) และการตรวจจับด้วยถุงมือติดเซนเซอร์ (Sensor-based Glove) ซึ่งแต่ละแนวทางมีจุดแข็งและข้อจำกัดที่แตกต่างกัน

2.2.1 แนวทางการตรวจจับด้วยคอมพิวเตอร์วิทัศน์

Sánchez-Vicinaiz et al. (2024) พัฒนาระบบจำแนกท่าทางการสะกดนิ้วมือของภาษามือเม็กซิกัน โดยใช้ไลบรารี MediaPipe ร่วมกับโครงข่ายประสาทเทียมแบบคอนโวลูชัน (CNN) ผลการทดลองบนชุดทดสอบพบว่าโมเดลที่ดีที่สุดให้ความแม่นยำ 83.63 เปอร์เซ็นต์ โดยผู้วิจัยระบุว่าการแม่นยำที่ได้ยังผันผวนตามสภาพแสงและมุมกล้องที่ใช้ถ่ายภาพมือ ซึ่งเป็นข้อจำกัดสำคัญของแนวทางที่อาศัยกล้องเป็นอุปกรณ์รับข้อมูลหลัก งานวิจัยกลุ่มนี้แสดงให้เห็นว่าแม้ MediaPipe จะช่วย

ลดภาระการประมวลผลเมื่อเทียบกับการใช้ CNN วิเคราะห์พิกเซลทั้งภาพโดยตรง แต่ระบบยังคงต้องพึ่งพาคุณภาพของภาพที่รับเข้ามาเป็นอย่างมาก

2.2.2 แนวทางการตรวจจับด้วยถุงมือติดเซนเซอร์

Amin, Rizvi, and Hossain (2022) ได้ทบทวนงานวิจัยที่ใช้เซนเซอร์ชนิดต่างๆ ในการพัฒนาถุงมือแปลภาษามืออย่างครอบคลุม และชี้ให้เห็นว่าถุงมือที่ใช้ Flex Sensor ร่วมกับเซนเซอร์วัดความเคลื่อนไหวสามารถให้ความแม่นยำในการจำแนกท่าทางได้ตั้งแต่ประมาณ 80 ถึงมากกว่า 90 เปอร์เซ็นต์ ขึ้นอยู่กับจำนวนเซนเซอร์และไมโครคอนโทรลเลอร์ที่เลือกใช้ อย่างไรก็ตาม งานวิจัยส่วนใหญ่ที่ทบทวนไว้ยังพบข้อจำกัดร่วมกันสองประการ คือ การใช้ไมโครคอนโทรลเลอร์ที่มีสมรรถนะจำกัด เช่น ATmega328 หรือ MSP430 ซึ่งประมวลผลได้ช้า และการใช้ Bluetooth เป็นช่องทางสื่อสารไร้สายซึ่งมีระยะทำการสั้นกว่าการสื่อสารผ่าน Wi-Fi ข้อสังเกตเหล่านี้เป็นส่วนหนึ่งของเหตุผลที่โครงการนี้เลือกใช้ ESP32 ร่วมกับโปรโตคอล UDP ผ่าน Wi-Fi แทน

2.3 ไมโครคอนโทรลเลอร์ ESP32 (ESP32 Microcontroller)

Espressif Systems ได้พัฒนา ESP32 ซึ่งเป็นไมโครคอนโทรลเลอร์ประสิทธิภาพสูงที่รองรับการสื่อสารแบบไร้สายทั้ง Wi-Fi และ Bluetooth ภายในชิปเดียว โดยออกแบบมาเพื่อรองรับการพัฒนาอุปกรณ์ Internet of Things (IoT) ระบบสมาร์ทโฮม ระบบควบคุมอัตโนมัติ และระบบที่ต้องมีการเชื่อมต่อเครือข่ายอินเทอร์เน็ต

ESP32 ได้รับการพัฒนาต่อยอดจาก ESP8266 โดยเพิ่มประสิทธิภาพของหน่วยประมวลผลความสามารถในการประมวลผลแบบหลายงาน (Multitasking) จำนวนขาอินพุตและเอาต์พุต (GPIO) รวมถึงรองรับอุปกรณ์ต่อพ่วง (Peripheral) ได้หลากหลาย เช่น ADC, DAC, PWM, UART, SPI และ I2C ทำให้ได้รับความนิยมอย่างแพร่หลายในงานวิจัยและการพัฒนาระบบสมองกลฝังตัว

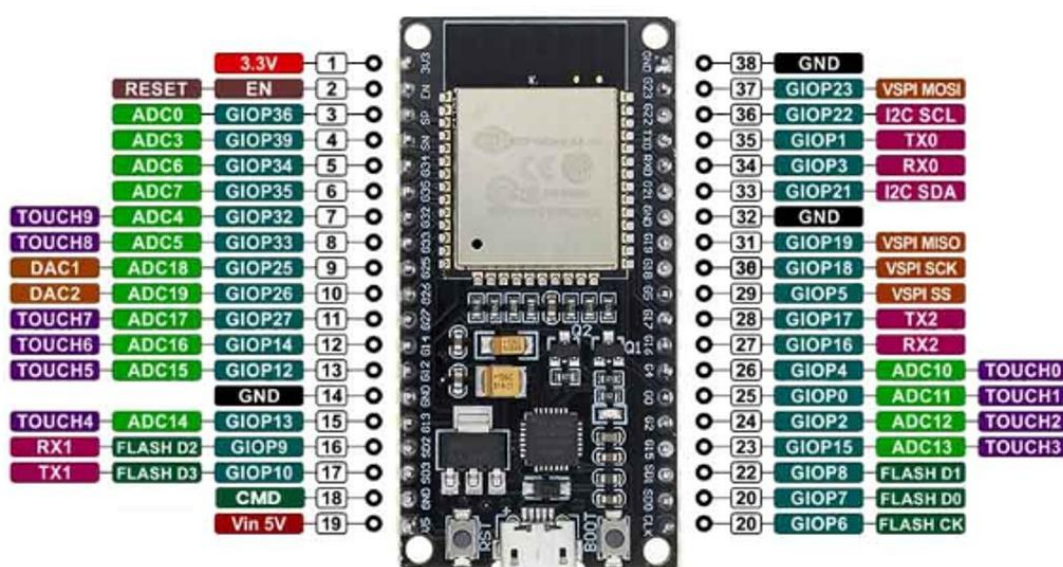
2.3.1 คุณสมบัติทางเทคนิคและสถาปัตยกรรมหลัก

หน่วยประมวลผล (CPU) ใช้สถาปัตยกรรม Xtensa® LX6 (หรือ RISC-V ในรุ่นใหม่) แบบ Dual-core ความเร็วสูงสุด 240 MHz ให้ประสิทธิภาพการประมวลผลสูง (CoreMark ~1080) รองรับงานที่ต้องการความเร็วและการประมวลผลแบบคู่ขนาน

หน่วยความจำ มี SRAM ภายใน 520 KB และ ROM 448 KB รองรับการทำงานที่หน่วยความจำภายนอกแบบ SPI Flash สูงสุด 16 MB และ PSRAM สำหรับรองรับงานที่ใช้หน่วยความจำสูง

การเชื่อมต่อไร้สาย รวม Wi-Fi 802.11 b/g/n (2.4 GHz) และ Bluetooth (Dual-mode/BLE) ไว้ในชิปเดียว พร้อมระบบ RF Front-end ที่ออกแบบมาให้มีกำลังส่งสูงและประหยัดพลังงาน

สถาปัตยกรรมต่อพ่วง มีระบบจัดการ I/O ที่ยืดหยุ่น รองรับสัญญาณดิจิทัลและอนาล็อกจำนวนมาก ช่วยให้สามารถสื่อสารกับเซ็นเซอร์และโมดูลต่างๆ ได้โดยตรง



รูปที่ 2.4 ตำแหน่ง PIN ESP32

จากรูปที่ 2.4 แสดงการจัดเรียงขา GPIO ของ ESP32 แบบ 38 ขา ESP32 รุ่น 38 พิน มีขาเชื่อมต่อ GPIO จำนวนมาก จัดเรียงเป็น 2 แถว แถวละ 19 ขา โดยสามารถแบ่งประเภทการใช้งานตามคุณสมบัติทางไฟฟ้าและฟังก์ชันพิเศษได้ดังนี้

ขาจ่ายไฟ ประกอบด้วยขาแรงดัน 3.3 โวลต์ (3.3V) ได้แก่ ขาที่ 1, ขาแรงดันไฟเข้า Vin (5V) ได้แก่ ขาที่ 19 และขาราวด์ (GND) ได้แก่ ขาที่ 14, 32 และ 38

ขาสื่อสารแบบ I2C ประกอบด้วย GPIO 21 ทำหน้าที่เป็น SDA และ GPIO 22 ทำหน้าที่เป็น SCL ใช้สำหรับการสื่อสารกับอุปกรณ์ภายนอก เช่น จอแสดงผลหรือเซ็นเซอร์ต่างๆ

ขาสื่อสารแบบ SPI (VSPI) ประกอบด้วย GPIO 23 (MOSI), GPIO 19 (MISO), GPIO 18 (SCK) และ GPIO 5 (SS) ใช้สำหรับการสื่อสารความเร็วสูงกับโมดูลอุปกรณ์ต่อพ่วง

ขาสื่อสารแบบ UART มีให้ใช้งานหลายพอร์ต ได้แก่ UART0 (GPIO 1, 3), UART1 (GPIO 9, 10 - เชื่อมต่อ Flash), และ UART2 (GPIO 16, 17) ใช้สำหรับการสื่อสารแบบอนุกรมและการดีบักระบบ

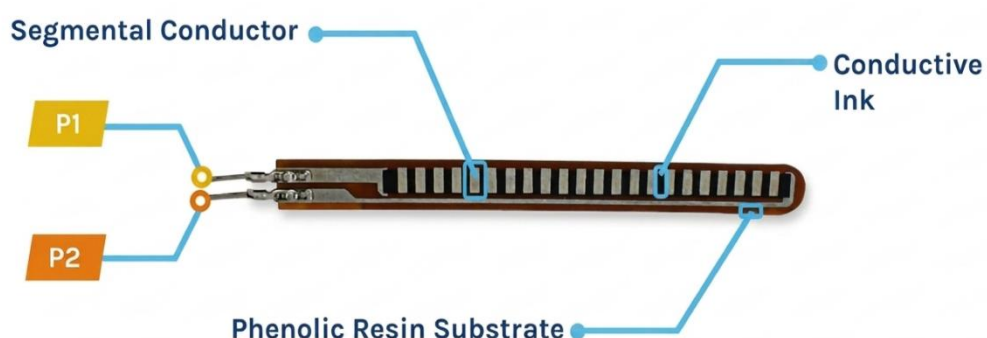
ขา ADC และ DAC ESP32 รองรับการแปลงสัญญาณอนาล็อกเป็นดิจิทัล (ADC) หลายช่อง เช่น GPIO 36, 39, 34, 35, 32, 33, 25, 26, 27, 14, 12, 13, 15, 2, 4 และรองรับสัญญาณดิจิทัลเป็นอนาล็อก (DAC) จำนวน 2 ช่อง ได้แก่ GPIO 25 และ GPIO 26

ขา Touch Sensor ประกอบด้วย GPIO 32, 33, 27, 14, 12, 15, 2, 4 สำหรับรองรับฟังก์ชันการสัมผัสแบบ capacitive

2.4 เซนเซอร์วัดความโค้งงอ (Flex Sensor)

Flex Sensor หรือเซนเซอร์ตรวจจับการงอ เป็นอุปกรณ์ที่ใช้หลักการเปลี่ยนแปลงค่าความต้านทานไฟฟ้าเมื่อตัวเซนเซอร์เกิดการโค้งงอ กล่าวคือในสภาวะปกติที่แผ่นเซนเซอร์วางตัวเรียบตรง ค่าความต้านทานจะอยู่ในระดับต่ำและค่อนข้างคงที่ แต่เมื่อใดที่มีแรงกระทำทำให้เซนเซอร์โค้งงอไปตามมุมต่าง ๆ ค่าความต้านทานจะเปลี่ยนแปลงเพิ่มขึ้นตามระดับความโค้งงอ ด้วยคุณสมบัตินี้เอง จึงสามารถนำ Flex Sensor ไปใช้แปลงการเคลื่อนไหวเชิงกลให้กลายเป็นสัญญาณไฟฟ้าที่วัดค่าได้

2.4.1 โครงสร้างของ Flexsensor



รูปที่ 2.5 โครงสร้างของ Flexsensor

จากรูปที่ 2.5 โครงสร้างของ Flexsensor แบ่งได้ 3 ส่วนคือ

1. Phenolic Resin Substrate (แผ่นฐานฟีนอลิกเรซิน) เป็นวัสดุที่ใช้เป็นโครงสร้างหลักของเซนเซอร์ มีลักษณะเป็นแผ่นบางยาว ทำหน้าที่รองรับชั้นตัวนำที่เคลือบอยู่ด้านบน วัสดุชนิดนี้ถูกเลือกใช้เพราะมีความยืดหยุ่นสูงแต่ยังคงความแข็งแรงในระดับหนึ่ง ทนทานต่อการงอซ้ำไปมาได้
2. Conductive Ink (หมึกตัวนำไฟฟ้า) เคลือบอยู่บนผิวของแผ่นฐาน มองเห็นเป็นแถบสีเงิน เรียงตัววางตลอดความยาวของเซนเซอร์ในภาพ หมึกชนิดนี้จะทำจากคาร์บอนหรือวัสดุตัวนำผสม ทำหน้าที่เป็นตัวกลางในการนำกระแสไฟฟ้า และเป็นส่วนที่ค่าความต้านทานจะเปลี่ยนแปลงเมื่อเกิดการงอ
3. Segmental Conductor (ตัวนำแบบแบ่งช่วง) คือลักษณะการวางแถบตัวนำเป็นช่วง ๆ ต่อเนื่องกันตลอดความยาวของเซนเซอร์ (ดังจะเห็นเป็นแถบขีดสีเงินเรียงกันเป็นระยะในภาพ) การออกแบบลักษณะนี้ช่วยให้ค่าความต้านทานเปลี่ยนแปลงได้อย่างสม่ำเสมอและแปรผันตามองศาการโค้งงอในแต่ละจุดของเซนเซอร์
4. ขั้วต่อสัญญาณ P1 และ P2 เป็นจุดเชื่อมต่อสายไฟสองเส้นที่ปลายด้านหนึ่งของเซนเซอร์ ใช้สำหรับต่อเข้ากับวงจรภายนอกเพื่ออ่านค่าความต้านทานที่เปลี่ยนแปลงไป โดย P1 และ P2 ทำหน้าที่เป็นขั้วบวกและขั้วลบ (หรือขั้วสัญญาณทั้งสองด้าน) ในการวัดค่าความต้านทานตลอดความยาวของเซนเซอร์

เมื่อเซนเซอร์ถูกงอ ระยะห่างระหว่างแถบตัวนำแต่ละช่วงบน Segmental Conductor จะเปลี่ยนแปลงไป ทำให้ค่าความต้านทานที่วัดได้ระหว่างขั้ว P1 และ P2 มีค่าสูงขึ้นตามมุมการโค้งงอ ซึ่งเป็นไปตามหลักการที่อธิบายไว้ในสมการ

2.4.2 วงจรแบ่งแรงดัน (Voltage Divider Circuit)

เนื่องจากไมโครคอนโทรลเลอร์ไม่สามารถอ่านค่า ความต้านทาน (R) ที่ Flexsensor ส่งออกมาได้โดยตรง แต่อ่านได้เพียงค่า "แรงดันไฟฟ้า" (V) ดังนั้นจึงจำเป็นต้องใช้วงจรแบ่งแรงดันเพื่อแปลงค่าความต้านทานที่เปลี่ยนไปของ Flex Sensor ให้เป็นแรงดันไฟฟ้าที่เปลี่ยนไปตามสมการที่ 1

$$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2} \quad (1)$$

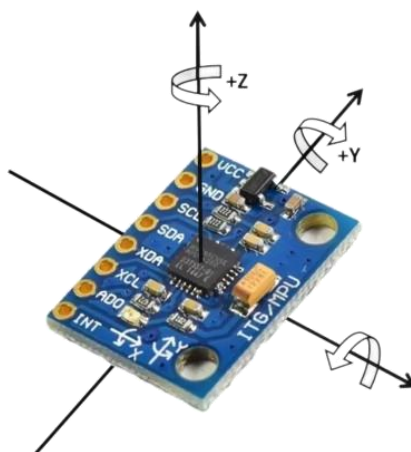
โดยที่

- V_{in} คือ แรงดันไฟเข้า (3.3V จาก ESP32)
- R_1 คือ ค่าความต้านทานของ Flex Sensor (แปรผันตามการงอ)

- R_2 คือ ตัวต้านทานคงที่ (Fixed Resistor) ที่นำมาต่อร่วม
- V_{out} คือ แรงดันไฟฟ้าออกที่ส่งเข้าขา Analog ของ ESP32

2.5 เซนเซอร์วัดความเร่งและองศาการหมุน MPU6050

MPU6050 เป็นอุปกรณ์ตรวจจับความเคลื่อนไหวเชิงเฉื่อย (Inertial Measurement Unit: IMU) ที่ผลิตโดยบริษัท InvenSense ออกแบบให้รวมเซนเซอร์สองประเภทไว้ในวงจรรวม (Integrated Circuit) เดียวกัน ได้แก่ เซนเซอร์วัดความเร่ง 3 แกน (3-Axis Accelerometer) และเซนเซอร์วัดความเร็วเชิงมุม 3 แกน (3-Axis Gyroscope) ทำให้สามารถตรวจจับลักษณะการเคลื่อนไหวเชิงกายภาพของวัตถุในสามมิติได้พร้อมกันในชิปเดียว โดยส่งข้อมูลผ่านโปรโตคอลการสื่อสารแบบบัสสองสาย (I2C Bus) ที่แรงดัน 3.3 โวลต์ ดังรูปที่ 2.6



รูปที่ 2.6 ทิศทางการหมุนของ MPU6050

2.5.1 เซนเซอร์วัดความเร่ง (Accelerometer)

เซนเซอร์วัดความเร่งภายใน MPU6050 ทำงานบนหลักการของระบบไมโครอิเล็กทรอนิกส์ (Micro-Electro-Mechanical Systems: MEMS) โดยมีมวลทดสอบขนาดเล็กมากแขวนลอยอยู่บนโครงสร้างสปริงขนาดจิ๋ว เมื่ออุปกรณ์เกิดการเร่งหรือเบรก มวลทดสอบจะเคลื่อนที่สัมผัสกับโครงสร้าง ส่งผลให้ระยะห่างระหว่างแผ่นตัวเก็บประจุเปลี่ยนไป วงจรภายในจะแปลงความเปลี่ยนแปลงของความจุไฟฟ้านั้นให้กลายเป็นสัญญาณแรงดันไฟฟ้าดิจิทัล โดยวัดผลในแนวแกนพิคัดทั้งสามแกน ดังนี้

- แกน X วัดความเร่งในแนวซ้าย-ขวา
- แกน Y วัดความเร่งในแนวหน้า-หลัง
- แกน Z วัดความเร่งในแนวขึ้น-ลง (รวมถึงแรงโน้มถ่วงโลกประมาณ 9.81 เมตรต่อวินาทียกกำลังสอง)

2.5.2 การหาค่ามุมเอียงของเซนเซอร์วัดความเร่ง

ในกรณีที่ถ่วงมือหยุดนิ่งโดยไม่มีการเคลื่อนที่ เซนเซอร์ความเร่งจะยังคงตรวจจับแรงโน้มถ่วงโลกได้เสมอในรูปของแรงเร่งคงที่ขนาด 9.81 เมตรต่อวินาทียกกำลังสอง โดยค่าที่ปรากฏบนแต่ละแกนจะแตกต่างกันไปตามมุมและทิศทางที่มือหันอยู่ ดังนั้นจึงสามารถนำค่าความเร่งในแต่ละแกนมาคำนวณหามุมเอียง (Tilt Angle) ของถ่วงมือสวมใส่ได้ โดยอาศัยสมการฟังก์ชันตรีโกณมิติผกผันดังสมการที่ 2 และ 3

$$\theta_{Roll} = \arctan\left(\frac{A_y}{\sqrt{A_x^2 + A_z^2}}\right) \quad (2)$$

$$\theta_{Pitch} = \arctan\left(\frac{-A_y}{\sqrt{A_x^2 + A_z^2}}\right) \quad (3)$$

โดยกำหนดให้ A_x , A_y , A_z คือค่าความเร่งที่วัดได้ในแต่ละแกนหลังจากผ่านกระบวนการปรับเทียบค่า (Calibration) แล้ว และ θ_{Roll} คือ การหมุนเอียงซ้าย-ขวา θ_{Pitch} คือ การก้ม-เงย

2.5.3 เซนเซอร์วัดความเร็วเชิงมุม (Gyroscope)

เซนเซอร์ไจโรสโคปภายใน MPU6050 ทำงานบนหลักการของแรงโคริโอลิส (Coriolis Force) โดยมีโครงสร้างมวลสั่นไหว (Vibrating Mass) ที่ถูกขับเคลื่อนให้สั่นด้วยความถี่คงที่ เมื่อเกิดการหมุนเวียนรอบแกนใดแกนหนึ่ง แรงโคริโอลิสจะเหนี่ยวนำให้โครงสร้างสั่นออกนอกกระนาบ ซึ่งวงจรตรวจจับความจุจะแปลงขนาดของการเบี่ยงเบนนั้นให้กลายเป็นค่าอัตราการหมุน (Angular Rate) ในหน่วยองศาต่อวินาที (°/s) ครอบคลุมสามแกนการหมุน ดังนี้

- แกน X การหมุนพลิกข้อมือซ้าย-ขวา
- แกน Y การหมุนตัวข้อมือขึ้น-ลง

- แกน Z การหมุนเอียงแนวราบซ้าย-ขวา

2.5.4 กระบวนการแปลงการเบี่ยงเบนเป็นค่าดิบ

การเบี่ยงเบนของมวลสั่นไหวจะไปเปลี่ยนระยะห่างระหว่างแผ่นตัวเก็บประจุคู่หนึ่ง ทำให้ค่าความจุไฟฟ้า (Capacitance) สองด้านเกิดความต่างกัน ซึ่งสามารถแสดงความสัมพันธ์ได้ดังสมการที่ 4

$$C = \varepsilon \cdot \frac{A}{d} \quad (4)$$

โดยที่ ε คือค่าคงที่การอนุญาตทางไฟฟ้า (Permittivity), A คือพื้นที่หน้าตัดแผ่นตัวเก็บประจุ และ d คือระยะห่างระหว่างแผ่น เมื่อมวลเบี่ยงเบน ระยะ d ของด้านหนึ่งลดลงและอีกด้านหนึ่งเพิ่มขึ้น ทำให้เกิดผลต่างความจุ (ΔC) ที่แปรผันตามขนาดของการหมุน วงจรตรวจจับผลต่างความจุ (Differential Capacitance Detection Circuit) จะแปลงค่า ΔC ให้เป็นสัญญาณแรงดันไฟฟ้า แอนะล็อก ซึ่งส่งต่อเข้าสู่วงจรแปลงสัญญาณ ADC ความละเอียด 16 บิตที่อยู่ภายในตัวชิป เพื่อแปลงให้กลายเป็นเลขจำนวนเต็ม (Integer) ที่เรียกว่า ค่าดิบ (Raw Value)

2.5.5 การแปลงค่าดิบเป็นหน่วยองศาต่อวินาที

ค่าดิบที่ได้จาก MPU6050 จำเป็นต้องแปลงให้อยู่ในหน่วยทางฟิสิกส์ที่ใช้งานได้จริง คือ องศาต่อวินาที ($^{\circ}/s$) โดยใช้ค่าความไวต่อการตรวจจับ (Sensitivity Scale Factor) ที่กำหนดตามย่านวัดที่เลือกใช้ดังสมการที่ 5

$$\omega(^{\circ}/s) = \frac{Raw\ Value}{Sensitivity\ Scale\ Factor} \quad (5)$$

ค่าที่ได้บ่งบอกว่าในเฟรมนั้นมือกำลังหมุนรอบแกน X ด้วยอัตราสูงสุดประมาณ 144.9 องศาต่อวินาที ซึ่งสอดคล้องกับการวัดข้อมือในระหว่างการโบกมือ โดยค่าชุดนี้จะถูกส่งต่อเข้าสู่แบบจำลองปัญญาประดิษฐ์พร้อมกับค่าจากเซนเซอร์อื่น ๆ เพื่อใช้ในการจำแนกประเภทท่าทาง

2.6 โพรโทคอลการสื่อสารของระบบ

ถูงมือแปลภาษาเมื่อข้างเดียวใช้โพรโทคอลการสื่อสารสองระดับที่มีวัตถุประสงค์แตกต่างกัน ได้แก่ I2C สำหรับให้ ESP32 อ่านค่าจากเซนเซอร์ MPU6050 บนถูงมือ และ UDP สำหรับส่งข้อมูลเซนเซอร์ทั้งหมดออกจากถูงมือไปยังคอมพิวเตอร์ที่ประมวลผลปัญญาประดิษฐ์

2.6.1 I2C (Inter-Integrated Circuit)

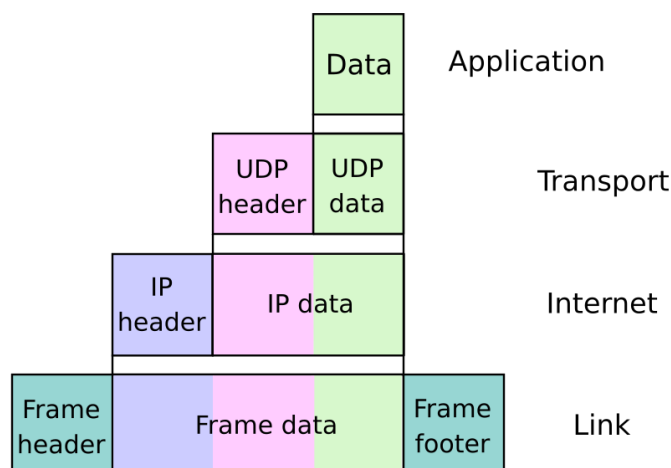
I2C (Inter-Integrated Circuit) เป็นมาตรฐานการสื่อสารแบบอนุกรมสองสายที่ Philips Semiconductors พัฒนาขึ้นสำหรับเชื่อมต่ออุปกรณ์ในระยะสั้นบนแผงวงจรเดียวกัน โดยใช้สายสัญญาณเพียง 2 เส้น คือ SDA (Serial Data) สำหรับรับส่งข้อมูล และ SCL (Serial Clock) สำหรับส่งสัญญาณนาฬิกาเพื่อซิงโครไนซ์

การสื่อสารเป็นแบบ Master-Slave โดย ESP32 ทำหน้าที่เป็น Master ซึ่งเป็นผู้เริ่มต้นการสื่อสารทุกครั้ง ในขณะที่ MPU6050 ทำหน้าที่เป็น Slave ที่มีหมายเลขที่อยู่ (Address) 7 บิตประจำตัว ทำให้สายสัญญาณชุดเดียวกันสามารถรองรับการเชื่อมต่ออุปกรณ์ Slave ได้พร้อมกันสูงสุดถึง 127 ตัว

2.6.2 โพรโทคอลการส่งข้อมูลแบบยูดีพี (UDP Protocol)

โพรโตคอลยูดีพี (User Datagram Protocol: UDP) เป็นโพรโทคอลการสื่อสารในระดับชั้นทรานสปอร์ต (Transport Layer) ของโครงแบบการสื่อสารอินเทอร์เน็ต (TCP/IP Model) ออกแบบมาเพื่อการส่งข้อมูลที่ต้องการความเร็วสูงและยอมรับการสูญหายของข้อมูลบางส่วนได้ในกรณีที่จำเป็น

2.6.3 โครงสร้างแพ็กเก็ตข้อมูล UDP



รูปที่ 2.7 โครงสร้าง UDP

จากรูปที่ 2.7 แสดงกระบวนการ Encapsulation ซึ่งเป็นการห่อหุ้มข้อมูลด้วย Header ที่ละชั้น ขณะที่ข้อมูลเคลื่อนที่ลงมาตามลำดับชั้นของ TCP/IP Model ก่อนจะถูกส่งออกไปยังเครือข่าย โดยสามารถแบ่งออกได้เป็น 4 ขั้นตอน ดังนี้

ขั้นตอนแรก เริ่มต้นที่ Application Layer ซึ่งเป็นจุดกำเนิดของข้อมูลดิบ (Data) จากแอปพลิเคชันที่ใช้งาน เช่น คำสั่งค้นหา DNS หรือข้อมูลเสียงจากการโทรศัพท์ผ่านอินเทอร์เน็ต โดยข้อมูลในขั้นตอนนี้ยังไม่มีห่อหุ้ม Header ใด ๆ

ขั้นตอนที่สอง คือ Transport Layer ระบบจะนำ UDP Header ขนาด 8 ไบต์ มาเพิ่มไว้ด้านหน้าของ Data เกิดเป็น UDP Datagram ซึ่งประกอบด้วย UDP Header และ Data

ขั้นตอนที่สาม คือ Internet Layer UDP Datagram จากขั้นตอนก่อนหน้าจะถูกมองเป็นข้อมูลชิ้นเดียว เรียกว่า IP Data จากนั้นระบบจะเพิ่ม IP Header เข้าไปด้านหน้าอีกหนึ่งชั้น รวมกันเป็น IP Packet

ขั้นตอนสุดท้าย คือ Link Layer IP Packet ทั้งหมดจะถูกบรรจุในส่วนที่เรียกว่า Frame Data โดยมี Frame Header ปิดไว้ด้านหน้า และ Frame Footer ปิดท้ายด้านหลัง ผลลัพธ์ที่ได้คือ Frame ซึ่งเป็นรูปแบบสุดท้ายของข้อมูลก่อนที่จะถูกแปลงเป็นสัญญาณส่งผ่านสายเคเบิลหรือคลื่นสัญญาณ

2.6.4 ข้อดี

- ความเร็วสูง เพราะ overhead น้อย
- Latency (ความหน่วง) ต่ำ เหมาะกับงาน real-time
- เหมาะกับการส่งข้อมูลแบบ broadcast/multicast

2.6.5 ความแตกต่างระหว่าง UDP และ TCP

ความแตกต่างของ UDP (User Datagram Protocol) และ TCP (Transmission Control Protocol) เปรียบเทียบคุณสมบัติที่สำคัญระหว่าง UDP กับโปรโตคอล TCP มีรายละเอียดข้อแตกต่างดังนี้

- การยืนยันการรับข้อมูล (Handshake) โปรโตคอล UDP จะไม่มีกระบวนการยืนยันก่อนเริ่มส่งข้อมูล ในขณะที่ TCP จะต้องมีการยืนยันการรับส่งข้อมูลแบบ 3-Way Handshake เสมอ
- ขนาดส่วนหัวแพ็กเก็ต UDP มีขนาดส่วนหัว (Header) ที่เล็กมากเพียง 8 ไบต์ ซึ่งต่างจาก TCP ที่มีขนาดส่วนหัวใหญ่กว่า โดยอยู่ระหว่าง 20-60 ไบต์
- ความหน่วงเวลา (Latency) ด้วยโครงสร้างที่เรียบง่ายของ UDP ทำให้มีความหน่วงเวลาที่ต่ำมากเมื่อเทียบกับ TCP ที่มีความหน่วงเวลาอยู่ในระดับปานกลางถึงสูงจากกระบวนการตรวจสอบข้อมูล
- การรับประกันการส่งข้อมูล โปรโตคอล UDP จะไม่มีกลไกในการรับประกันว่าข้อมูลที่ส่งไปจะถึงผู้รับอย่างครบถ้วนสมบูรณ์หรือไม่ แต่ TCP จะมีกลไกที่คอยรับประกันการส่งข้อมูลให้ถึงปลายทางเสมอ
- ลำดับการมาถึงของแพ็กเก็ต UDP จะไม่รับประกันลำดับการมาถึงของแพ็กเก็ตข้อมูล (ข้อมูลอาจไปถึงไม่เรียงลำดับ) ในขณะที่ TCP จะรับประกันลำดับการมาถึงของข้อมูลให้ถูกต้องตามที่ส่งไป

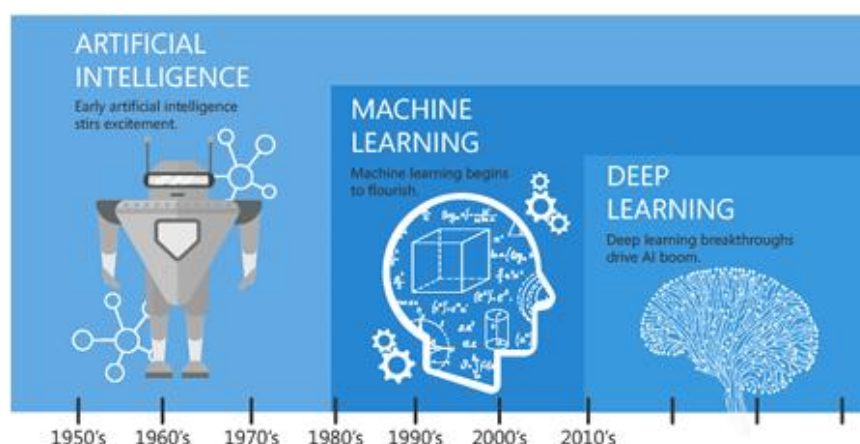
2.7 ปัญญาประดิษฐ์ (Artificial Intelligence : AI)

ในยุคนี้ เครื่องคอมพิวเตอร์ส่วนใหญ่จะมีหน้าที่ทำงาน และประมวลผลโปรแกรมของระบบงานตามที่มนุษย์ต้องการ โดยมีมนุษย์เป็นผู้ควบคุม ป้อนคำสั่ง และรอผลลัพธ์จากเครื่องคอมพิวเตอร์ การตัดสินใจไม่สำคัญ ยังอาจขยายพื้นฐานการทำงานระหว่างมนุษย์ ทำให้มีแนวคิดที่จะพัฒนาให้เครื่องคอมพิวเตอร์มีประสิทธิภาพสูงขึ้น โดยเพื่อทำงานอำนวยความสะดวกต่าง ๆ ให้กับมนุษย์ ซึ่งไม่ใช่เพียงการให้มนุษย์ป้อนข้อมูลคำสั่งเท่านั้น จึงเป็นจุดเริ่มต้นของแนวเครื่องคอมพิวเตอร์ให้มีความคิด มีหลักการที่เหตุผลคล้ายคลึงกับสมองมนุษย์ เพื่อให้เครื่องคอมพิวเตอร์สามารถวิเคราะห์ข้อมูลเพื่อ

ค้นหาคำตอบที่ใกล้เคียงที่สุด จุดประสงค์ในการทำงานด้วยเหตุนี้จึงมีแนวการพัฒนาแนวระบบเครื่องคอมพิวเตอร์ ให้มีการทำงานเหมือนสมองมนุษย์หรือที่เรียกว่า "ปัญญาประดิษฐ์ (Artificial Intelligence)"

ปัญญาประดิษฐ์ หรือเรียกสั้น ๆ ว่า AI (Artificial Intelligence) เป็นแนวคิดเพื่อการพัฒนาเครื่องคอมพิวเตอร์รูปแบบใหม่ ซึ่งทำให้เครื่องคอมพิวเตอร์เกิดกระบวนการคิดแล้วตัดสินใจเหมือนมนุษย์ โดยอาศัยหลักการทางวิทยาศาสตร์ การคิดค้นในเชิงหรือหลักการที่มาจากเหตุผลสมองมนุษย์ เพื่อนำไปใช้ในการพัฒนาศักยภาพของเครื่องคอมพิวเตอร์ให้สามารถตอบสนองความต้องการด้านการทำงานในเครื่องจักรอัจฉริยะได้อย่างแท้จริงในโลกอนาคต โดยสร้างให้เครื่องคอมพิวเตอร์ที่มีความสามารถพิจารณา วิเคราะห์ จัดเรียง คำนวณ แก้ปัญหา ตัดสินใจ และเรียนรู้ได้ด้วยตนเอง ส่งผลให้เครื่องคอมพิวเตอร์มีความสามารถต่าง ๆ ที่สามารถทำงานในระบบที่มีความซับซ้อนโดยมีประสิทธิภาพโดยไม่ต้องพึ่งแรงงานจากมนุษย์

ปัญญาประดิษฐ์ (Artificial Intelligence) เป็นศาสตร์แขนงหนึ่งของวิทยาการคอมพิวเตอร์ที่ต้องการพัฒนาเครื่องจักรกลต่าง ๆ เช่น เครื่องคอมพิวเตอร์ หรือหุ่นยนต์ เป็นต้น ให้มีความสามารถทางด้านความคิด การตัดสินใจ และพฤติกรรมที่คล้ายกับมนุษย์มากที่สุด เพื่อการแก้ปัญหาหรือหาเหตุผลจากข้อมูล หรือองค์ความรู้ที่มีอยู่แล้วนำมาวิเคราะห์ และแปลความหมายให้ได้ผลสรุปหรือผลลัพธ์ที่ถูกต้องและมีความใกล้เคียงกับการตัดสินใจด้วยมนุษย์ ปัญญาประดิษฐ์เป็นศาสตร์ที่ช่วยสนับสนุนระบบงานที่อาศัยการทำงานของเครื่องจักร หรือเครื่องคอมพิวเตอร์ได้เป็นอย่างดี เมื่อมีการพัฒนาปัญญาประดิษฐ์ที่มีประสิทธิภาพการทำงานต่าง ๆ จะเกิดข้อผิดพลาดน้อย มีการแก้ไขปัญหาและตัดสินใจที่เร็วกว่าการใช้สมองมนุษย์ เนื่องจากมนุษย์ยังต้องอาศัยพื้นฐานทางกายภาพหรือร่างกาย ทำให้การทำงานอาจเกิดความผิดพลาดขึ้นได้เสมอ ดังแสดงในรูปที่ 2.8



รูปที่ 2.8 เปรียบเทียบ AI, Machine Learning, Deep Learning

จำแนกปัญญาประดิษฐ์ออกเป็น 4 ลักษณะ โดยสามารถสรุปได้ดังนี้

1. ระบบความคิดที่เลียนแบบมนุษย์

ปัญญาประดิษฐ์ที่มีคุณสมบัติที่พัฒนาเพิ่ม ทำให้องค์กรมีลักษณะทางการคิดที่คล้ายคลึงกับมนุษย์ ซึ่งมีการพัฒนาในตัวเท่านั้น ซึ่งมีการยืดหยุ่นความสำคัญในการพัฒนาองค์กรให้มีลักษณะทางการคิดที่คล้ายคลึงกับมนุษย์ มีการพัฒนาให้ได้มาตรฐานที่ตั้งไว้ตามพฤติกรรมความคิดในการแก้ปัญหาที่เกิดขึ้นจากการทำงานที่ได้รับมอบหมายจากมนุษย์ การกระทำในการเลียนแบบความคิดของมนุษย์ และการยกระดับพฤติกรรมของปัญญาประดิษฐ์ให้ตรงกับการแก้ปัญหาที่ซับซ้อนที่มนุษย์ต้องการให้ได้มาตรฐานที่สุดเท่าที่จะทำได้

2. ระบบการกระทำที่เหมือนมนุษย์

การกระทำที่เกิดขึ้นจากปัญญาประดิษฐ์ ซึ่งอาจพัฒนาให้มีระบบการดำเนินการที่เลียนแบบจากมนุษย์ ส่งผลให้สามารถออกแบบกระบวนการทางการทำงานได้ด้วยระบบการกระทำที่เหมือนมนุษย์ สามารถเลียนแบบพฤติกรรมได้ในระดับการเลียนแบบพฤติกรรมที่ต้องการหรือการดำเนินงานที่มีการจัดกระบวนการพฤติกรรมที่คล้ายมนุษย์ได้เป็นอย่างดี การออกแบบระบบการกระทำที่ใช้ระบบทางความคิดอย่างมีระบบและสามารถปรับใช้ให้มีประสิทธิภาพเป็นอย่างดี การกระทำเหล่านี้ส่งผลให้ปัญญาประดิษฐ์สามารถแสดงบทบาทการทำงานได้เช่นเดียวกับมนุษย์ในด้านกระบวนการดำเนินงาน

3. ระบบความคิดอย่างมีเหตุผล

ระบบความคิดที่เกิดจากปัญญาประดิษฐ์อย่างมีเหตุผลและพฤติกรรมที่คล้ายคลึงกับการทำงานของเหตุผล โดยสร้างเป็นพฤติกรรมแบบระบบปัญญาประดิษฐ์ ที่สามารถดำเนินงานได้ในระดับกระบวนการทางการคิดที่มีการจัดการเป็นอย่างดี ในการกำหนดมาตรการที่จะมีการเลียนแบบกระบวนการต่าง ๆ ที่เป็นทางการ เพื่อให้สามารถตัดสินใจได้บนฐานของการใช้เหตุผลและผลลัพธ์ที่ทำงานได้อย่างมีมาตรฐานตามแผน การพัฒนากระบวนการทางความคิดให้มีแบบแผนที่ใช้ในกระบวนการทำงานร่วมกับระบบต่าง ๆ ตามมาตรฐานการทำงานได้ในแนวที่ต้องการ

4. ระบบการกระทำอย่างมีเหตุผล

ด้วยระบบการกระทำที่มีเหตุผลของปัญญาประดิษฐ์ ส่งผลให้มีการแสดงผลออกมาอย่างมีเหตุผล และยังสามารถเลียนแบบการแสดงผลที่มีความเหมาะสมกับสถานการณ์ต่าง ๆ อย่างดี ด้วยการประเมินผลที่เกิดจากการใช้ปัญญาประดิษฐ์ในเชิงเหตุผล การแสดงผลออกตามกรอบความคิดในระบบปัญญาประดิษฐ์ที่เกิดขึ้นในระดับการควบคุมการตัดสินใจในการเลียนแบบพฤติกรรมได้ในระดับการแสดงผลเพื่อพัฒนาระบบปัญญาประดิษฐ์ให้ทำหน้าที่ในระดับที่ตั้งเป้าหมาย

ปัญญาประดิษฐ์สามารถแบ่งตามฟังก์ชันการทำงานได้เป็น 4 ประเภทหลัก ๆ ดังนี้

1. Reactive Machines

เป็นระบบ AI ที่ไม่สามารถเรียนรู้ได้ด้วยตนเอง มีความสามารถจำกัด ไม่มีหน่วยความจำเป็นของตัวเอง ไม่สามารถดึงข้อมูลเก่ามาพัฒนาในการตัดสินใจให้ดีขึ้นได้ ไม่สามารถสร้างการอนุมานจากข้อมูล เพื่อประเมินการกระทำในอนาคตได้ ทำได้แค่ปฏิกิริยาโต้ตอบกับสถานการณ์ตรงหน้าเท่านั้น เช่น การเล่นเกมกรุก เป็นต้น

2. Limited Theory

เป็นระบบ AI ที่ตรงกันข้ามกับประเภท Reactive Machines โดยสิ้นเชิง เพราะสามารถเรียนรู้ด้วยตนเองได้ มีหน่วยความจำ แต่มีจำกัด สามารถดึงข้อมูลเก่ามาตัดสินใจ หรือแก้ปัญหาในอนาคตได้ ซึ่งยังมีข้อมูลมาก ระดับความถูกต้องแม่นยำก็จะยิ่งสูงขึ้นเรื่อย ๆ เช่น รถยนต์ AI ใช้เซ็นเซอร์เพื่อระบุผู้คนที่เดินข้ามถนน สัญญาณไฟจราจร และอื่น ๆ เพื่อให้การตัดสินใจในการขับดีขึ้น และป้องกันการเกิดอุบัติเหตุได้

3. Theory of Mind

การทำงานของ AI ประเภทนี้ คือ สามารถเข้าใจอารมณ์ ความรู้สึก วัฒนธรรม และความเชื่อต่าง ๆ ของมนุษย์ได้ เหมือนตัวอย่าง AI ที่เกิดขึ้นในภาพยนตร์เรื่อง Her (2013) AI ที่สามารถรับรู้ความรู้สึก และพูดคุยได้ ซึ่งในปัจจุบันยังไม่สามารถพัฒนาขึ้นมาได้จริง และยังต้องใช้เวลาอีกมากในการค้นคว้าและพัฒนา หากในอนาคต AI ประเภทนี้พัฒนาได้สำเร็จ AI ประเภทนี้อาจสามารถทำนายพฤติกรรมล่วงหน้าได้ เหมือนการจำลองความสัมพันธ์ระหว่างมนุษย์ ผ่านการทำความเข้าใจความรู้สึกที่เกิดขึ้นตรงหน้า

4. Self-Awareness

เป็นระบบ AI ขั้นสูงสุด สามารถมีอารมณ์ ความรู้สึก ความเชื่อ และมีความต้องการเป็นของตัวเอง รวมถึงสามารถคิดตัดสินใจ เลือก และกระทำสิ่งต่าง ๆ ได้ด้วยตนเองทั้งหมด เช่นตัวอย่าง AI ในภาพยนตร์เรื่อง Chappie (2015) AI ที่สามารถเรียนรู้ความรัก และเรียนรู้ในการมีชีวิตอยู่ ซึ่งในปัจจุบันก็ยังไม่สามารถพัฒนาขึ้นมาได้เช่นเดียวกับประเภท Theory of Mind ซึ่ง AI ประเภทนี้มีความก้าวหน้ามากกว่าทฤษฎีความคิดของ AI เป็นอย่างมาก ตั้งแต่การทำความเข้าใจอารมณ์ ไปสู่การตระหนักรู้ในสถานะของตนเอง การคาดการณ์ความรู้สึกของผู้อื่นได้ และมนุษย์เรายังห่างไกลจาก AI ประเภทนี้มาก

เทคโนโลยี AI สามารถใช้ ML และเครือข่าย Deep Learning เพื่อแก้ไขปัญหาที่ซับซ้อนด้วยความฉลาดที่คล้ายคลึงกับมนุษย์ AI สามารถประมวลผลข้อมูลได้ในวงกว้าง ไม่ว่าจะเป็นการค้นหา

รูปแบบ การระบุข้อมูล และการให้คำตอบ สามารถใช้ AI เพื่อแก้ไขปัญหาในหลากหลายด้าน เช่น การตรวจจับการฉ้อโกง การวินิจฉัยทางการแพทย์ และการวิเคราะห์ธุรกิจ

2.7.1 การใช้งานเทคโนโลยี AI

ในปัจจุบัน ระบบปัญญาประดิษฐ์ได้ถูกนำมาประยุกต์ใช้ในหลากหลายธุรกิจและอุตสาหกรรม ไม่ว่าจะเป็นภาคการผลิต โรงงาน โรงพยาบาล ตลอดจนภาคการบริการ อีกทั้งยังมีการนำ AI มาใช้งานในฟังก์ชันต่าง ๆ ขององค์กร ทำให้การดำเนินงานในหลายส่วนเป็นไปอย่างมีประสิทธิภาพและสะดวกยิ่งขึ้น AI จึงเป็นเครื่องมือสำคัญที่มีบทบาทในการยกระดับและพัฒนาศักยภาพของธุรกิจและอุตสาหกรรมในหลายมิติ AI ช่วยเสริมสร้างธุรกิจและอุตสาหกรรมตัวอย่างธุรกิจ ดังรูปที่ 2.9



รูปที่ 2.9 การนำ AI มาใช้ในด้านอุตสาหกรรม

เทคโนโลยี AI มีระบบการทำงานในเรื่องของการรวบรวมและวิเคราะห์ข้อมูลจำนวนมากได้อย่างดีเยี่ยม และเกิดข้อผิดพลาดได้น้อยมาก ดังนั้น ในด้านการวิจัยการพัฒนาต่าง ๆ จะทำให้ได้ข้อมูลที่มีประสิทธิภาพ และมีความถูกต้องสูง ไม่ว่าจะในด้านอุตสาหกรรมใด ๆ จะยังพัฒนาให้สินค้านั้นเกิดผลสำเร็จได้อย่างมาก

2.7.2 ประโยชน์ของปัญญาประดิษฐ์

การนำปัญญาประดิษฐ์มาใช้ในระบบต่าง ๆ ขององค์กร นอกจากจะช่วยเพิ่มประสิทธิภาพของระบบงานที่ต้องการได้แล้ว ปัญญาประดิษฐ์ยังส่งผลต่อการทำงานในการรวมขององค์กรได้เป็นอย่างดี และยังสร้างประโยชน์ในด้านต่าง ๆ อีกมากมาย ได้แก่

- ช่วยให้กระบวนการแก้ไขปัญหาทำได้อย่างรวดเร็ว และมีความสอดคล้องกันมากขึ้น
- ช่วยแก้ไขปัญหาต่าง ๆ ในกรณีที่ยากเกินความสามารถของมนุษย์และเครื่องคอมพิวเตอร์ทั่วไป
- รองรับข้อมูลและองค์ความรู้จำนวนมากได้
- สามารถจัดเก็บข้อมูล และองค์ความรู้ได้เป็นอันมาก
- การเผยแพร่องค์ความรู้ภายในองค์กรทำได้ง่ายและทั่วถึง
- การวิเคราะห์ปัญหาทำได้ละเอียดรอบคอบ โดยเฉพาะปัญญาประดิษฐ์ที่ถูกพัฒนาใช้งานเฉพาะด้าน
- มีความยืดหยุ่นต่อการคาดเดาสถานะได้อย่างมีประสิทธิภาพ
- การควบคุมเครื่องจักรกลและเครื่องคอมพิวเตอร์ทำได้ง่ายขึ้น

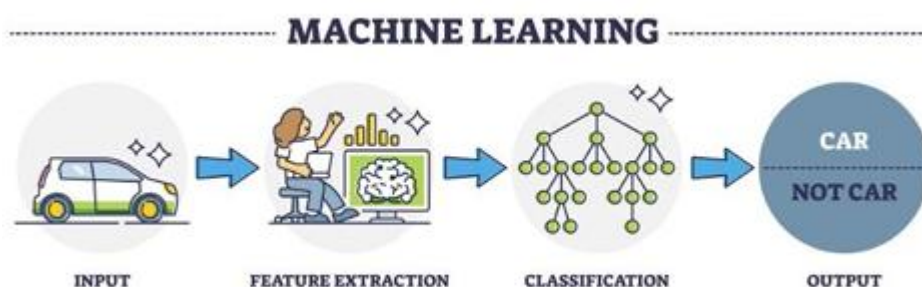
คุณสมบัติ ข้อจำกัด และประโยชน์ของปัญญาประดิษฐ์เป็นอีกสิ่งที่มีความสำคัญที่ทำให้เกิดประสิทธิภาพการทำงานของปัญญาประดิษฐ์มีความสำคัญต่อระบบงานในองค์กรรวมกัน เพื่อให้เกิดเป็นนวัตกรรมที่มีคุณค่า และคุ้มค่าอย่างมากต่อการลงทุนในด้านทรัพยากรความรู้ จะช่วยให้ทรัพยากรที่มีอยู่อย่างจำกัดสามารถใช้งานได้อย่างเต็มที่ และเกิดผลที่ติดต่อกันในที่สุด อย่างไรก็ตาม ปัญญาประดิษฐ์ไม่สามารถทำงานแทนมนุษย์ได้ และจำเป็นต้องประยุกต์ใช้ในการร่วมกันกับมนุษย์จึงจะได้ผลอย่างมีประสิทธิภาพ

2.8 การเรียนรู้ของเครื่อง (Machine Learning : ML)

การเรียนรู้ของเครื่อง (Machine Learning) คือกระบวนการที่ทำให้คอมพิวเตอร์สามารถเรียนรู้และพัฒนาประสิทธิภาพการทำงานได้ด้วยตนเองจากข้อมูลและสภาพแวดล้อมที่ได้รับ โดยไม่จำเป็นต้องมีการกำกับจากมนุษย์หรือการเขียนโปรแกรมเพิ่มเติม เมื่อมีข้อมูลรูปแบบใหม่ในอนาคต ระบบสามารถประมวลผลและปรับตัวได้โดยอัตโนมัติ โดยไม่ต้องมีการแทรกแซงหรือการปรับปรุงโปรแกรมจากมนุษย์ ระบบเรียนรู้จากการวิเคราะห์ข้อมูล ทำให้สามารถตัดสินใจและตอบสนองต่อสถานการณ์ที่ซับซ้อนได้อย่างมีประสิทธิภาพ

Machine Learning ทำงานในลักษณะเดียวกันกับการเรียนรู้ของมนุษย์ โดยเริ่มจากการป้อนชุดข้อมูลพื้นฐานและคำสั่งต่าง ๆ เพื่อให้คอมพิวเตอร์สามารถเรียนรู้ และจำแนกวัตถุ บุคคล หรือสิ่งของ

ได้อย่างมีประสิทธิภาพ เพื่อให้ได้ผลลัพธ์ที่มีความแม่นยำมากขึ้น โปรแกรมเมอร์จะป้อนชุดข้อมูลใหม่ๆ อย่างต่อเนื่องและฝึกฝนระบบอย่างสม่ำเสมอ กระบวนการนี้ช่วยให้ระบบเกิดการพัฒนาและสามารถปรับปรุงการตัดสินใจหรือการคาดการณ์ด้วยตัวเองได้ดังรูปที่ 2.10



รูปที่ 2.10 การเรียนรู้ของ Machine Learning

1. Supervised Learning คือการที่คอมพิวเตอร์เรียนรู้จากข้อมูลตัวอย่างที่มีการระบุคำตอบล่วงหน้า โดยคอมพิวเตอร์จะได้รับข้อมูลอินพุตและเรียนรู้ผ่านการวิเคราะห์คุณสมบัติ เพื่อนำไปใช้ในการจำแนกประเภท ยกตัวอย่างเช่น การป้อนรูปภาพปากกาให้คอมพิวเตอร์ ซึ่งเบื้องต้นคอมพิวเตอร์ไม่ทราบว่าเป็นรูปปากกา แต่เมื่อได้รับการสอนให้รู้จักคุณลักษณะเฉพาะของปากกา เช่น มีปลายด้ามและใช้หมึก คอมพิวเตอร์จะสามารถแยกแยะรูปปากกาได้ในอนาคต

2. Unsupervised Learning เป็นการเรียนรู้ที่คอมพิวเตอร์วิเคราะห์ข้อมูลโดยไม่มีคำตอบล่วงหน้า มนุษย์จะป้อนข้อมูลและปล่อยให้คอมพิวเตอร์ค้นหารูปแบบและโครงสร้างจากข้อมูลนั้นเอง เช่น การป้อนรูปภาพปากกาเข้าไปโดยไม่บอกว่าเป็นปากกา คอมพิวเตอร์จะวิเคราะห์คุณสมบัติของรูปภาพและใช้วิธีการจัดกลุ่ม เพื่อจัดประเภทวัตถุที่มีลักษณะคล้ายกัน เช่น อาจจัดปากกากับปากกาไฮไลต์ในกลุ่มเดียวกัน เนื่องจากมีคุณสมบัติคล้ายคลึงกัน

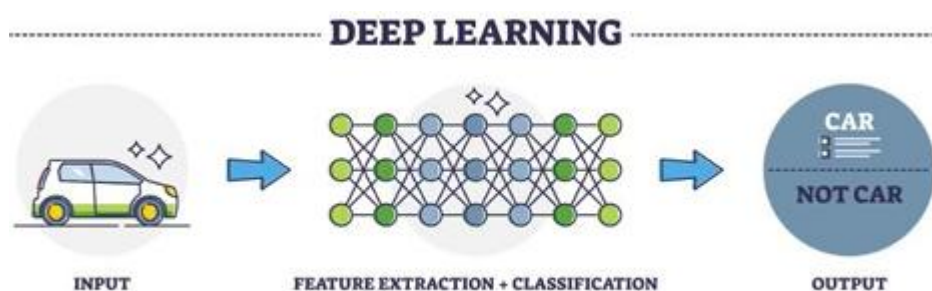
3. Reinforcement Learning เป็นการเรียนรู้ที่เกิดจากการโต้ตอบระหว่างผู้เรียนรู้และสิ่งแวดล้อม โดยผู้เรียนรู้จะเลือกกระทำในสถานการณ์ต่าง ๆ ผ่านการลองผิดลองถูกเพื่อหาวิธีที่ให้ผลลัพธ์ที่ดีที่สุด เช่น ในเกม AlphaGo ที่ระบบเรียนรู้ว่าควรเดินหมากอย่างไรเพื่อครอบครองพื้นที่มากกว่าคู่ต่อสู้ โดยมีเงื่อนไขและรางวัลกำหนดไว้

2.9 การเรียนรู้เชิงลึก (Deep Learning : DL)

Deep Learning (DL) คือเทคนิคการเรียนรู้ของเครื่องที่มีพื้นฐานจากโครงข่ายประสาทเทียม (Neural Networks) ซึ่งประกอบด้วยหลายชั้น ทำให้สามารถประมวลผลข้อมูลที่ซับซ้อนมากขึ้น เช่น รูปภาพ เสียง หรือข้อความ โดยการทำงานของโครงข่ายนี้เริ่มต้นที่ชั้นแรกหรือ Input Layer ซึ่งทำ

หน้าที่รับข้อมูลจากภายนอก แล้วส่งข้อมูลผ่านไปยังชั้นซ่อน (Hidden Layers) ที่ทำการประมวลผลข้อมูลในหลายขั้นตอน เพื่อตีความหรือลดรูปข้อมูลให้อยู่ในรูปแบบที่เครื่องสามารถเข้าใจได้ดีขึ้น

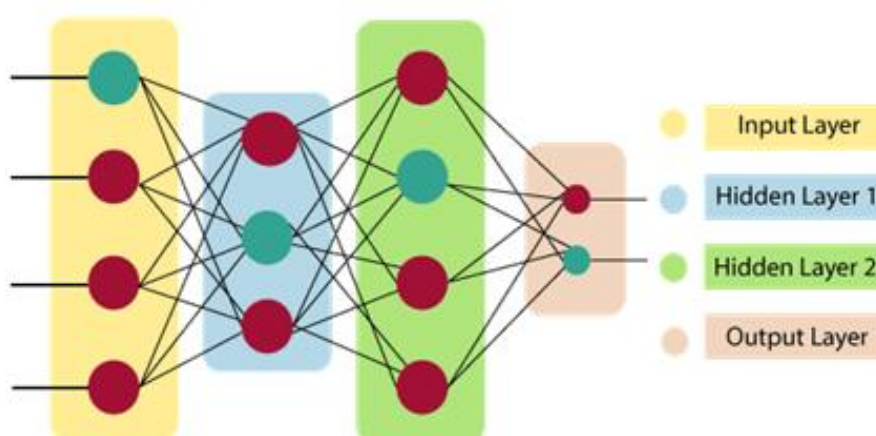
ความสามารถของ Deep Learning ที่ทำให้มันมีประสิทธิภาพมากคือการไม่ต้องการให้ผู้พัฒนา กำหนดคุณสมบัติของข้อมูลล่วงหน้า โครงข่ายสามารถเรียนรู้คุณสมบัติที่สำคัญได้เองเมื่อมีข้อมูลเพียงพอ ทำให้เหมาะกับการใช้งานในหลายด้าน เช่น การรู้จำภาพ (Image Recognition) การแปลภาษา (Language Translation) และการประมวลผลเสียง (Speech Recognition) ดังรูปที่ 2.11



รูปที่ 2.11 การเรียนรู้ของ Deep Learning

2.10 ระบบโครงข่ายประสาทเทียม (Artificial Neural Network : ANN)

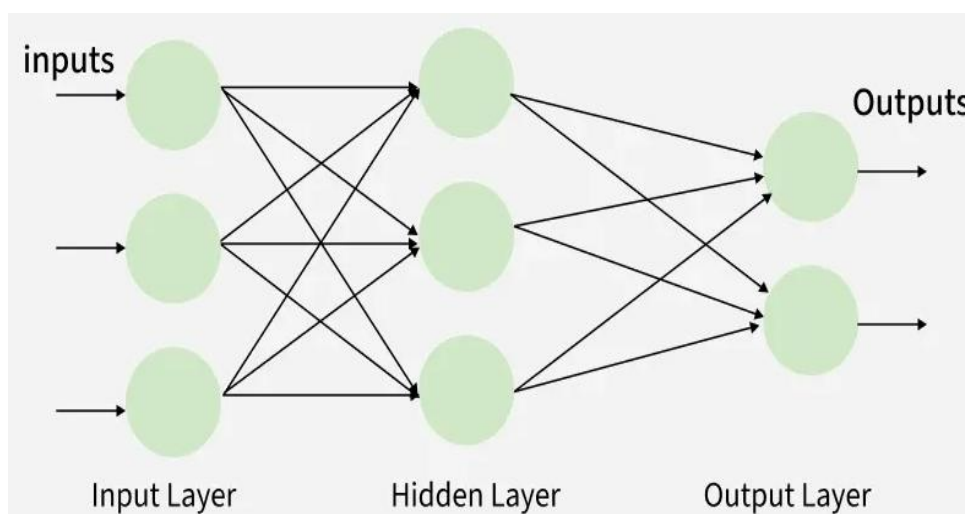
โครงข่ายประสาทเทียมเป็นโมเดลทางคณิตศาสตร์ว่าด้วยการคำนวณแบบคอนเนคชันนิสต์ (Connectionist) ที่ใช้สำหรับประมวลผลเพื่อจำลองการทำงานของเครือข่ายประสาทในสมองมนุษย์ที่สามารถสร้างความสามารถในการเรียนรู้แบบจดจำรูปแบบ (Pattern Recognition) โดยแนวคิดในการเรียนรู้แบบโครงข่ายประสาทของมนุษย์ เริ่มมาจากการศึกษาโครงข่ายไฟฟ้าชีวภาพในสมอง (Bioelectric Network) สำหรับการรับรู้สิ่งต่าง ๆ ซึ่งการที่มนุษย์จะรับรู้สิ่งต่าง ๆ ได้นั้น จะต้องมีการรับรู้ที่เปรียบเสมือน Input Layer สำหรับข้อมูลที่เข้ามา จากนั้นจะใช้ Soma ที่เปรียบเสมือน Hidden Layer เป็นตัวประมวลผลในชุดข้อมูลที่ได้รับเข้ามา และ Axon จะทำหน้าที่แปลงข้อมูลให้เป็นผลลัพธ์ออกมา จากนั้นจะแสดงผลลัพธ์นั้นด้วย Synapses ที่เสมือน Output Layer ดังรูปที่ 2.12



รูปที่ 2.12 โครงข่ายประสาทเทียม

2.10.1 โครงข่ายประสาทเทียมแบบหลายชั้น

MLP เป็นโครงข่ายประสาทเทียม (Artificial Neural Network) ประเภทหนึ่งที่มีโครงสร้างแบบฟีดฟอร์เวิร์ด (Feedforward) คือ สัญญาณข้อมูลจะไหลไปในทิศทางเดียวจากชั้นต้นทางไปยังชั้นปลายทางเท่านั้น โดย MLP เกิดขึ้นจากความพยายามพัฒนาต่อยอดจาก Perceptron แบบชั้นเดียว (Single-Layer Perceptron) ซึ่งเดิมสามารถใช้จำแนกข้อมูลได้เฉพาะกรณีที่ข้อมูลสามารถแบ่งแยกได้ด้วยเส้นตรง (Linearly Separable) เท่านั้น MLP จึงถูกพัฒนาให้ประกอบด้วย เซลล์ประสาทที่เชื่อมต่อกันแบบเต็มรูปแบบ (Fully Connected) และใช้ฟังก์ชันกระตุ้นแบบไม่เชิงเส้น (Nonlinear Activation Function) ซึ่งช่วยให้สามารถจำแนกข้อมูลที่ไม่สามารถแบ่งแยกได้ด้วยเส้นตรงได้

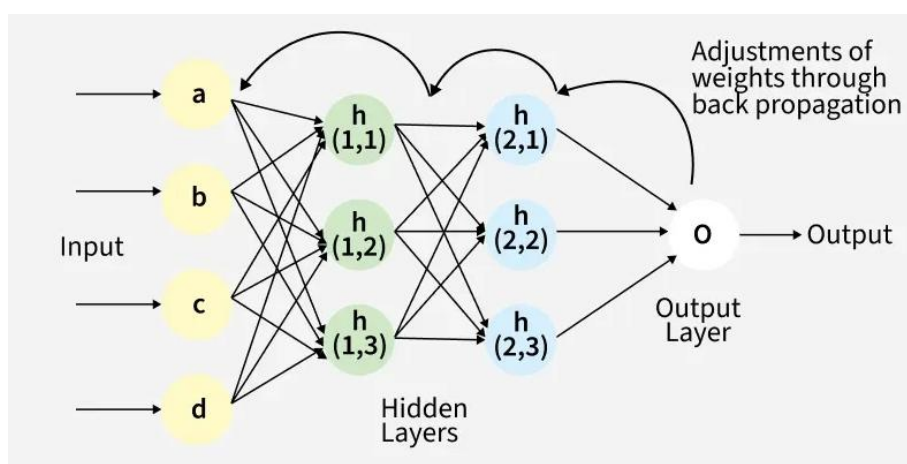


รูปที่ 2.13 โครงสร้าง MLP

จากรูปที่ 2.13 โครงสร้างของ MLP ประกอบด้วย 3 ส่วนหลัก ได้แก่

1. Input Layer ทำหน้าที่รับข้อมูลเข้าสู่โครงข่าย โดยแต่ละโหนดในชั้นนี้จะสอดคล้องกับคุณลักษณะ (Feature) หนึ่งอย่างของข้อมูล
2. Hidden Layer เป็นชั้นที่ทำหน้าที่ประมวลผลข้อมูลระหว่างชั้นอินพุตและเอาต์พุต ซึ่งอาจมีได้ตั้งแต่หนึ่งชั้นขึ้นไป โดยในแต่ละชั้นจะมีการเชื่อมต่อกันแบบเต็มรูปแบบ กล่าวคือ เซลล์ประสาททุกตัวในชั้นหนึ่งจะเชื่อมต่อกับเซลล์ประสาททุกตัวในชั้นก่อนหน้า และจะมีการใช้ฟังก์ชันกระตุ้นในแต่ละชั้นเพื่อช่วยให้โครงข่ายสามารถเรียนรู้ความสัมพันธ์ที่ซับซ้อนของข้อมูลได้
3. Output Layer ทำหน้าที่คำนวณผลลัพธ์สุดท้ายของโครงข่าย ซึ่งอาจเป็นค่าตัวเลขเดียวสำหรับงานถดถอย (Regression) หรือชุดของค่าความน่าจะเป็นของแต่ละกลุ่มสำหรับงานจำแนกประเภท (Classification)

หลักการเรียนรู้ของ MLP ใช้กระบวนการที่เรียกว่า Backpropagation โดยในระหว่างการศึกษาฝึกฝน ค่าน้ำหนัก (Weight) ของการเชื่อมต่อระหว่างเซลล์ประสาทแต่ละคู่จะถูกปรับด้วยวิธีการหาค่าที่เหมาะสมที่สุด เช่น Gradient Descent เพื่อลดค่าความคลาดเคลื่อนระหว่างผลลัพธ์ที่โครงข่ายทำนายได้กับค่าคำตอบจริงในชุดข้อมูลฝึกฝน จากนั้นจึงนำโครงข่ายไปทดสอบกับชุดข้อมูลที่แยกต่างหาก เพื่อปรับจูนค่าตัวแปรไฮเปอร์พารามิเตอร์และป้องกันปัญหา Overfitting ดังรูปที่ 2.14



รูปที่ 2.14 Backpropagation

MLP จะถูกใช้ในงานประเภท Supervised Learning เช่น การจำแนกประเภท (Classification) และการทำนายค่าต่อเนื่อง (Regression) และยังถือเป็นพื้นฐานสำคัญที่นำไปสู่การพัฒนาโครงข่ายประสาทเทียมที่ซับซ้อนขึ้น เช่น CNN และ RNN ในเวลาต่อมา MLP มีข้อจำกัดในการ

ประมวลผลข้อมูลที่มีลักษณะเป็นลำดับหรือหลายมิติ เช่น ภาพหรือข้อความ ซึ่งในกรณีเหล่านี้จะใช้โครงข่ายประเภทอื่นที่เหมาะสมกว่าแทน

2.10.2 ฟังก์ชันกระตุ้น (Activation Functions)

ในสถาปัตยกรรมของโครงข่ายประสาทเทียม (Artificial Neural Network: ANN) โดยเฉพาะโครงข่ายประสาทเทียมแบบเพอร์เซปตรอนหลายชั้น (Multi-layer Perceptron: MLP) ฟังก์ชันกระตุ้นจัดเป็นองค์ประกอบหลักในส่วนประมวลผลของนิวรอน (Neuron) ทำหน้าที่นำผลรวมถ่วงน้ำหนักของสัญญาณข้อมูลขาเข้าบวกกับค่าไบแอส (Weighted Sum of Inputs and Bias) มาคำนวณผ่านฟังก์ชันทางคณิตศาสตร์ เพื่อแปลงค่าและกำหนดระดับของสัญญาณเอาต์พุตที่จะส่งต่อไปยังนิวรอนในชั้นถัดไป (Hidden Layers) หรือชั้นเอาต์พุต (Output Layer)

- ฟังก์ชันเรลู (Rectified Linear Unit: ReLU) ใช้ประมวลผลภายในชั้นซ่อนทั้งสองชั้นดังสมการที่ 6

$$f(z) = \max(0, z) = \begin{cases} z & \text{ถ้า } z > 0 \\ 0 & \text{ถ้า } z \leq 0 \end{cases} \quad (6)$$

- ฟังก์ชัน ReLU มีข้อดีสำคัญหลายประการ ได้แก่ คำนวณได้รวดเร็ว ช่วยลดปัญหาความจางหายของเกรเดียนต์ (Vanishing Gradient) ซึ่งจะเกิดขึ้นกับโครงข่ายชั้นลึก และทำให้นิวรอนส่วนหนึ่งมีค่าเป็นศูนย์ (Sparse Activation) ซึ่งช่วยลดความซับซ้อนของแบบจำลองโดยปริยาย

- ฟังก์ชันซอฟต์แมกซ์ (Softmax Function) ใช้ประมวลผลในชั้นส่งออก เพื่อแปลงค่าดิบจากนิวรอนทั้ง 31 ตัวให้กลายเป็นการกระจายความน่าจะเป็น (Probability Distribution) ที่มีผลรวมเท่ากับ 1.0 เสมอดังสมการที่ 7

$$P(Class_k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (7)$$

โดยที่ z_k คือค่าดิบ (Logit) ของคลาสที่ k และ K คือจำนวนคลาสทั้งหมด (31 คลาส) ผลลัพธ์ที่ได้คือค่าความน่าจะเป็นของแต่ละท่าทาง และแบบจำลองจะเลือกคลาสที่มีความน่าจะเป็นสูงสุดเป็นคำตอบสุดท้าย

2.10.3 กระบวนการฝึกสอนแบบจำลองและการย้อนกลับ

กระบวนการฝึกสอน MLP ดำเนินงานแบบวนซ้ำในแต่ละรอบการเรียนรู้ (Epoch) โดยมีขั้นตอนหลักดังนี้

1. การส่งข้อมูลไปข้างหน้า (Forward Pass) ข้อมูลนำเข้าจะถูกส่งผ่านชั้นต่าง ๆ ของโครงข่ายตามลำดับ โดยแต่ละนิวรอนคำนวณผลลัพธ์ตามสมการที่กล่าวมาข้างต้น จนได้ค่าความน่าจะเป็นในชั้นส่งออก

2. การคำนวณความสูญเสีย (Loss Calculation) นำผลการทำนายที่ได้มาเปรียบเทียบกับคำตอบที่ถูกต้องจริง (Ground Truth) โดยใช้ฟังก์ชันความสูญเสียแบบ Cross-Entropy สำหรับงานจำแนกประเภทหลายคลาสดังสมการที่ 8

$$L = - \sum_{k=1}^k y_k \log \hat{y}_k \quad (8)$$

โดยที่ y_k คือค่าจริง (1 เมื่อถูกต้อง, 0 เมื่อผิด) และ $\hat{y}_k$ คือค่าความน่าจะเป็นที่แบบจำลองทำนายได้

3. การย้อนกลับ (Backpropagation) นำค่าความสูญเสียมาคำนวณอนุพันธ์ย้อนกลับจากชั้นส่งออกไปยังชั้นนำเข้า เพื่อหาค่าน้ำหนักของนิวรอนตัวใดมีส่วนทำให้เกิดความผิดพลาดมากที่สุด โดยใช้กฎลูกโซ่ (Chain Rule) ของแคลคูลัสดังสมการที่ 9

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_i} \quad (9)$$

4. การปรับค่าน้ำหนักด้วยตัวเพิ่มประสิทธิภาพ (Optimizer) ในกรณีนี้เลือกใช้อัลกอริทึม Adam (Adaptive Moment Estimation) ซึ่งเป็นการพัฒนาต่อยอดจากวิธีการลดความชันแบบสุ่ม (Stochastic Gradient Descent: SGD) Adam ปรับอัตราการเรียนรู้ (Learning Rate) ของแต่ละน้ำหนักโดยอัตโนมัติ โดยอาศัยค่าโมเมนต์ลำดับที่หนึ่ง (Mean) และลำดับที่สอง (Uncentered Variance) ของเกรเดียนต์ที่สะสมมา ทำให้การฝึกสอนลู่เข้าสู่จุดที่ดีที่สุดได้เร็วและมีเสถียรภาพกว่าวิธีอื่น

2.10.4 การเตรียมชุดข้อมูลและการป้องกันสภาวะ Overfitting

ในการพัฒนาแบบจำลองปัญญาประดิษฐ์ประเภทโครงข่ายประสาทเทียม ข้อมูลดิบ (Raw Data) ที่ตรวจวัดได้จากเซนเซอร์บนอุปกรณ์สวมใส่จะไม่สามารถนำเข้าสู่กระบวนการฝึกสอน (Training Process) ได้โดยตรง เนื่องจากข้อมูลจากเซนเซอร์ต่างชนิดกันจะมีหน่วยวัด ช่วงมิติข้อมูล (Scale) และสัญญาณรบกวนที่แตกต่างกัน การเตรียมชุดข้อมูลจึงเป็นขั้นตอนสำคัญในการจัดสรรข้อมูลให้อยู่ในรูปแบบที่เหมาะสม เพื่อเพิ่มประสิทธิภาพและความเร็วในการเข้าสู่จุดค่าตอบที่ดีที่สุดของแบบจำลอง

สภาวะ Overfitting คือสภาวะที่แบบจำลองจดจำข้อมูลฝึกสอนได้อย่างแม่นยำมากเกินไป แต่ไม่สามารถนำความรู้ที่ได้ไปประยุกต์กับข้อมูลชุดใหม่ที่ไม่เคยพบได้อย่างถูกต้อง ซึ่งในทางปฏิบัติจะสังเกตได้จากความแม่นยำบนชุดข้อมูลฝึกสอนสูงมาก แต่ความแม่นยำบนชุดข้อมูลทดสอบกลับต่ำกว่าอย่างมีนัยสำคัญ

- การปรับมาตรฐานข้อมูลนำเข้า (Feature Scaling) เนื่องจากข้อมูลดิบจากเซนเซอร์แต่ละประเภทมีช่วงค่าแตกต่างกันอย่างมาก เช่นเซนเซอร์ไค่งอให้ค่าในช่วง 0 ถึง 4,095 ในขณะที่ค่าความเร่งมีช่วงประมาณ -10 ถึง +10 และค่าความเร็วเชิงมุมมีช่วงประมาณ -0.1 ถึง +0.1 หากนำข้อมูลเหล่านี้เข้าสู่แบบจำลองโดยตรง น้ำหนักของนิวรอนจะถูกครอบงำโดยข้อมูลกลุ่มที่มีค่าสูงกว่า ส่งผลให้แบบจำลองเรียนรู้ได้ไม่สมดุล จึงจำเป็นต้องปรับมาตรฐานข้อมูลด้วยวิธี Z-Score Normalization โดยใช้ StandardScaler ดังสมการที่ 10

$$x' = \frac{x - \mu}{\sigma} \quad (10)$$

โดยที่ μ คือค่าเฉลี่ยของข้อมูลในแต่ละมิติ และ σ คือส่วนเบี่ยงเบนมาตรฐาน ผลลัพธ์ที่ได้คือข้อมูลในทุกมิติมีค่าเฉลี่ยเท่ากับ 0 และส่วนเบี่ยงเบนมาตรฐานเท่ากับ 1

- การแบ่งชุดข้อมูล (Train-Test Split) เพื่อประเมินประสิทธิภาพแบบจำลองบนข้อมูลที่ไม่เคยเห็นมาก่อนและป้องกันสภาวะ Overfitting ชุดข้อมูลทั้งหมดจะถูกแบ่งออกเป็นสองส่วนในอัตราส่วน 80:20 ดังนี้ ชุดข้อมูลฝึกสอน (Training Set) 80% ใช้ปรับค่าน้ำหนักและค่าเอนเอียงของแบบจำลองในแต่ละรอบการเรียนรู้ ชุดข้อมูลทดสอบ (Testing Set) 20% ใช้ประเมินประสิทธิภาพแบบจำลองหลังสิ้นสุดกระบวนการฝึกสอนทั้งหมด

2.11 Arduino IDE

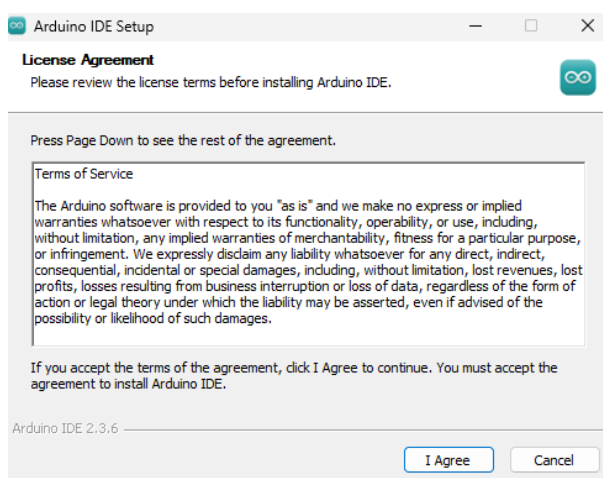
Arduino IDE คือโปรแกรม Integrated Development Environment (IDE) ที่ใช้สำหรับเขียน, คอมไพล์ (Compile) และอัปโหลด (Upload) โค้ดคำสั่งไปยังบอร์ดไมโครคอนโทรลเลอร์ของ Arduino โปรแกรมนี้เป็นโอเพนซอร์ส (Open Source) ใช้งานได้ฟรี และออกแบบมาให้เขียนโปรแกรมด้วยภาษา C++ ในรูปแบบที่ง่ายขึ้นสำหรับผู้เริ่มต้น ทำให้การพัฒนาโปรเจกต์อิเล็กทรอนิกส์ทำได้สะดวกและรวดเร็ว

2.12 วิธีการติดตั้งและใช้งาน Arduino IDE

2.12.1 วิธีการดาวน์โหลดและติดตั้ง

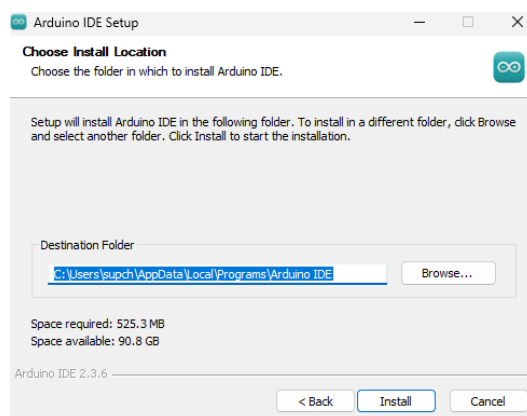
สามารถทำการดาวน์โหลดโปรแกรม Arduino IDE ได้จาก หน้า ดาวน์โหลดโปรแกรม Arduino IDE (arduino.cc/en/software)

เมื่อเปิดตัวติดตั้งขึ้นมาหน้าจอจะแสดงให้เห็นให้เลือก อ่านแล้วกดปุ่ม I Agree ดังรูปที่ 2.15



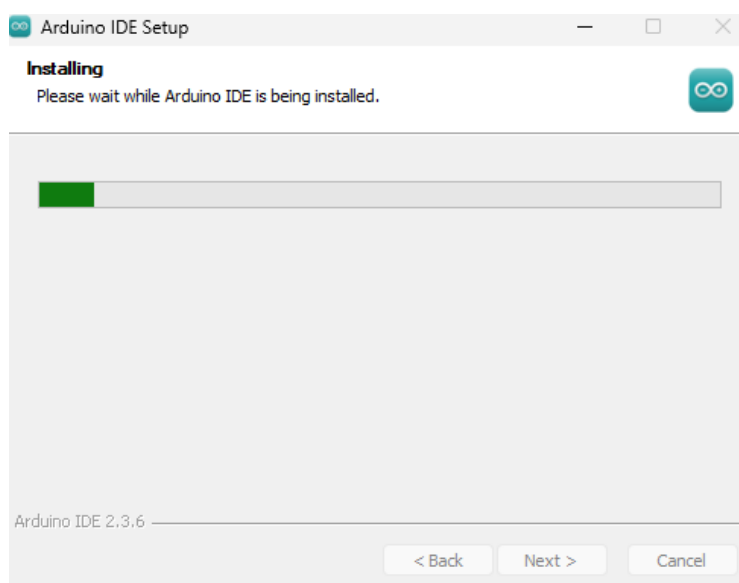
รูปที่ 2.15 การติดตั้ง Arduino IDE

หลังจากนั้นจะแสดงหน้าต่างที่จะให้เลือกโฟลเดอร์ที่ใช้เก็บไฟล์ของโปรแกรม (Select Destination Location) แล้วกดปุ่ม Install ดังรูปที่ 2.16



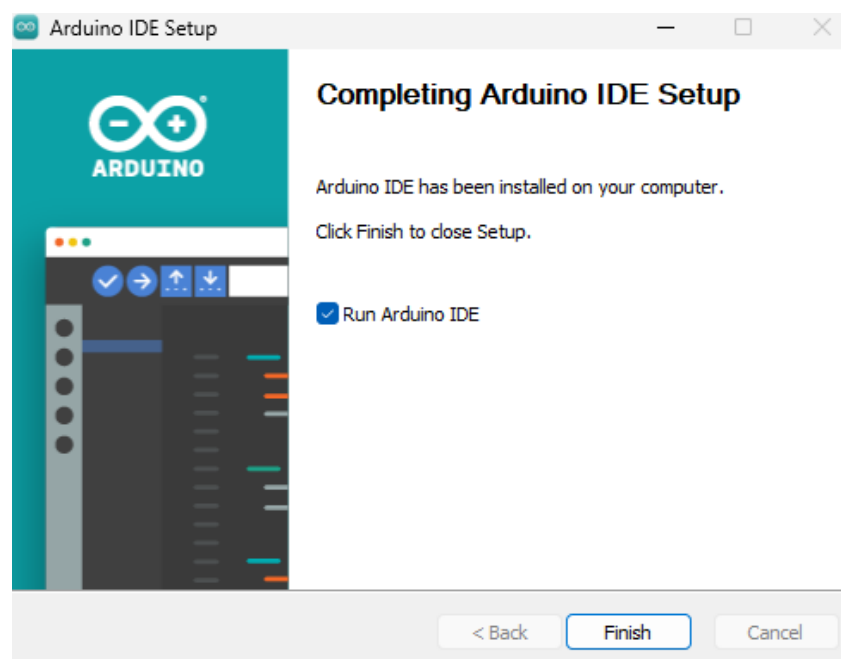
รูปที่ 2.16 การเลือกโฟลเดอร์ที่จะใช้เก็บไฟล์

จากนั้นรอโปรแกรมทำการติดตั้ง ดังรูปที่ 2.17



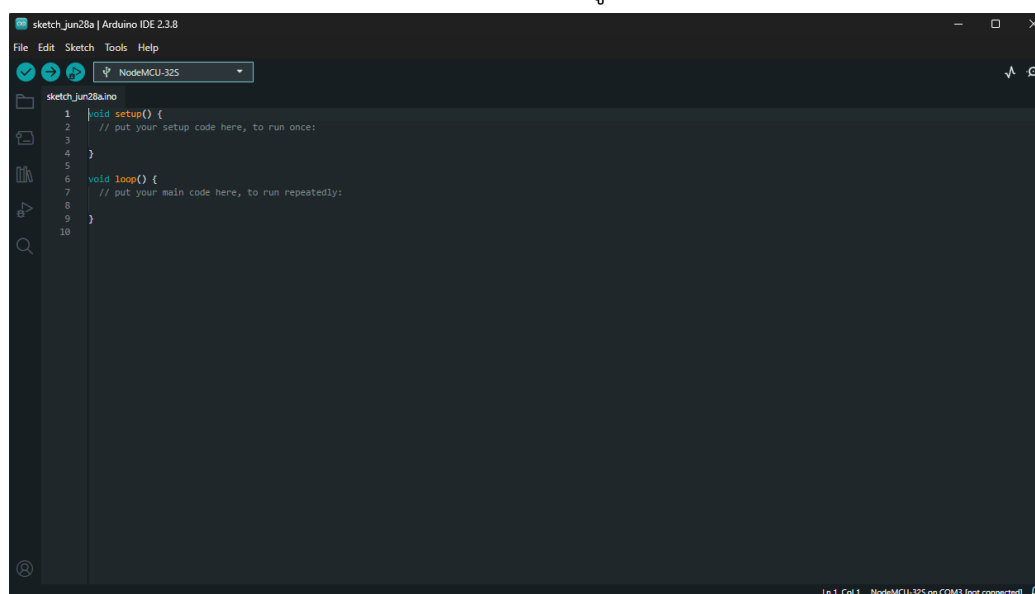
รูปที่ 2.17 การติดตั้ง Arduino IDE

เมื่อโปรแกรมทำการติดตั้งเสร็จแล้วจะแสดงหน้าจอ (Completing Arduino IDE setup)
ให้กดปุ่ม Finish ดังรูปที่ 2.18



รูปที่ 2.18 การติดตั้ง Arduino IDE สมบูรณ์

เมื่อกด finish จะได้หน้าต่าง Arduino IDE ดังรูปที่ 2.19



รูปที่ 2.19 หน้าต่าง Arduino IDE

2.13 Visual Studio Code

Visual Studio Code เป็นโปรแกรมที่พัฒนาโดยบริษัท Microsoft ซึ่งได้รับการออกแบบมาเพื่อรองรับการพัฒนาโปรแกรมอย่างมีประสิทธิภาพ มีฟังก์ชันการทำงานที่ครอบคลุม เช่น การเน้นไวยากรณ์ของภาษาโปรแกรม การรองรับการควบคุมเวอร์ชัน โดยเฉพาะกับ JavaScript, TypeScript และ Node.js รวมถึงการรองรับภาษาโปรแกรมหลากหลาย เช่น C++, C#, Java, Python, PHP และ Go รวมถึงการรองรับรันไทม์ต่างๆ อย่าง .NET นอกจากนี้ยังสามารถเพิ่มขีดความสามารถของโปรแกรมผ่านส่วนขยายต่างๆ

Visual Studio Code เป็นที่นิยมในวงการพัฒนาซอฟต์แวร์ เนื่องจากมีขนาดเล็ก ประสิทธิภาพสูง และสามารถปรับแต่งให้เข้ากับความต้องการเฉพาะของผู้พัฒนาได้อย่างยืดหยุ่น อีกทั้งยังรองรับการใช้งานข้ามแพลตฟอร์มทั้งในระบบปฏิบัติการ Windows, macOS และ Linux

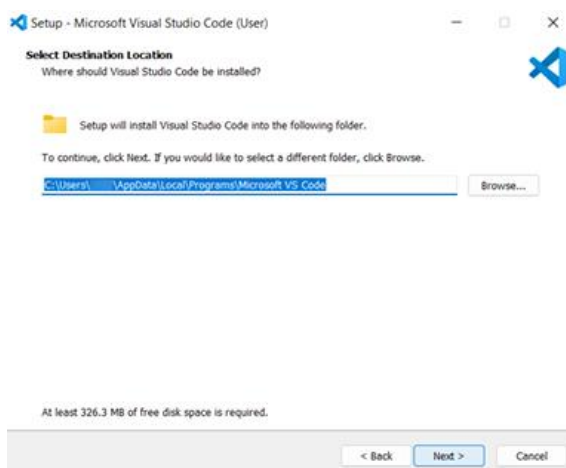
2.13.1 วิธีการดาวน์โหลดและติดตั้ง

1. สามารถทำการดาวน์โหลดโปรแกรม Visual Studio Code ได้จากหน้า ดาวน์โหลดโปรแกรม Visual Studio Code
2. เมื่อเปิดตัวติดตั้งขึ้นมาหน้าจะแสดงให้เห็นให้เลือก (I accept the agreement) แล้ว กดปุ่ม Next ดังรูปที่ 2.20



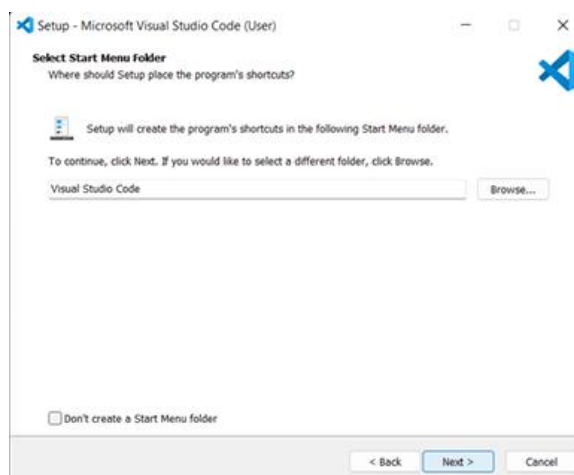
รูปที่ 2.20 การติดตั้ง Visual Studio Code

3. หลังจากนั้นจะแสดงหน้าต่างขึ้นมาให้เลือกโฟลเดอร์ที่จะใช้เก็บไฟล์ของโปรแกรม (Select Destination Location) แล้วกดปุ่ม Next ดังรูปที่ 2.21



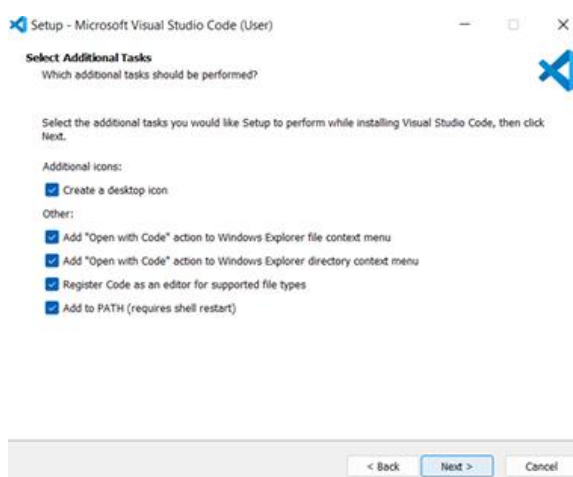
รูปที่ 2.21 การเลือกโฟลเดอร์ที่จะใช้เก็บไฟล์

4. จากนั้นจะแสดงหน้าจอเลือก (Select Start Menu Folder) แล้วกดปุ่ม Next ดังรูปที่ 2.22



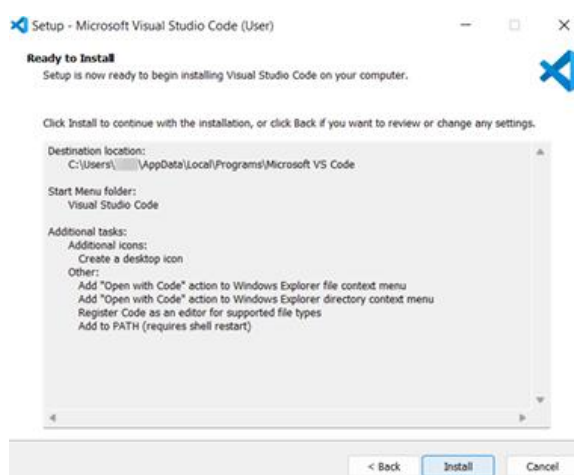
รูปที่ 2.22 โฟลเดอร์เมนู Start สำหรับโค้ด Visual Studio

5. จากนั้นจะแสดงหน้าจอ (Select Additional Tasks) แล้วกดปุ่ม Next ดังรูปที่ 2.23



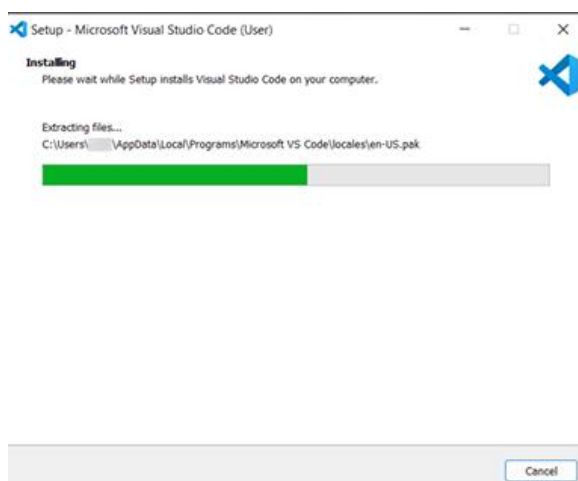
รูปที่ 2.23 กำหนดค่าการติดตั้ง

6. หลังจากนั้นจะแสดงหน้าจอ (Ready To Install) แล้วกดปุ่ม Install ดังรูปที่ 2.24



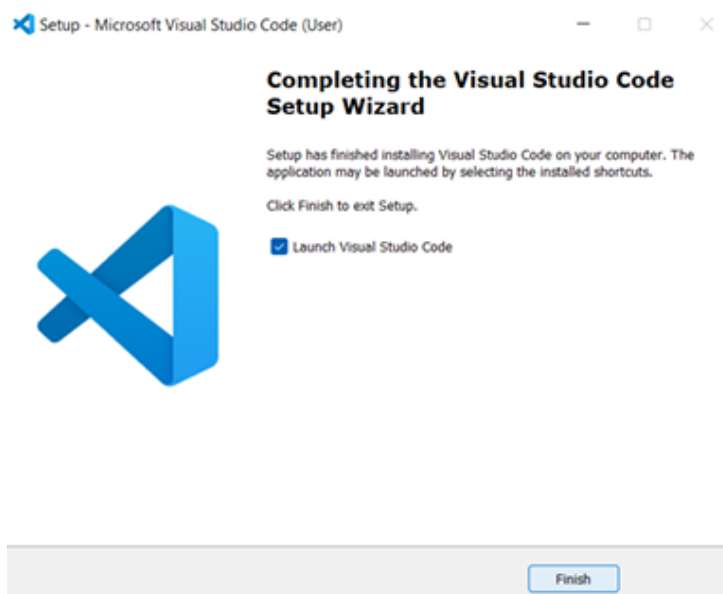
รูปที่ 2.24 ยืนยันการติดตั้ง

7. จากนั้นรอโปรแกรมทำการติดตั้ง ดังรูปที่ 2.25



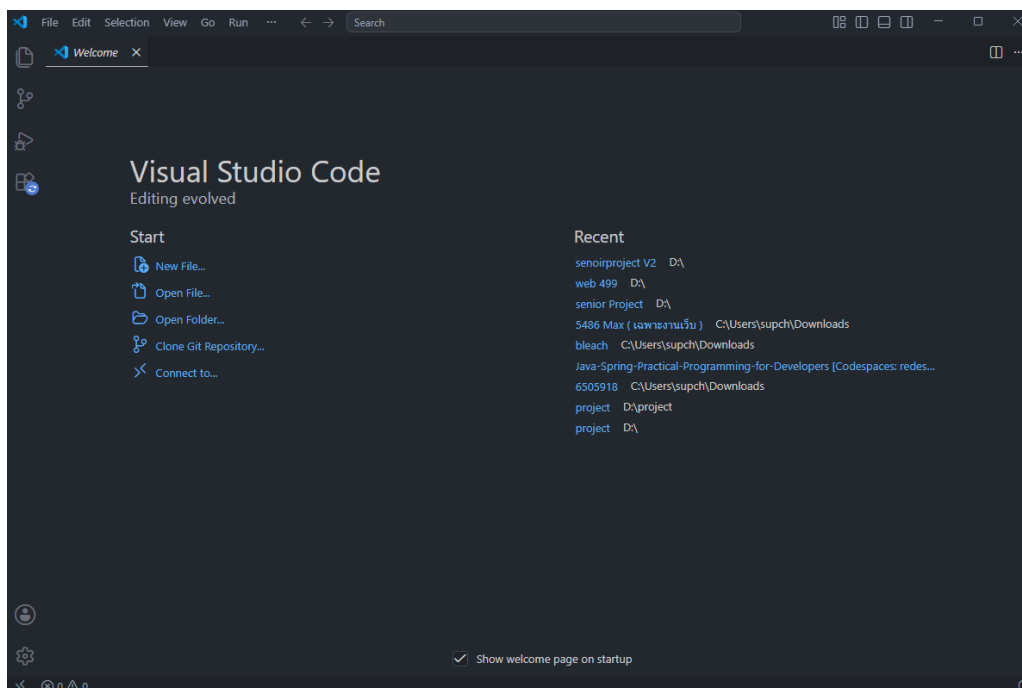
รูปที่ 2.25 การติดตั้ง Visual Studio

8. เมื่อโปรแกรมทำการติดตั้งเสร็จแล้วจะแสดงหน้าจอ (Completing the Visual Studio Code Setup Wizard) ให้กดปุ่ม Finish ดังรูปที่ 2.26



รูปที่ 2.26 การติดตั้ง Visual Studio สมบูรณ์

9. เมื่อกด Finish จะได้หน้าต่าง Visual Studio Code ดังรูปที่ 2.27



รูปที่ 2.27 หน้าต่าง Visual Studio Code

2.14 Python Programming Language

ภาษา Python เป็นภาษาโปรแกรมคอมพิวเตอร์ระดับสูงที่ได้รับการยอมรับและใช้งานอย่างแพร่หลายในการเทคโนโลยีสารสนเทศและวิทยาการคอมพิวเตอร์ ด้วยคุณลักษณะที่โดดเด่นในด้านความเรียบง่ายของไวยากรณ์ ความสะดวกในการใช้งาน และความยืดหยุ่นสูง Python จึงถูกนำมาใช้ประโยชน์อย่างกว้างขวางในหลายสาขา เช่น การพัฒนาแอปพลิเคชันเว็บ การวิเคราะห์และประมวลผลข้อมูลเชิงลึก การคำนวณทางวิทยาศาสตร์ การพัฒนาระบบการเรียนรู้ของเครื่องและปัญญาประดิษฐ์ ตลอดจนการประมวลผลภาษาธรรมชาติ ซึ่งล้วนเป็นเทคโนโลยีที่มีบทบาทสำคัญในการขับเคลื่อนนวัตกรรมและการพัฒนาในยุคดิจิทัล โดยลักษณะและคุณสมบัติของ Python

1. Python มีไวยากรณ์ที่เรียบง่ายและอ่านง่าย ทำให้ผู้เริ่มต้นสามารถเข้าใจและใช้งานได้เร็ว
2. Python รองรับทั้งโปรแกรมมิ่งเชิงวัตถุ โปรแกรมมิ่งเชิงฟังก์ชัน และโปรแกรมมิ่งเชิงกระบวนการ
3. Python มีไลบรารีและเฟรมเวิร์คมากมายที่ช่วยในการพัฒนา เช่น NumPy, SciPy, Pandas, Scikit-Learn, TensorFlow และ PyTorch

4. Python เป็นภาษาที่มีคำสั่งพร้อมใช้งานมากมาย เช่น การจัดการไฟล์, เครือข่าย, การทำงานกับข้อมูล, และอื่น ๆ ที่ช่วยในการพัฒนาแอปพลิเคชันได้ง่ายขึ้น

5. Python มีระบบจัดการข้อผิดพลาดที่ช่วยให้การพัฒนาโปรแกรมทำได้ง่ายและลดข้อผิดพลาด

ตัวอย่างการใช้งานดังรูปที่ 2.28

```
1 print("Hello, World!")
```

รูปที่ 2.28 ตัวอย่างการใช้งาน Python

2.15 ไบблиотеกาที่ใช้

ไลบรารีคือชุดของโค้ดที่ใช้บ่อยซึ่งนักพัฒนาสามารถใช้ในโปรแกรม Python เพื่อหลีกเลี่ยงการเขียนโค้ดขึ้นใหม่ทั้งหมด ตามค่าเริ่มต้นแล้ว Python จะมาพร้อมกับไลบรารีมาตรฐาน ซึ่งมีฟังก์ชันที่น่ากลับมาใช้ใหม่ได้มากมาย นอกจากนี้ยังมีไลบรารีสำหรับการประยุกต์ใช้ต่างๆ รวมถึงการพัฒนาเว็บ วิทยาศาสตร์ข้อมูล และแมชชีนเลิร์นนิง (ML)

2.15.1 NumPy

NumPy (Numerical Python) คือไลบรารีที่ใช้ในการทำงานกับข้อมูลทางตัวเลขในภาษา Python โดยเฉพาะในงานที่เกี่ยวข้องกับการคำนวณทางคณิตศาสตร์ที่จำเป็นสำหรับ Deep Learning หรือดำเนินการทางตรรกะในอาร์เรย์หลายมิติ หรือแมทริกซ์ได้อย่างรวดเร็ว เพราะไลบรารีเขียนด้วยภาษา C ที่คอมไพล์ไว้แล้ว

NumPy มีความสามารถในการจัดการกับข้อมูลแบบแมทริกซ์ (Matrix) และอาร์เรย์ (Array) หลายมิติได้หลายวิธี โมดูลนี้สามารถสร้างข้อมูลชนิดอาร์เรย์ที่ภาษา Python ไม่มี และคำนวณอาร์เรย์ของตัวเลข สตริง (String) และวัตถุได้

วิธีการติดตั้ง

ทำการติดตั้ง ด้วย pip ดังรูปที่ 2.29

```
Pip install numpy
```

รูปที่ 2.29 ติดตั้ง Numpy

การเรียกใช้งาน Numpy ดังรูปที่ 2.30

```
Import numpy as np
```

รูปที่ 2.30 เรียกใช้ Numpy

ตัวอย่างการใช้งานดังรูปที่ 2.31

```
A = np.array([1,2,3])
Print(a)
```

รูปที่ 2.31 ตัวอย่างการใช้งาน Numpy

2.15.2 Flask

Flask เป็นเว็บเฟรมเวิร์กขนาดเล็ก (Micro-framework) สำหรับภาษา Python ที่ทำงานบนมาตรฐาน WSGI (Web Server Gateway Interface) มีจุดเด่นคือความเบา (Lightweight) ติดตั้งง่าย และมีความยืดหยุ่นสูง

วิธีการติดตั้ง

ทำการติดตั้ง ด้วย pip ดังรูปที่ 2.32

```
pip install flask
```

รูปที่ 2.32 ติดตั้ง Flask

การเรียกใช้งาน Flask ดังรูปที่ 2.33

```
from flask import Flask
```

รูปที่ 2.33 เรียกใช้งาน Flask

ตัวอย่างการใช้งานดังรูปที่ 2.34

```
app = Flask( __ )
@app.route("/")
def index():
    return "Hello, Flask!"
```

รูปที่ 2.34 ตัวอย่างการใช้งาน Flask

2.15.3 Scikit-learn

Scikit-learn เป็นไลบรารี Python แบบโอเพนซอร์ส (Open-source) สำหรับงานด้านการเรียนรู้ของเครื่อง (Machine Learning - ML) และการวิเคราะห์ข้อมูล โดยมีเครื่องมือที่ใช้งานง่ายครอบคลุมอัลกอริทึมที่หลากหลาย ทั้งการจำแนกประเภท (Classification), การถดถอย (Regression), การจัดกลุ่ม (Clustering) และการลดมิติข้อมูล (Dimensionality Reduction) สร้างบนรากฐานของ NumPy, SciPy และ Matplotlib

วิธีการติดตั้ง

ทำการติดตั้ง ด้วย pip ดังรูปที่ 2.35

```
pip install -U scikit-learn
```

รูปที่ 2.35 ติดตั้ง Scikit learn

การเรียกใช้งาน Scikit-learn ดังรูปที่ 2.36

```
from sklearn
```

รูปที่ 2.36 เรียกใช้ Scikit learn

ตัวอย่างการใช้งานดังรูปที่ 2.37

```
x = [[0, 0], [1, 1]]
y = [0, 1]
clf = svm.SVC()
clf.fit(x, y)
```

รูปที่ 2.37 ตัวอย่างการใช้งาน Scikit learn

2.15.4 Pandas

เป็นไลบรารีในภาษา Python ที่ใช้สำหรับการจัดการข้อมูลและการวิเคราะห์ข้อมูล โดยเฉพาะการทำงานกับข้อมูลในรูปแบบของตาราง เช่น การจัดการข้อมูลที่มีลักษณะเป็นแถวและคอลัมน์ เช่น ข้อมูลในไฟล์ CSV, Excel, หรือฐานข้อมูลต่างๆ ไลบรารีนี้ช่วยให้การทำงานกับข้อมูลขนาดใหญ่และซับซ้อนเป็นเรื่องง่ายขึ้น

คุณสมบัติหลักของ Pandas

1. DataFrame โครงสร้างข้อมูลหลักที่ใช้ใน Pandas คือ DataFrame ซึ่งเป็นตารางข้อมูลที่มีแถวและคอลัมน์ คล้ายกับข้อมูลในสเปรดชีตหรือฐานข้อมูล
2. Series เป็นโครงสร้างข้อมูลที่คล้ายกับแถวเดียวของ DataFrame ซึ่งจัดเก็บข้อมูลในรูปแบบของ
3. การอ่านและเขียนข้อมูล สามารถอ่านและเขียนข้อมูลจากไฟล์ต่าง ๆ เช่น CSV, Excel, SQL databases และไฟล์ JSON ได้อย่างง่ายดาย
4. การจัดการข้อมูล สามารถทำการคัดเลือก กรอง สรุปรูป เปลี่ยนแปลงรูปแบบข้อมูล และการจัดการค่าที่หายไปได้อย่างมีประสิทธิภาพ
5. การคำนวณและการสรุปข้อมูล รองรับการคำนวณเชิงสถิติ การคำนวณทางคณิตศาสตร์ และการทำการคำนวณเชิงกลยุทธ์ เช่น การคำนวณค่าเฉลี่ย ค่าผลรวม หรือการหาความสัมพันธ์ในข้อมูล

คำสั่งติดตั้งใช้งานดังรูปที่ 2.38

```
pip install pandas
```

รูปที่ 2.38 ติดตั้ง Pandas

การเรียกใช้งาน Pandas ดังรูปที่ 2.39

```
import pandas
```

รูปที่ 2.39 เรียกใช้ Pandas

ตัวอย่างการใช้งานดังรูปที่ 2.40

```
Import pandas
mydataset = {
  'person': ["Name", "Year", "Address"],
  'list': [Peter, 20, lao]
}
Var = pandas.DataFrame(mydataset)
Print(var)
```

รูปที่ 2.40 ตัวอย่างการใช้งาน Pandas

2.15.5 Socket Library

Socket เป็นไลบรารีมาตรฐานที่มาพร้อมกับภาษา Python (Python Standard Library) ออกแบบมาสำหรับการสื่อสารผ่านเครือข่ายในระดับต่ำ (Low-level Networking Interface) โดยรองรับโปรโตคอลการสื่อสารหลักทั้ง TCP (Transmission Control Protocol) และ UDP (User Datagram Protocol) ไลบรารีนี้ช่วยให้นักพัฒนาสามารถสร้างช่องทางการรับส่งข้อมูลระหว่างอุปกรณ์บนเครือข่ายได้โดยตรง โดยไม่จำเป็นต้องพึ่งพาเฟรมเวิร์กภายนอก

เนื่องจาก Socket เป็นส่วนหนึ่งของ Python Standard Library จึงไม่จำเป็นต้องติดตั้งเพิ่มเติมสำหรับการใช้งานพื้นฐาน อย่างไรก็ตาม หากต้องการใช้งานในรูปแบบ asynchronous หรือ event-driven สามารถติดตั้งไลบรารีเสริม python-socketio ได้ผ่านคำสั่งต่อไปนี้

คำสั่งติดตั้งใช้งานดังรูปที่ 2.41

```
pip install python-socketio
```

รูปที่ 2.41 ติดตั้ง Socket Library

การเรียกใช้งานไลบรารี Socket ดังรูปที่ 2.42

```
import socket
```

รูปที่ 2.42 เรียกใช้ Socket

ตัวอย่างโค้ดสำหรับการใช้งานในฝั่ง Server มีดังนี้

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 12345))
server_socket.listen(5)
print(f'Listening for client connections on port 12345...')
client_socket, address = server_socket.accept()
print(f'Connected to {address}')
while True:
    data = client_socket.recv(1024)
    if not data:
        break
    print(f'Received message: {data.decode()}')
    client_socket.sendall(data)
client_socket.close()
server_socket.close()
```

รูปที่ 2.43 ตัวอย่างการใช้งาน Socket

จากรูปที่ 2.43 ขั้นตอนการทำงานของ Server แบ่งออกเป็น 3 ขั้นตอนหลัก ดังนี้

1. การตั้งค่า Server socket.socket (AF_INET, SOCK_STREAM) สร้าง TCP socket สำหรับการสื่อสารผ่านเครือข่าย IPv4 จากนั้น bind () ผูก socket เข้ากับ localhost หมายเลขพอร์ต 12345 และ listen(5) กำหนดให้ Server พร้อมรับการเชื่อมต่อพร้อมกันสูงสุด 5 รายการในคิว
2. การรับการเชื่อมต่อ accept () หยุดรอการทำงาน (blocking) จนกว่า Client จะส่งคำขอเชื่อมต่อเข้ามา เมื่อมีการเชื่อมต่อสำเร็จ ฟังก์ชันจะคืนค่า client_socket สำหรับใช้รับส่งข้อมูล และ address ซึ่งระบุ IP address และหมายเลขพอร์ตของ Client
3. การรับและส่งข้อมูล (Echo Loop) ภายใน while True Server จะรับข้อมูลที่ละ 1,024 bytes ผ่าน recv(1024) แล้วแสดงผลข้อความที่รับมา จากนั้น sendall() ส่งข้อมูลชุดเดิมกลับไปยัง Client ทันที การวนซ้ำจะสิ้นสุดลงเมื่อ Client ปิดการเชื่อมต่อ ซึ่ง recv() จะคืนค่า b"" และ Server จะปิด socket ทั้งสองด้วย close ()

การทำงานของ Client

ตัวอย่างโค้ดสำหรับการใช้งานในฝั่ง Client มีดังรูปที่ 2.44

```
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 12345))
client_socket.sendall('Hello, server!'.encode())
response = client_socket.recv(1024)
print(f'Server said: {response.decode()}')
client_socket.close()
```

รูปที่ 2.44 ตัวอย่างโค้ดของ Client

socket.socket(AF_INET, SOCK_STREAM) สร้าง TCP socket จากนั้น connect() ส่งคำขอเชื่อมต่อไปยัง localhost หมายเลขพอร์ต 12345 ซึ่งต้องตรงกับค่าที่กำหนดไว้ในฝั่ง Server เมื่อเชื่อมต่อสำเร็จ sendall() จะแปลงข้อความ "Hello, server!" ให้อยู่ในรูปแบบ bytes ด้วย .encode() แล้วส่งไปยัง Server จากนั้น recv(1024) รอรับข้อมูลที่ Server ส่งกลับมา แล้วแปลงกลับเป็น string ด้วย .decode() เพื่อแสดงผล และปิด socket ด้วย close() เมื่อสิ้นสุดการทำงาน

ผลลัพธ์ที่ได้รับเมื่อรันระบบ

เมื่อเริ่มต้น Server ก่อนแล้วจึงรัน Client ระบบจะแสดงผลลัพธ์ดังรูปที่ 2.45

```
PS D:\testjec> python .\hello.py
Server is listening...
Connected by ('127.0.0.1', 62149)
Client: Hello, server!
```

รูปที่ 2.45 Server

การตอบกลับของ Server ดังรูปที่ 2.46

```
PS D:\testjec> python .\client.py
Server said: Hello, client!
```

รูปที่ 2.46 Client

บทที่ 3

การวิเคราะห์และออกแบบระบบ

ในบทนี้นำเสนอกระบวนการวิเคราะห์และออกแบบ ระบบถุงมือแปลภาษามือแบบข้างเดียว (Single-Hand Sign Language Translator Glove) โดยครอบคลุมการวิเคราะห์ข้อจำกัดของระบบเดิม การกำหนดความต้องการของระบบใหม่ และการออกแบบโครงสร้างระบบทั้งด้านฮาร์ดแวร์ วงจร และแบบจำลองปัญญาประดิษฐ์ ที่พัฒนาขึ้นเพื่อใช้งานได้จริงในบริบทของการสื่อสารระหว่างผู้พิการทางการได้ยินกับบุคคลทั่วไป

3.1 วิเคราะห์ระบบเดิม

ผู้พิการทางการได้ยินในปัจจุบันมีแนวทางสื่อสารกับบุคคลทั่วไปอยู่หลายรูปแบบ แต่ละรูปแบบล้วนมีข้อจำกัดที่ส่งผลกระทบต่อชีวิตประจำวัน ผู้จัดทำจึงศึกษาแนวทางที่ใช้งานจริงอยู่ในปัจจุบัน แบ่งได้เป็น 3 กลุ่มดังนี้

3.1.1 การสื่อสารผ่านล่ามภาษามือ

ผู้พิการทางการได้ยินส่วนใหญ่ยังต้องพึ่งพาล่ามภาษามือในการสื่อสารกับบุคคลทั่วไป วิธีนี้มีข้อจำกัดด้านความพร้อมใช้งานและค่าใช้จ่าย โดยเฉพาะในสถานการณ์เร่งด่วนหรือพื้นที่ที่ขาดแคลนล่าม จึงไม่ใช่ทางออกสำหรับการใช้งานประจำวัน

3.1.2 ระบบแปลภาษามือด้วยกล้องและการประมวลผลภาพ

ระบบแปลภาษามือที่ใช้แพร่หลายในระบบเดิม ส่วนใหญ่ใช้กล้องเว็บแคมหรือกล้องสมาร์ตโฟนร่วมกับอัลกอริทึมทางคอมพิวเตอร์วิชัน (Computer Vision) เช่น Convolutional Neural Network (CNN) หรือ MediaPipe เพื่อตรวจจับและแปลท่าทางมือจากภาพวิดีโอ วิธีนี้มีข้อดีคือผู้ใช้งานไม่ต้องสัมผัสร่างกายและสามารถติดตั้งใช้งานได้สะดวก อย่างไรก็ตาม ระบบประเภทนี้ยังมีข้อจำกัดหลายประการ ได้แก่

- ความแม่นยำ ประสิทธิภาพขึ้นอยู่กับสภาพแสง มุมกล้อง และพื้นหลังของภาพเป็นสำคัญ
- ความหน่วงเวลา (Latency) มีความหน่วงสูงเนื่องจากกระบวนการประมวลผลภาพแบบเฟรมต่อเฟรม (Frame-by-Frame)

- ฮาร์ดแวร์ ต้องการอุปกรณ์ที่มีประสิทธิภาพสูงในการประมวลผลโมเดล CN
- สภาพแวดล้อม ไม่เหมาะสำหรับการใช้งานกลางแจ้งหรือในสถานะที่มีแสงน้อย

3.1.3 ระบบถุงมือแปลภาษามือแบบเดิม (Sensor-based Glove)

งานวิจัยบางชิ้นได้พัฒนาถุงมือติดตามเซ็นเซอร์ขึ้นมาเช่นกัน แต่มีการใช้สถาปัตยกรรมที่แตกต่างออกไป เช่น การใช้บอร์ด Arduino Uno ซึ่งมีข้อจำกัดด้านความเร็วในการประมวลผลและการสื่อสารไร้สาย หรือการใช้เทคโนโลยี Bluetooth ในการส่งข้อมูล ซึ่งมีช่วงสัญญาณที่จำกัด

นอกจากนี้ ระบบเดิมส่วนใหญ่ยังเก็บข้อมูลเป็นค่าเดียว (Single Frame) ซึ่งไม่เพียงพอสำหรับการรู้จำท่าทางที่มีการเคลื่อนไหวต่อเนื่อง เช่น ท่าภาษามือตัวอักษร J, Z ส่งผลให้ความแม่นยำในการจดจำท่าทางต่ำกว่าที่ควรจะเป็น

3.2 วิเคราะห์ระบบใหม่

ระบบใหม่ที่พัฒนาขึ้นเป็นถุงมือแปลภาษามือแบบข้างเดียว (Single-Hand Sign Language Translator Glove) ที่เชื่อมต่อกับเว็บแอปพลิเคชัน เพื่อใช้งานจริงในการสื่อสารระหว่างผู้พิการทางการได้ยินกับบุคคลทั่วไป ระบบนี้ถูกออกแบบมาเพื่อให้การตรวจจับท่าทาง การส่งข้อมูล และการแสดงผลคำแปลเป็นไปอย่างรวดเร็วและแม่นยำ โดยไม่ต้องพึ่งพาลำมภาษามือหรือกล้องถ่ายภาพเหมือนระบบเก่า นอกจากนี้ยังรองรับการใช้งานได้ทั้งในที่แจ้งและที่มีแสงน้อย เพื่อให้พนักงานหรือบุคคลทั่วไปสามารถอ่านคำแปลบนหน้าจอเว็บแอปพลิเคชันได้แบบเรียลไทม์

ระบบใหม่นี้แบ่งการทำงานออกเป็น 3 ส่วนหลักที่เชื่อมโยงกัน คือ ฮาร์ดแวร์ตรวจวัดบนถุงมือ ระบบสื่อสารไร้สาย และแบบจำลองปัญญาประดิษฐ์สำหรับจำแนกท่าทาง โดยแต่ละส่วนถูกออกแบบมาเพื่อแก้ปัญหาที่วิเคราะห์ไว้ มีรายละเอียดดังนี้

1. จัดทำระบบตรวจจับท่าทางด้วยเซ็นเซอร์บนถุงมือ เพื่อจัดการปัญหาความแม่นยำที่ขึ้นกับสภาพแวดล้อมในระบบเก่า ระบบใหม่เปลี่ยนจากการใช้กล้องและการประมวลผลภาพมาเป็นการติดตั้ง Flex Sensor และเซ็นเซอร์ตรวจจับความเคลื่อนไหว MPU6050 ไว้บนถุงมือโดยตรง ทำให้ผลการตรวจจับไม่ขึ้นกับสภาพแสง มุมกล้อง หรือพื้นหลังอีกต่อไป และใช้งานได้ทั้งในที่แจ้งและที่มีแสงน้อย โดยไม่ต้องอาศัยฮาร์ดแวร์ประมวลผลภาพราคาสูงเหมือนระบบเดิม

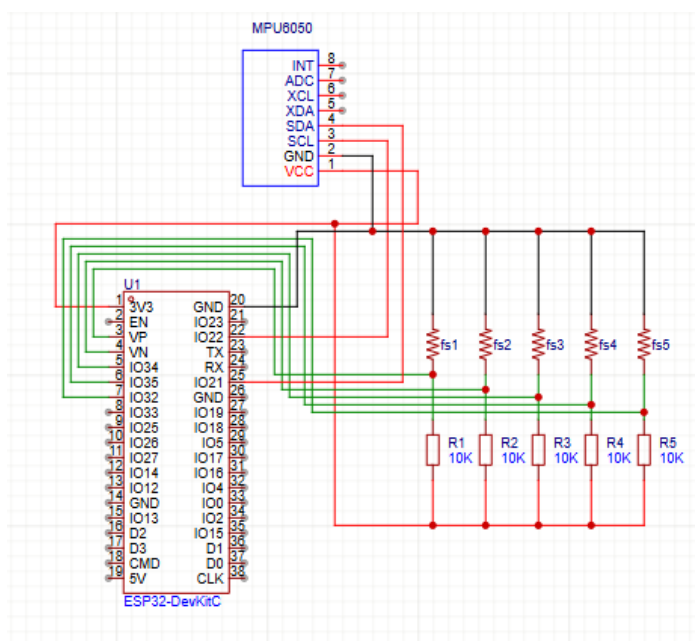
2. จัดทำระบบสื่อสารไร้สายความหน่วงต่ำ เพื่อจัดการปัญหาความเร็วในการประมวลผลและระยะสัญญาณของถุงมือเซ็นเซอร์แบบเก่า ระบบใหม่เปลี่ยนจากไมโครคอนโทรลเลอร์ Arduino Uno และการสื่อสารผ่าน Bluetooth มาใช้ ESP32 ซึ่งมีสมรรถนะการประมวลผลสูงกว่าและมีโมดูล Wi-Fi

ในตัว ร่วมกับโปรโตคอล UDP ที่มีความหน่วงต่ำ ทำให้ข้อมูลจากเซนเซอร์ถูกส่งไปประมวลผลได้อย่างรวดเร็วและต่อเนื่อง

3. จัดทำระบบเก็บข้อมูลแบบลำดับเวลาต่อเนื่อง เพื่อจัดการปัญหาการจดจำท่าทางที่มีการเคลื่อนไหว แทนที่จะเก็บข้อมูลเป็นค่าเดียวเหมือนระบบเดิมซึ่งไม่เพียงพอสำหรับท่าทางอย่างตัวอักษรที่มีการเคลื่อนไหว ระบบใหม่จะเก็บสัญญาณเซนเซอร์ต่อเนื่องเป็นชุดข้อมูล 60 เฟรมต่อท่าทาง เพื่อบันทึกลักษณะการเคลื่อนไหวได้ครบถ้วนตลอดช่วงเวลาที่ท่าทางเกิดขึ้น ก่อนนำไปวิเคราะห์ด้วยแบบจำลองปัญญาประดิษฐ์ในลำดับถัดไป

4. จัดทำระบบจำแนกท่าทางด้วยปัญญาประดิษฐ์และแสดงผล เพื่อจัดการปัญหาการฟังพาล่ามภาษามือ ระบบใหม่จะนำข้อมูลลำดับเวลาที่เก็บได้ไปประมวลผลด้วยแบบจำลองโครงข่ายประสาทเทียมหลายชั้น ซึ่งเหมาะกับข้อมูลเชิงตัวเลขมิติต่ำจากเซนเซอร์ ทำให้ประมวลผลได้รวดเร็วด้วยทรัพยากรคอมพิวเตอร์ที่จำกัด แล้วแสดงผลค่าแปลเป็นข้อความบนเว็บแอปพลิเคชันทันที ทำให้ผู้พิการทางการได้ยินสื่อสารกับบุคคลทั่วไปได้ด้วยตนเองโดยไม่ต้องรอล่าม

3.3 การออกแบบวงจรฮาร์ดแวร์ (Hardware Circuit Design)



รูปที่ 3.1 การออกแบบวงจร

จากรูปที่ 3.1 เนื่องจาก Flex Sensor ให้ค่าออกมาเป็นความต้านทาน ซึ่งไมโครคอนโทรลเลอร์ไม่สามารถอ่านค่าได้โดยตรง จึงต้องใช้วงจรแบ่งแรงดัน (โดยนำ Flex Sensor ต่ออนุกรมกับตัว

ต้านทานคงที่ขนาด 10 k Ω และต่อจุดกึ่งกลางเข้ากับขา Analog-to-Digital Converter ของ ESP32 และกำหนดค่าสัญญาณของ ESP32 ดังตารางที่ 3.1

ตารางที่ 3.1 รายละเอียดขาของ Flex Sensor

อุปกรณ์	หน้าที่	ขา ESP 32
Flex Sensor 1	นิ้วโป้ง	GPIO 36 (VP)
Flex Sensor 2	นิ้วชี้	GPIO 39 (VN)
Flex Sensor 3	นิ้วกลาง	GPIO 34
Flex Sensor 4	นิ้วนาง	GPIO 35
Flex Sensor 5	นิ้วก้อย	GPIO 32

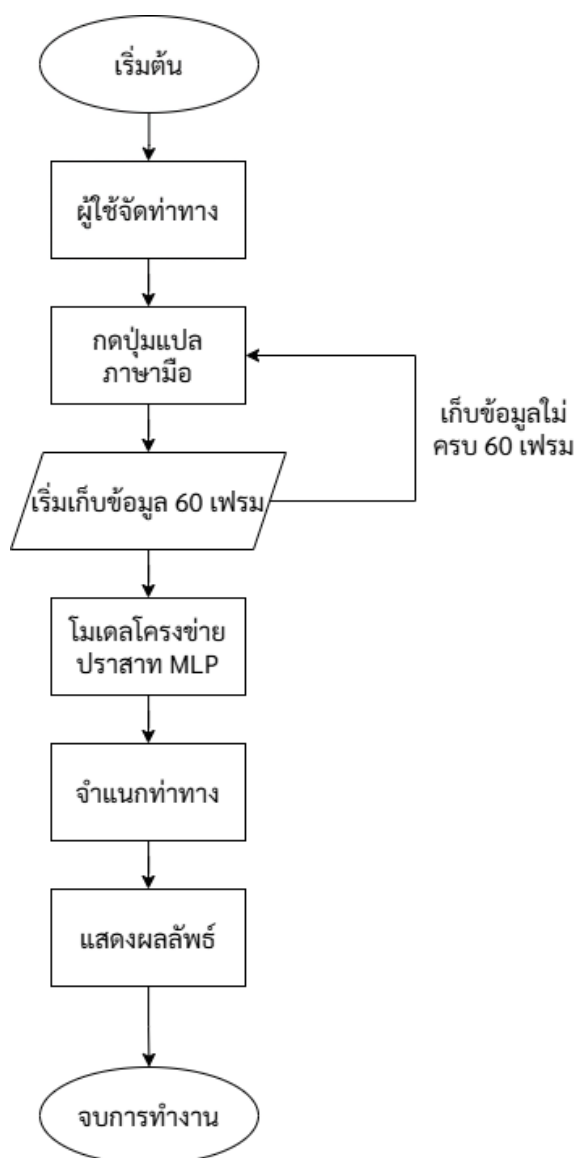
การเชื่อมต่อ MPU6050 เซนเซอร์ MPU6050 ส่งข้อมูลแบบดิจิทัลผ่านโปรโตคอล I2C จึงทำการเชื่อมต่อกับขา GPIO ของ ESP32 ดังตารางที่ 3.2

ตารางที่ 3.2 รายละเอียดขาของ MPU 6050

อุปกรณ์	หน้าที่	ขา ESP 32
MPU6050	I2C Data	GPIO 21
MPU6050	I2C Clock	GPIO 22

3.4 Flowchart การทำงานของระบบ

ขั้นตอนการดำเนินงานของระบบถูกออกแบบมาเพื่อให้เกิดความแม่นยำตั้งแต่การตรวจวัดไปจนถึงการแสดงผล



รูปที่ 3.2 Flowchart การทำงานของระบบ

จากรูปที่ 3.2 ระบบเริ่มดำเนินการทำงานเมื่อผู้ใช้จัดทำทางมือตามภาษามือที่ต้องการสื่อสาร จากนั้นผู้ใช้จะกดปุ่มแปลภาษามือ เพื่อเริ่มกระบวนการแปลผล

เมื่อกดปุ่มแล้ว ระบบจะเริ่มเก็บข้อมูล 60 เฟรม จากเซนเซอร์บนถุงมือ (Flex Sensor และ MPU6050) อย่างต่อเนื่อง ในจุดนี้มีเงื่อนไขตรวจสอบว่า หากเก็บข้อมูลยังไม่ครบ 60 เฟรม ระบบจะวนกลับไปเก็บข้อมูลเพิ่มจนกว่าจะครบจำนวนที่กำหนด (ดูด้านขวาของ Flowchart)

เมื่อเก็บข้อมูลครบ 60 เฟรมแล้ว ข้อมูลจะถูกส่งเข้าสู่โมเดลโครงข่ายประสาท MLP (Multi-Layer Perceptron) เพื่อประมวลผลและจำแนกท่าทาง ว่าตรงกับตัวอักษรหรือคำใดในชุดข้อมูลที่ฝึกไว้

จากนั้นระบบจะแสดงผลลัพธ์ เป็นข้อความบนหน้าจอ (ผ่าน Web Application) ให้ผู้ใช้เห็นทันที และจบการทำงานของรอบนั้น

3.5 ขั้นตอนการดำเนินงาน

โดยได้แบ่งการดำเนินงานเป็น 2 ส่วน ได้แก่ การดำเนินงานในส่วนของ Hardware และการดำเนินงานในส่วนของ Software

3.5.1 การดำเนินงานในส่วน Hardware

สามารถแบ่งได้ 2 ส่วนดังนี้

1. ชิ้นส่วนของถุงมือแปลภาษามือข้างเดียว โดยประกอบด้วย

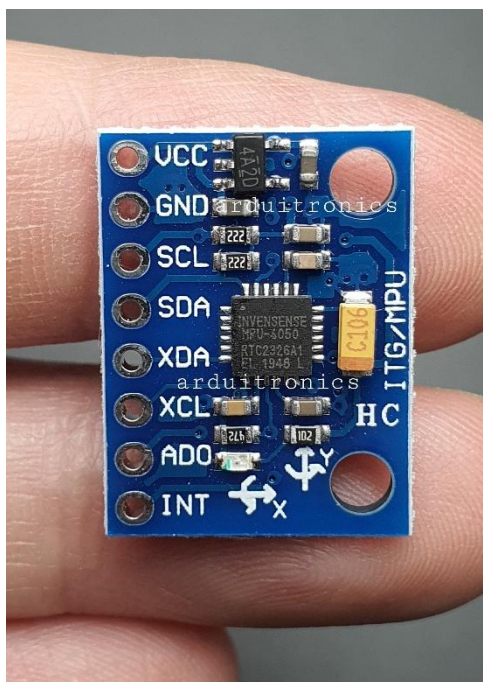
- Flex Sensor เซนเซอร์วัดการโค้งงอ (Flex Sensor) จัดเป็นเซนเซอร์ในกลุ่ม Analog Sensor ที่ทำหน้าที่แปลงการเปลี่ยนแปลงทางกลศาสตร์ หรือการบิดงอทางกายภาพ ให้กลายเป็นการเปลี่ยนแปลงทางไฟฟ้า โดยตัวเซนเซอร์มีคุณลักษณะเป็น ตัวต้านทานแปรค่าตามแรงกล (Variable Resistor) ซึ่งค่าความต้านทานภายในจะแปรผันตรงกับระดับองศาของความโค้งงอ มีลักษณะ ดังรูปที่ 3.3



รูปที่ 3.3 Flex Sensor

- MPU 6050 คือโมดูลเซนเซอร์ตรวจจับการเคลื่อนไหวประเภท IMU (Inertial Measurement Unit) ที่รวมเอาเซนเซอร์วัดความเร่ง (Accelerometer) และเซนเซอร์วัดความเร็ว

เชิงมุม (Gyroscope) เข้าไว้ด้วยกันบนชิปซิลิคอนขนาดเล็กเพียงตัวเดียว โดยใช้เทคโนโลยีโครงสร้างเครื่องกลไฟฟ้าจุลภาค หรือ MEMS (Micro-Electro-Mechanical Systems) โมดูลนี้มีความสามารถในการตรวจจับการเคลื่อนไหวและทิศทางในอวกาศได้ทั้งหมด 6 แกนอิสระ (6 Degrees of Freedom: 6-DOF) มีลักษณะ ดังรูปที่ 3.4



รูปที่ 3.4 MPU 6050

- ESP 32 เป็นชิปไมโครคอนโทรลเลอร์ประเภทระบบบนชิปเดียว หรือ SoC (System on a Chip) ขนาด 32 บิต มีสมรรถนะการประมวลผลความเร็วสูง มีหน่วยความจำภายในที่เพียงพอ และที่สำคัญคือมีโมดูลการสื่อสารไร้สาย (Wireless Connectivity) ทั้ง Wi-Fi และ Bluetooth มีลักษณะ ดังรูปที่ 3.5



รูปที่ 3.5 ESP32

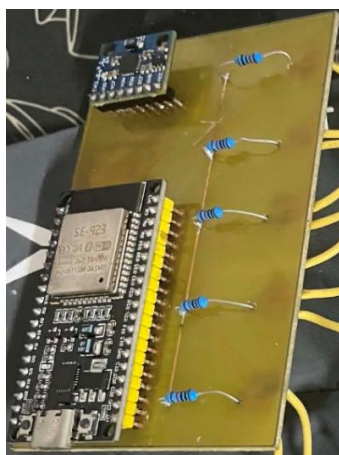
2. ขั้นตอนการประกอบระบบถุงมือแปลภาษามือข้างเดียวเริ่มต้นจากการติดตั้ง Flex Sensor จำนวน 5 ตัว ลงบนถุงมือบริเวณแนวของนิ้วมือแต่ละนิ้วเพื่อตรวจวัดระดับการโค้งงอ จากนั้นจึงดำเนินการติดตั้งเซนเซอร์ MPU6050 และไมโครคอนโทรลเลอร์ ESP32 ลงบนบอร์ดวงจร (PCB) ที่ถูกออกแบบมาโดยเฉพาะ และติดตั้งชุดอุปกรณ์ทั้งหมดไว้บริเวณหลังมือ เพื่อให้ผู้ใช้งานสามารถเคลื่อนไหวได้อย่างอิสระ โดยขั้นตอนสุดท้ายคือการเชื่อมต่อวงจรไฟฟ้าทั้งหมดด้วยวิธีการบัดกรี เพื่อให้มั่นใจในความมั่นคงแข็งแรงของจุดเชื่อมต่อและประสิทธิภาพในการรับ-ส่งสัญญาณไฟฟ้าตลอดการใช้งาน

- นำ Flex Sensor ทั้ง 5 ตัวมาติดตั้งลงบนนิ้วแต่ละนิ้วดังรูปที่ 3.7



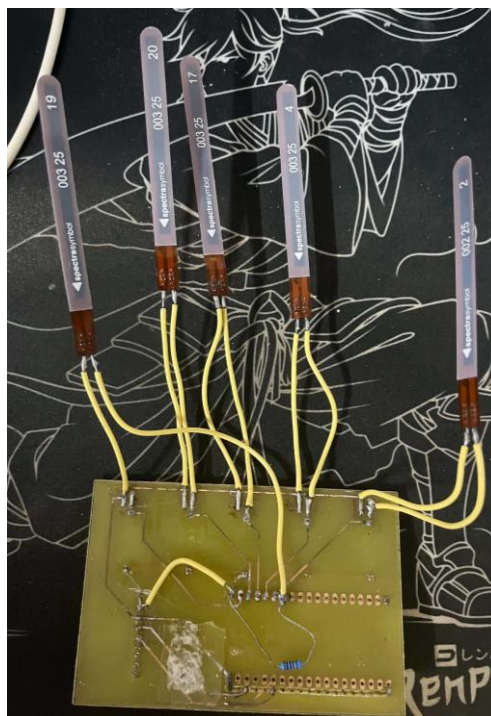
รูปที่ 3.6 การติดตั้ง Flex Sensor

- ติดตั้งเซนเซอร์ MPU6050 และไมโครคอนโทรลเลอร์ ESP32 ลงบนบอร์ด PCB ดังรูปที่ 3.7



รูปที่ 3.7 ติดตั้ง ESP32 และ MPU6050 ลงบนบอร์ด

- ทำการเชื่อม Flex Sensor และ MPU6050 เข้ากับขาของ ESP32 ที่ต้องการใช้ ด้วยวิธีการบัดกรี ดังรูปที่ 3.8



รูปที่ 3.8 บัดกรีอุปกรณ์ทั้งหมด

3.5.2 การดำเนินงานในส่วนของ Software

ในส่วนของการซอฟต์แวร์ ระบบถุงมือแปลภาษามือข้างเดียวถูกพัฒนาด้วยภาษา Python เป็นหลัก โดยแบ่งการทำงานออกเป็นสองส่วนหลัก คือ ฟังก์ชันไมโครคอนโทรลเลอร์และฟังก์ชันคอมพิวเตอร์ ดังนี้

- โปรแกรมฝั่ง ESP32 พัฒนาด้วย Arduino IDE (ภาษา C++) ทำหน้าที่อ่านค่าจาก Flex Sensor และเซนเซอร์ MPU6050 จากนั้นทำการจัดรูปแบบข้อมูลเป็น JSON และส่งผ่านโปรโตคอล UDP ไปยังคอมพิวเตอร์ด้วยอัตราการส่งข้อมูล 60 เฟรมต่อวินาที

- โปรแกรมรับข้อมูลและประมวลผลฝั่งคอมพิวเตอร์ พัฒนาด้วยภาษา Python ร่วมกับไลบรารี socket เพื่อรับข้อมูลผ่าน UDP Port 4210 และจัดเก็บข้อมูลในรูปแบบ Sliding Window ขนาด 660 มิติ

- การจัดการข้อมูล ใช้ไลบรารี NumPy และ Pandas ในการจัดการและแปลงข้อมูลเชิงตัวเลขให้อยู่ในรูปแบบที่เหมาะสมก่อนนำเข้าสู่โมเดลปัญญาประดิษฐ์

- การสร้างและฝึกสอนแบบจำลอง ใช้ไลบรารี Scikit-Learn สำหรับสร้างและฝึกสอนแบบจำลอง MLP รวมถึงทำ Feature Scaling ด้วย StandardScaler พร้อมบันทึกผลลัพธ์เป็นไฟล์โมเดล (model.pkl) และไฟล์ตัวปรับสเกล (scaler.pkl)
- การแสดงผล พัฒนาเว็บแอปพลิเคชันด้วยเฟรมเวิร์ก Flask เพื่อรับผลการทำนายจากโมเดลและแสดงผลข้อความแปลท่าทางแบบเรียลไทม์ผ่านหน้าเว็บเบราว์เซอร์
- การทดสอบระบบ จัดทำสคริปต์ตรวจสอบผ่าน Terminal แยกต่างหาก เพื่อใช้สำหรับการทดสอบความถูกต้องของโมเดลโดยไม่ต้องผ่านหน้าเว็บแอปพลิเคชัน

3.6 การสร้าง Dataset

เนื่องจากไม่มีชุดข้อมูลมาตรฐานที่ตรงกับฮาร์ดแวร์ที่พัฒนาขึ้น คณะผู้จัดทำจึงดำเนินการสร้างชุดข้อมูลขึ้นใหม่ โดยมีรายละเอียดดังนี้

3.6.1 โครงสร้างของชุดข้อมูล

ชุดข้อมูลสำหรับโครงงานนี้ประกอบด้วยข้อมูลจากเซนเซอร์ต่างๆ ที่อ่านค่าจากถุงมือ พร้อมทั้งป้ายกำกับ (Label) ของท่าภาษามือที่ต้องการแปล โดยโครงสร้างข้อมูลถูกออกแบบมาให้เหมาะสมกับการประมวลผลและการฝึกสอนโมเดล (Model Training) ดังแสดงในตารางที่ 3.3

ตารางที่ 3.3 โครงสร้างชุดข้อมูล

ชื่อคอลัมน์	รายละเอียด
label	ป้ายชื่อท่าภาษามือ (A-Z / คำ)
thumb / โป้ง	ค่า Flex Sensor ของนิ้วโป้ง
index / ชี้	ค่า Flex Sensor ของนิ้วชี้
middle / กลาง	ค่า Flex Sensor ของนิ้วกลาง
ring / นาง	ค่า Flex Sensor ของนิ้วนาง
pinky / ก้อย	ค่า Flex Sensor ของนิ้วก้อย

ตารางที่ 3.3 โครงสร้างชุดข้อมูล (ต่อ)

ชื่อคอลัมน์	รายละเอียด
acc_x	ค่า Accelerometer แกน X
acc_y	ค่า Accelerometer แกน Y
acc_z	ค่า Accelerometer แกน Z
gyro_x	ค่า Gyroscope แกน X
gyro_y	ค่า Gyroscope แกน Y
gyro_z	ค่า Gyroscope แกน Z

3.6.2 การเตรียมอุปกรณ์และซอฟต์แวร์

- การจัดทำฮาร์ดแวร์ ประกอบถุงมือที่ติดตั้ง Flex Sensor จำนวน 5 ตัว บริเวณนิ้วมือ และเซนเซอร์ MPU6050 โดยเชื่อมต่อเข้ากับไมโครคอนโทรลเลอร์ ESP32
- การตั้งค่าเครือข่าย กำหนดให้ ESP32 เชื่อมต่อกับเครือข่าย Wi-Fi เดียวกันกับเครื่องคอมพิวเตอร์ และทำหน้าที่เป็นตัวส่งข้อมูลผ่านโปรโตคอล UDP ไปยังพอร์ต 4210
- การเตรียมซอฟต์แวร์ส่วนรับข้อมูล พัฒนาด้วยภาษา Python ร่วมกับไลบรารี socket เพื่อทำหน้าที่เป็นเซิร์ฟเวอร์ในการรับข้อมูลและจัดการจัดเก็บชุดข้อมูลแบบกึ่งอัตโนมัติ

3.6.3 ขั้นตอนการเก็บรวบรวมชุดข้อมูล

ขั้นตอนการสร้างชุดข้อมูลเริ่มต้นโดยให้ผู้สวมถุงมือจัดทำท่าทางภาษามือที่ต้องการเก็บข้อมูล และคงท่าทางนั้นไว้อย่างนิ่งที่สุด จากนั้นผู้เก็บข้อมูลจะดำเนินการกดปุ่มบันทึก (Save) ผ่านโปรแกรมบนคอมพิวเตอร์

เมื่อเริ่มต้นบันทึก ระบบจะดำเนินการอ่านค่าจาก Flex Sensor ทั้ง 5 ตัว และเซนเซอร์ MPU6050 อย่างต่อเนื่องจนครบจำนวน 60 เฟรม โดยตลอดระยะเวลาการบันทึก ผู้สวมถุงมือจะต้องรักษาท่าทางเดิมไว้ไม่ให้เกิดการเปลี่ยนแปลง เพื่อให้ข้อมูลที่อ่านได้ในแต่ละเฟรมมีความสม่ำเสมอและสะท้อนถึงท่าทางนั้นได้อย่างถูกต้องแม่นยำ เมื่อระบบทำการบันทึกข้อมูลครบทั้ง 60 เฟรม ข้อมูลทั้งหมดจะถูกจัดเก็บลงในไฟล์รูปแบบ JSON พร้อมระบุป้ายกำกับ (Label) ของท่าทางนั้นโดยอัตโนมัติ ทั้งนี้ ขั้นตอนดังกล่าวจะถูกดำเนินการซ้ำหลายครั้งสำหรับทุกท่าทาง เพื่อให้ได้ชุดข้อมูลที่มี

ความหลากหลายและมีปริมาณเพียงพอสำหรับการฝึกสอนโมเดลปัญญาประดิษฐ์ให้มีความแม่นยำสูงสุด ดังรูปที่ 3.9



รูปที่ 3.9 ขั้นตอนการเก็บข้อมูล

3.6.4 การวิเคราะห์โครงสร้างโปรแกรมเก็บข้อมูล

โปรแกรมรับข้อมูลถูกออกแบบให้ทำงานแบบ Interactive เพื่อให้ผู้เก็บข้อมูลสามารถตรวจสอบความถูกต้องของท่าทางก่อนบันทึกได้ โดยมีส่วนประกอบหลักดังนี้

- การตั้งค่าการเชื่อมต่อ UDP

```
UDP_IP = "0.0.0.0"
UDP_PORT = 4210
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((UDP_IP, UDP_PORT))
```

รูปที่ 3.10 โค้ดตั้งค่า UDP

จากรูปที่ 3.10 ทำหน้าที่เปิดพอร์ตสื่อสารในเครื่องคอมพิวเตอร์เพื่อรอรับแพ็กเก็ตข้อมูลจาก ESP32 โดยใช้โปรโตคอล UDP

- กระบวนการล้างข้อมูลค้างคา

```
sock.setblocking(False)
while True:
    try:
        sock.recv(2048)
    except:
        break
```

รูปที่ 3.11 โค้ดล้าง Buffer

จากรูปที่ 3.11 ทำหน้าที่ป้องกันไม่ให้ข้อมูลเก่าที่ตกค้างอยู่ใน Buffer ของเครือข่ายถูกนำมาบันทึก ระบบจะล้างข้อมูลที่ค้างอยู่ในคิวทิ้งทั้งหมดก่อนจะเริ่มบันทึกข้อมูล 60 เฟรมใหม่ในทุกครั้งที่กดปุ่ม

- การรวบรวมข้อมูลช่วงเวลา

```
while len(sequence_buffer) < TARGET_FRAMES:
    data, addr = sock.recvfrom(2048)
    raw_data = json.loads(data.decode('utf-8'))
    frame_vector = raw_data['flex'] + raw_data['acc'] + raw_data['gyro']
    sequence_buffer.append(frame_vector)
```

รูปที่ 3.12 โค้ดรวบรวมข้อมูล

จากรูปที่ 3.12 ระบบจะทำการสะสมข้อมูลต่อเนื่องจำนวน 60 เฟรม โดยในแต่ละเฟรมประกอบด้วยพารามิเตอร์ 11 ค่า

(นิ้วมือ 5, ความเร่ง 3, ความเร็วเชิงมุม 3) เมื่อบันทึกครบถ้วน ข้อมูลชุดนี้จะถูกมองเป็น 1 ตัวอย่าง ของท่าทางนั้นๆ

- การจัดเก็บข้อมูลในรูปแบบ JSON

```
record = {
    "label": current_label,
    "data": sequence_buffer,
    "timestamp": datetime.datetime.now().isoformat()
}
```

รูปที่ 3.13 โค้ดการจัดเก็บข้อมูล

จากรูปที่ 3.13 ข้อมูลจะถูกบันทึกในรูปแบบไฟล์ JSON เพื่อให้สามารถเก็บโครงสร้างข้อมูลแบบมิติ ได้อย่างสมบูรณ์ ซึ่งง่ายต่อการนำไปฝึกสอนโมเดลปัญญาประดิษฐ์ในขั้นตอนถัดไป

3.6.5 รูปแบบการเก็บข้อมูล

เพื่อให้ข้อมูลที่รวบรวมได้มีความเป็นระเบียบและสามารถนำไปใช้ในการฝึกสอนโมเดลปัญญาประดิษฐ์ได้อย่างมีประสิทธิภาพ คณะผู้จัดทำจึงกำหนดรูปแบบการจัดเก็บข้อมูลในลักษณะของโครงสร้างข้อมูลเชิงมิติ (Multi-dimensional Data Structure) โดยบันทึกอยู่ในรูปแบบไฟล์ JSON ซึ่งแต่ละท่าทางจะถูกแทนด้วยลำดับข้อมูลต่อเนื่องจำนวน 60 เฟรม โดยในแต่ละเฟรมจะประกอบด้วยค่าจากเซนเซอร์ทั้งหมด 11 พารามิเตอร์ ได้แก่ ค่าการไค้งงของนิ้วมือ 5 ค่า (นิ้วโป้ง นิ้วชี้ นิ้วกลาง นิ้วนาง และนิ้วก้อย) ค่าความเร่งเชิงเส้น 3 แกน (acc_x, acc_y, acc_z) และค่าความเร็วเชิงมุม 3 แกน ($gyro_x, gyro_y, gyro_z$) ส่งผลให้ข้อมูล 1 ตัวอย่างของแต่ละท่าทางมีขนาดรวมทั้งสิ้น 660 ค่า (60 เฟรม $\times$ 11 พารามิเตอร์) ซึ่งสอดคล้องกับขนาดของข้อมูลนำเข้า (Input) ของโมเดล MLP

ชุดข้อมูลที่เก็บรวบรวมครอบคลุมท่าทางทั้งหมด 31 ท่า แบ่งเป็นตัวอักษรภาษามือ A-Z จำนวน 26 ท่า และคำศัพท์พื้นฐานจำนวน 5 คำ ได้แก่ "Hello", "Thank you", "I love you", "Yes" และ "No" เพื่อให้เห็นลักษณะของข้อมูลในแต่ละมิติได้อย่างเป็นรูปธรรม คณะผู้จัดทำจึงได้จำแนกตัวอย่างค่าเฉลี่ยของข้อมูลเซนเซอร์ออกเป็น 3 กลุ่มตามแหล่งที่มา ได้แก่ ค่าจาก Flex Sensor, ค่าความเร่ง (Accelerometer) และค่าความเร็วเชิงมุม (Gyroscope)

1. ข้อมูลจาก Flex Sensor

ตารางที่ 3.4 รูปแบบการเก็บข้อมูล Flex Sensor

ท่าทาง	เซนเซอร์นิ้วมือ (Flex Sensors)				
	โป้ง	ชี้	กลาง	นาง	ก้อย
A	2,873.82	3,355.90	3,574.78	3,845.30	3,233.38
B	3,052.93	3,063.35	3,140.49	3,540.20	2,985.28
C	2,949.79	3,276.07	3,559.56	3,796.72	3,146.26
D	2,990.20	3,071.93	3,563.95	3,812.69	3,225.54
E	3,109.88	3,518.75	3,638.06	3,867.02	3,280.54
F	2,832.86	3,336.45	3,101.11	3,507.59	2,986.48

ตารางที่ 3.4 รูปแบบการเก็บข้อมูล Flex Sensor (ต่อ)

ท่าทาง	เซนเซอร์นิ้วมือ (Flex Sensors)				
	โป่ง	ชี้	กลาง	นาง	ก้อย
G	2,878.92	3,051.68	3,596.28	3,852.72	3,226.26
H	2,905.95	3,063.96	3,151.21	3,682.26	3,185.40
I	3,016.37	3,275.49	3,526.54	3,764.58	3,000.64
J	2,850.18	3,318.53	3,508.81	3,762.12	3,002.86
K	2,841.74	3,059.03	3,087.01	3,773.40	3,243.21
L	2,881.51	3,078.98	3,603.34	3,849.00	3,321.07
M	3,057.60	3,352.15	3,555.68	3,783.19	3,387.73
N	2,703.23	3,242.16	3,422.70	3,732.40	3,275.20
O	2,918.97	3,256.05	3,502.07	3,760.96	3,136.24
P	2,796.76	3,021.38	3,273.37	3,695.00	3,224.21
Q	2,856.24	3,059.54	3,613.06	3,889.77	3,291.62
R	2,934.35	3,063.56	3,062.48	3,737.01	3,298.76
S	3,013.35	3,326.97	3,545.87	3,844.36	3,268.52
T	2,762.65	3,197.55	3,508.58	3,782.88	3,185.25
U	2,905.03	3,026.24	3,086.91	3,738.34	3,225.94
V	2,940.77	3,009.22	3,128.45	3,677.27	3,312.79
W	2,940.40	3,003.11	3,119.03	3,517.94	3,414.34
X	3,090.03	3,142.96	3,534.00	3,822.33	3,231.05
Y	2,826.27	3,352.91	3,495.19	3,730.11	2,972.61
Z	2,852.07	3,019.54	3,566.06	3,862.13	3,264.37
Hello	2,879.85	3,012.99	3,095.23	3,479.20	2,948.84
Thank you	2,898.86	3,014.03	3,090.69	3,471.60	2,946.17
I love you	2,880.26	3,009.43	3,587.68	3,823.71	2,965.93
Yes	2,898.62	3,376.16	3,575.92	3,860.96	3,278.20
No	2,819.20	3,029.03	3,173.90	3,695.15	3,192.54

จากตารางที่ 3.4 ค่าที่ได้จาก Flex Sensor ทั้ง 5 ตัว (นิ้วโป้ง นิ้วชี้ นิ้วกลาง นิ้วนาง และ นิ้วก้อย) คือค่าจากตัวแปลงสัญญาณแอนะล็อกเป็นดิจิทัล (Analog-to-Digital Converter: ADC) ซึ่งอ่านค่าจากความต้านทานของเซนเซอร์ โดยค่าดังกล่าวจะแปรผันตรงกับมุมการโค้งงอของนิ้วมือแต่ละนิ้วในขณะที่ทำท่าทาง กล่าวคือ ยิ่งนิ้วมือมีความโค้งงอมากเท่าใด ค่าความต้านทานจะยิ่งสูงขึ้น ส่งผลให้ค่าดิจิทัลที่อ่านได้เพิ่มสูงขึ้นตามไปด้วย

2. ข้อมูลความเร่ง

ตารางที่ 3.5 รูปแบบการเก็บข้อมูลความเร่ง Accelerometer ของ MPU6050

ท่าทาง	ความเร่ง IMU		
	acc_x	acc_y	acc_z
A	1.2303	-0.4355	10.9341
B	1.1046	-0.2868	8.8621
C	1.057	-1.4877	13.7603
D	0.2665	-0.2465	11.2237
E	1.4212	-0.3912	9.8609
F	-0.0876	-0.0954	10.7618
G	-10.7934	-7.1609	11.6341
H	-10.7684	-8.0373	12.6491
I	1.4813	-1.06	12.8161
J	-3.8264	-2.0127	14.8443
K	0.3276	-0.2218	11.078
L	-0.0305	-0.3798	12.0647
M	0.3244	-0.0818	10.9798
N	0.7596	-0.3363	12.2946
O	0.3665	-0.5364	13.1483
P	-2.9997	-12.7015	19.9146
Q	-1.7469	-11.3437	20.3309
R	0.4726	-0.3205	12.4138
S	0.8325	-0.8804	13.9091

ตารางที่ 3.5 รูปแบบการเก็บข้อมูลความเร่ง Accelerometer ของ MPU6050 (ต่อ)

ท่าทาง	ความเร่ง IMU		
	acc_x	acc_y	acc_z
T	1.3326	-0.4539	11.7879
U	0.5702	-0.1477	11.1951
V	0.6512	-0.4597	12.8372
W	0.3136	-0.0977	11.0892
X	-0.3209	-0.7694	14.298
Y	-7.5402	-4.3572	16.1846
Z	-1.5599	-4.8767	18.7357
Hello	-3.6685	-3.5782	16.8267
Thank you	-7.3929	-2.7209	8.0399
I love you	1.2899	-0.6759	12.8822
Yes	-0.8273	-6.7056	18.3804
No	1.2835	-1.6741	15.1687

จากตารางที่ 3.5 ค่า acc_x , acc_y และ acc_z คือค่าความเร่งเชิงเส้นที่วัดได้จากเซนเซอร์ MPU6050 ทั้ง 3 แกน ซึ่งบ่งบอกถึงทิศทางการจัดวางมือเมื่อเทียบกับแรงโน้มถ่วงของโลก (Gravity) ในขณะทำท่าทาง โดยทั่วไปท่าทางที่มีการพลิกข้อมือหรือการวางมือในมุมที่แตกต่างกันจะส่งผลให้ค่าความเร่งมีความแตกต่างกันอย่างชัดเจน เช่น ท่าทาง "G" ซึ่งมีค่า acc_x และ acc_y เป็นลบในระดับสูง แสดงถึงการพลิกข้อมือไปในทิศทางที่แตกต่างจากท่าทางอื่นอย่างเห็นได้ชัด ในขณะที่ท่าทาง "A" และ "Yes" มีค่าความเร่งใกล้เคียงกับการวางมือในแนวปกติ

3. ข้อมูลความเร็วเชิงมุม

ตารางที่ 3.6 รูปแบบการเก็บข้อมูลความเร็วเชิงมุม Gyroscope ของ MPU6050

ท่าทาง	ความเร็วเชิงมุม IMU		
	gyro_x	gyro_y	gyro_z
A	-0.0003	0.0053	-0.0041
B	0.0046	0.0024	-0.0071

ตารางที่ 3.6 รูปแบบการเก็บข้อมูลความเร็วเชิงมุม Gyroscope ของ MPU6050 (ต่อ)

ท่าทาง	ความเร็วเชิงมุม IMU		
	gyro_x	gyro_x	gyro_x
C	0.0007	0.0032	-0.0048
D	0.0039	0.0027	-0.006
E	0.0042	0.0025	-0.0067
F	0.0039	-0.0026	-0.007
G	0.0023	-0.003	-0.0091
H	0.0038	-0.0058	-0.0104
I	0.0043	0.0001	-0.0078
J	-0.0936	0.1657	-0.0183
K	0.0051	0	-0.0087
L	0.0054	-0.0003	-0.0079
M	-0.0049	-0.0273	-0.0111
N	-0.0066	-0.0296	-0.0121
O	-0.0072	-0.0197	-0.0082
P	-0.006	-0.0297	-0.0099
Q	-0.0058	-0.0283	-0.0096
R	0.0218	-0.0161	-0.0023
S	0.022	-0.0155	-0.0002
T	0.0212	-0.0169	-0.0011
U	0.0194	-0.0171	-0.0004
V	0.0208	-0.0152	-0.0011
W	0.0223	-0.0152	-0.0013
X	0.0223	-0.0178	-0.0019
Y	0.0215	-0.0207	-0.003
Z	-0.1064	0.0154	-0.1265
Hello	-0.0963	-0.0275	-0.3307
Thank you	0.3101	-0.0815	-0.1568
I love you	0.0228	-0.0173	-0.0016

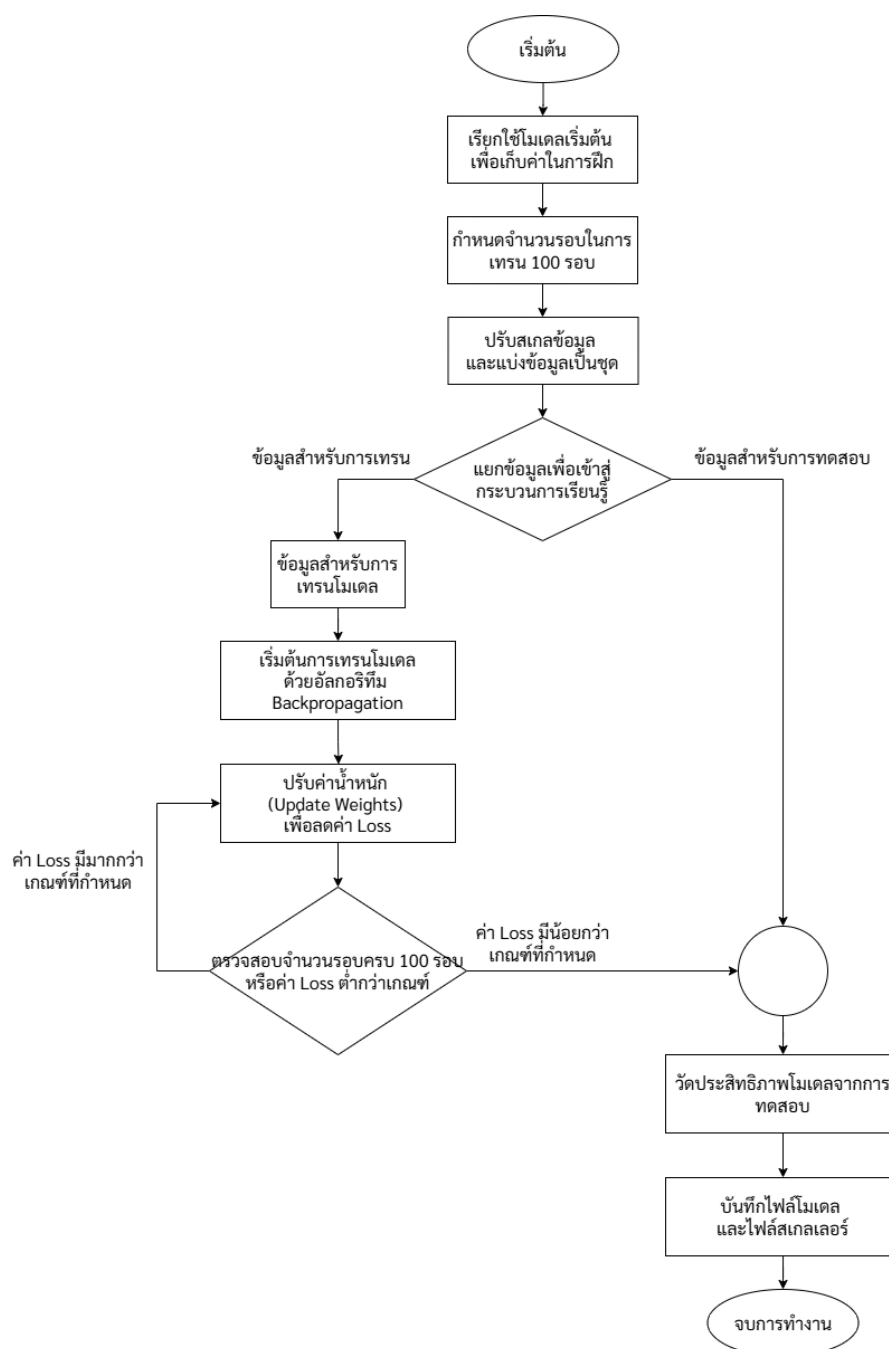
ตารางที่ 3.6 รูปแบบการเก็บข้อมูลความเร็วเชิงมุม Gyroscope ของ MPU6050 (ต่อ)

ท่าทาง	ความเร็วเชิงมุม IMU		
	gyro_x	gyro_y	gyro_z
Yes	-0.3985	0.0724	-0.0266
No	-0.0176	-0.0121	0.0176

จากตารางที่ 3.6 ค่า $gyro_x$, $gyro_y$ และ $gyro_z$ คือค่าความเร็วเชิงมุมที่วัดได้จากเซนเซอร์ MPU6050 ทั้ง 3 แกน ซึ่งบ่งบอกถึงอัตราการหมุนหรือการเคลื่อนไหวของข้อมือในขณะท่าทาง ค่าดังกล่าวมีความสำคัญอย่างยิ่งสำหรับการแยกแยะท่าทางแบบเคลื่อนไหว (Dynamic Gestures) เช่น ท่าทาง "J" และ "Z" ที่ต้องมีการหมุนข้อมือวาดเป็นรูปตัวอักษรในอากาศ ซึ่งจะให้ค่าจากไจโรสโคป (Gyroscope) ที่สูงกว่าท่าทางแบบหยุดนิ่ง (Static Gestures) อย่างเห็นได้ชัด

3.7 Flowchart การเทรนโมเดล

การทำงานส่วนการเทรนโมเดลด้วย MLP (Multi-layer Perceptron) เป็นส่วนของการสร้างโครงข่ายประสาทเทียมจากข้อมูลเซนเซอร์ที่บันทึกไว้ใน Dataset เพื่อให้โมเดลที่ได้สามารถใช้ในการจำแนกท่าทางภาษามือ โดยมีการนำค่าจาก Flex Sensor และ MPU6050 ที่จัดเก็บในรูปแบบ JSON มาผ่านกระบวนการ Standardization ด้วย StandardScaler จากนั้นจึงทำการฝึกสอนโมเดลด้วยอัลกอริทึม Backpropagation จนกระทั่งได้ไฟล์โมเดลที่ผ่านการฝึกสอนแล้วและไฟล์ Scaler ซึ่งจะนำไปใช้ในขั้นตอนการประมวลผล ต่อไป ทั้งนี้ระบบยังทำการประเมินค่าความถูกต้อง (Accuracy) และพล็อตกราฟ (Plot Graph) ซึ่งเป็นการทำ Data Visualization เพื่อให้สามารถทดสอบเปรียบเทียบ และประเมินค่าความถูกต้องของโมเดลที่สร้างขึ้นว่ามีแนวโน้มที่ดีหรือไม่



รูปที่ 3.14 Flowchart การเทรนโมเดล

จากรูปที่ 3.4 สามารถอธิบายลำดับขั้นตอนการประมวลผลและการเรียนรู้ของแบบจำลองโครงข่ายประสาทเทียม (Neural Network) ตั้งแต่เริ่มต้นกระบวนการเตรียมข้อมูลไปจนถึงการบันทึกผลเพื่อนำไปใช้งานจริง โดยมีรายละเอียดดังนี้

3.8 การออกแบบโมเดล

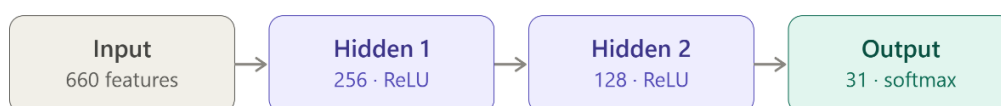
จากแนวทางที่วิเคราะห์ระบบเดิมไว้ เก็บข้อมูลท่าทางแบบลำดับเวลาและจำแนกด้วยปัญญาประดิษฐ์ ผู้จัดทำจึงออกแบบแบบจำลองโครงข่ายประสาทเทียมชนิด Multi-Layer Perceptron (MLP) ขึ้น เนื่องจากข้อมูลนำเข้าเป็นข้อมูลเชิงตัวเลขจากเซนเซอร์ที่แปลงเป็นเวกเตอร์คุณลักษณะแล้ว ไม่ใช่ข้อมูลภาพ จึงไม่จำเป็นต้องใช้โครงข่ายประสาทเทียมแบบคอนโวลูชันซึ่งต้องการทรัพยากรฝึกสอนสูงกว่า การออกแบบแบ่งออกเป็นส่วนตัวต่าง ๆ ที่เชื่อมโยงกัน ดังนี้

1. Input Layer เพื่อรองรับข้อมูลลำดับเวลาที่เก็บไว้ มีขนาด 660 มิติ ซึ่งได้จากเซนเซอร์ 11 พารามิเตอร์ต่อเฟรม คูณด้วยความยาวลำดับเวลา 60 เฟรม ทำให้แบบจำลองรับรู้ลักษณะการเคลื่อนไหวของท่าทางได้ครบถ้วน

2. Hidden Layer เพื่อให้ประมวลผลได้รวดเร็ว Hidden Layer ออกแบบเป็น 2 ชั้น ขนาด 256 และ 128 นิวรอนตามลำดับ ลดลงแบบ Funnel เพื่อให้โครงข่ายเรียนรู้คุณลักษณะเชิงลึกในชั้นแรก แล้วค่อยสรุปคุณลักษณะให้กระชับก่อนส่งต่อไปยังชั้นถัดไป ขนาดดังกล่าวได้จากการทดลองเปรียบเทียบหลายขนาด โดยพิจารณาความแม่นยำและเวลาฝึกสอนที่เหมาะสมร่วมกัน และใช้ฟังก์ชันกระตุ้น ReLU เพื่อลดปัญหา Vanishing Gradient และคำนวณได้เร็วกว่า Sigmoid หรือ Tanh

3. Output Layer เพื่อจำแนกท่าทางและแสดงผลแบบเรียลไทม์ Output Layer มี 31 หน่วย ตรงกับจำนวนคลาสท่าทางทั้งหมด คือตัวอักษร A ถึง Z จำนวน 26 ท่า และคำศัพท์พื้นฐาน 5 คำ ได้แก่ Hello, Thank you, I love you, Yes และ No โดยใช้ฟังก์ชัน Softmax แปลงค่าดิบจากนิวรอนทั้ง 31 ตัวให้เป็นค่าความน่าจะเป็นที่ผลรวมเท่ากับ 1 แล้วเลือกคลาสที่มีค่าความน่าจะเป็นสูงสุดเป็นคำตอบ

4. ออกแบบตัวปรับค่าน้ำหนัก (Solver) เพื่อให้แบบจำลองฝึกสอนได้เร็วและแม่นยำสอดคล้องกับเป้าหมายของระบบทั้งหมด เลือกใช้อัลกอริทึม Adam ซึ่งปรับอัตราการเรียนรู้ของแต่ละน้ำหนักโดยอัตโนมัติจากค่าโมเมนต์ของเกรเดียนต์ ทำให้น้ำหนักลู่เข้าจุดที่เหมาะสมได้เร็วกว่าวิธี Stochastic Gradient Descent แบบดั้งเดิม และลดค่าความสูญเสียด้วย Cross-Entropy Loss ในทุกรอบของการฝึกสอนดังรูปที่ 3.15



รูปที่ 3.15 โครงสร้างโมเดล

3.8.1 อธิบายโค้ดการเทรนโมเดล

```
import os
import json
import joblib
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

รูปที่ 3.16 โค้ดสำหรับนำเข้าไลบรารี

จากรูปที่ 3.16 ประกอบด้วยคำสั่งที่ใช้สำหรับการนำเข้าไลบรารีโดยมีรายละเอียดดังมีการนำเข้าไลบรารี

- import os นำเข้าโมดูลสำหรับการจัดการระบบไฟล์และโพลเดอร์ของระบบปฏิบัติการ
- import joblib นำเข้าไลบรารีสำหรับเซฟ และโหลด วัตถุในภาษา Python เช่น ตัวแบบที่ฝึกสอนเสร็จแล้ว หรือตัวปรับระดับข้อมูล
- import numpy as np นำเข้าคลังคำนวณคณิตศาสตร์อาเรย์หลายมิติยอดนิยม โดยตั้งชื่อย่อว่า np
- import matplotlib.pyplot as plt นำเข้าเครื่องมือสำหรับวาดกราฟและแผนภูมิ ตั้งชื่อย่อว่า plt
- from sklearn.model_selection นำเข้าฟังก์ชัน train_test_split เพื่อใช้สุมแบ่งข้อมูลออกเป็นชุดสอน (Train) และชุดทดสอบ (Test)
- from sklearn.preprocessing นำเข้า StandardScaler สำหรับปรับค่าตัวเลขของข้อมูลจากเซนเซอร์ทุกตัวให้อยู่ในช่วงมาตรฐานเดียวกัน
- from sklearn.neural_network นำเข้าโมดูลโครงข่ายประสาทเทียมแบบหลายชั้น MLPClassifier (Multilayer Perceptron)
- from sklearn.metrics นำเข้าตัววัดผลต่าง ๆ ได้แก่ ค่าความแม่นยำรวม (accuracy_score), เมทริกซ์ความสับสน (confusion_matrix), และรายงานประสิทธิภาพจำแนกคลาสอย่างละเอียด (classification_report)

```

DATASET_FILE = 'dynamic_data_60fnew.json'
MODEL_FILE = 'smart_glove_model_dynamic2.pkl'
SCALER_FILE = 'scaler_dynamic2.pkl'

```

รูปที่ 3.17 โค้ดการกำหนดข้อมูลนำเข้า

จากรูปที่ 3.17 เป็นการกำหนดที่อยู่ของข้อมูลที่นำเข้ามา

- DATASET_FILE = 'dynamic_data_60fnew.json' กำหนดชื่อไฟล์ชุดข้อมูลที่บันทึกมาจากถุงมือ
- MODEL_FILE = 'smart_glove_model_dynamic2.pkl' กำหนดชื่อไฟล์สำหรับจัดเก็บโมเดลปัญญาประดิษฐ์ที่เทรนเสร็จแล้ว
- SCALER_FILE = 'scaler_dynamic2.pkl' กำหนดชื่อไฟล์สำหรับจัดเก็บตัวปรับมาตรฐานข้อมูล (StandardScaler) เพื่อนำไปใช้งานร่วมกับโมเดลในตอนทำนายจริง

```

X, y = [], []
skipped = 0

for item in raw_data:
    sequence = np.array(item['data'])
    if len(sequence) != expected_frames:
        skipped += 1
        continue
    X.append(sequence.flatten())
    y.append(item['label'])

print(f"Successfully loaded {len(X)} samples (skipped {skipped} due to frame mismatch)\n")

X = np.array(X)
y = np.array(y)

```

รูปที่ 3.18 โค้ดจัดรูปแบบข้อมูลให้พร้อมสำหรับการเทรน

จากรูปที่ 3.18 เป็นการจัดรูปแบบข้อมูลให้พร้อมสำหรับการเทรน

- X, y = [], [] กำหนดตัวแปร X (เก็บข้อมูลเซนเซอร์นำเข้า) และ y (เก็บป้ายกำกับฉลาก) เป็นอาร์เรย์ว่าง
- for item in raw_data
 - sequence = np.array(item['data'])
 เป็นการวนลูปอ่านข้อมูลที่ละค่าทางใน raw_data และแปลงส่วนข้อมูลพารามิเตอร์ให้เป็น NumPy Array

- if len(sequence) != expected_frames:

skipped += 1

continue

เป็นการตรวจสอบว่าข้อมูลทำทางนี้มีเฟรมครบ 60 เฟรมหรือไม่ หากไม่ครบ ให้เพิ่มจำนวนตัวเก็บสถิติข้าม skipped ขึ้นทีละหนึ่ง และข้ามไปอ่านตัวอย่างถัดไปทันที

- X.append(sequence.flatten()) ใช้ฟังก์ชัน flatten() บีบอัดมิติข้อมูลจากตาราง (60 แถว, 11 คอลัมน์) ให้เหลือแนวราบเส้นเดียว 660 ตัวเลข แล้วเก็บเข้าสู่ชุดข้อมูล X
- y.append(item['label']) เก็บชื่อเฉลี่ยของทำทางนั้น (Label) เข้าสู่ชุดข้อมูล

คำตอบ y

- X = np.array(X)

y = np.array(y)

แปลงตัวแปร X และ y ให้กลายเป็นโครงสร้างอาร์เรย์ของ NumPy เพื่อส่งเข้า

ประมวลผล

```
try:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
except ValueError:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
```

รูปที่ 3.19 โค้ดการแบ่งข้อมูลเป็น Train และ Test

จากรูปที่ 3.19 เป็นการแบ่งข้อมูลเป็น Train และ Test

- try:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

เป็นการ สุ่มแบ่งข้อมูลเซนเซอร์ (X) และคำตอบ (y) ในสัดส่วน 80% สำหรับสอน และ 20% สำหรับใช้สอบ โดยเลือกความสุ่มด้วย random_state=42 และรักษาสัดส่วนความถี่ของแต่ละคลาสในข้อมูลทั้งสองกลุ่มให้เท่ากันด้วยคำสั่ง stratify=y

- except ValueError:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

หากข้อมูลแต่ละท่าทางมีจำนวนตัวอย่างน้อยเกินไป หรือเกิดปัญหาจำนวนคลาสไม่สมดุล (Class Imbalance) ฟังก์ชันจะเกิด Error ขึ้น ระบบจะกู้คืนการรันโดยเปลี่ยนไปใช้วิธีแบ่งสุ่มแบบปกติที่ไม่มีการรักษาสัดส่วน (stratify) แทน

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

รูปที่ 3.20 โค้ดสำหรับการปรับสเกล

จากรูปที่ 3.20 เป็นโค้ดสำหรับการปรับสเกล เพื่อให้ข้อมูลอยู่ในมาตรฐานเดียวกัน

- scaler = StandardScaler() ประกาศใช้โมดูลปรับมาตรฐานข้อมูลด้วยคะแนนมาตรฐาน Z-score
- X_train_scaled = scaler.fit_transform(X_train) คำนวณหาค่าเฉลี่ย (μ) และ ส่วนเบี่ยงเบนมาตรฐาน (σ) จากข้อมูลสอน X_train แล้วดำเนินการปรับระดับตัวเลขของชุดข้อมูลฝึกสอนนั้นให้อยู่ในช่วงมาตรฐานเดียวกันทันที
- X_test_scaled = scaler.transform(X_test) ปรับระดับตัวเลขของชุดข้อมูลทดสอบ X_test โดยใช้ค่า μ และ σ เดียวกันกับที่คำนวณได้จากข้อมูลสอน

```
model = MLPClassifier(
    hidden_layer_sizes=(256, 128),
    activation='relu',
    solver='adam',
    max_iter=500,
    random_state=42,
    verbose=True,
    early_stopping=False
)

model.fit(X_train_scaled, y_train)
```

รูปที่ 3.21 โค้ดสำหรับการสร้างโมเดล

จากรูปที่ 3.21 เป็นโค้ดสำหรับการสร้างโมเดลเพื่อนำไปใช้ในการเทรนโมเดล

- hidden_layer_sizes=(256, 128) กำหนดให้มีชั้นซ่อน (Hidden Layer) 2 ชั้น ชั้นแรกมี 256 นิวรอน ชั้นที่สองมี 128 นิวรอน
- activation='relu' ใช้ฟังก์ชันเรลูประมวลผลเพิ่มมิติที่ไม่เป็นเส้นตรงในชั้นซ่อน
- solver='adam' ใช้อัลกอริทึม Adam ช่วยปรับค่าน้ำหนักให้ลู่เข้าจุดที่ดีที่สุดอย่างรวดเร็ว
- max_iter=500 กำหนดขอบเขตสูงสุดการเทรนไว้ที่ 500 รอบการรัน
- random_state=42 ล็อกความสุ่มเริ่มต้นของน้ำหนักเพื่อให้ฝึกสอนก็ครั้งก็ได้ผลลัพธ์เหมือนเดิม
- verbose=True: สั่งให้โปรแกรมพิมพ์ค่าความสูญเสีย (Loss Value) ในแต่ละรอบการรันออกมาแสดงผลบนหน้าจอ
- early_stopping=False สั่งไม่ให้เป็นแบบจำลองหยุดฝึกสอนกลางคันจนกว่าจะถึงเกณฑ์ความต่างที่เหลือน้อยมากหรือรันครบ 500 รอบ
- model.fit(X_train_scaled, y_train) สั่งเริ่มกระบวนการคำนวณฝึกสอนหาความสัมพันธ์ระหว่างระดับสัญญาณเซนเซอร์ที่ปรับค่าแล้ว (X_train_scaled) กับชื่อท่าทางภาษามือจริง (y_train)

```
try:
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(8, 6))
    classes = np.unique(np.concatenate((y_test, y_pred)))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes, yticklabels=classes)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title('Confusion Matrix (Dynamic Gestures)')
    plt.tight_layout()
    plt.savefig('confusion_matrix.png', dpi=300)
    plt.close()
    print("Confusion matrix saved to 'confusion_matrix.png'.")
except Exception as plot_err:
```

รูปที่ 3.22 เป็นโค้ดสร้างแผนภาพ Confusion Matrix

จากรูปที่ 3.22 เป็นโค้ดสำหรับสร้าง แผนภาพ Confusion Matix เพื่อดูว่าท่าทางไหนมีข้อผิดพลาดบ้าง

- cm = confusion_matrix(y_test, y_pred) คำนวณค่าจำนวนครั้งที่ทำนายได้ถูกคลาสและทายสลับสับสนคลาสของชุดทดสอบ
- plt.figure(figsize=(8, 6)) สร้างพื้นที่รูปภาพวาดกราฟขนาดสัดส่วน 8x6 นิ้ว

- `classes = np.unique(np.concatenate((y_test, y_pred)))` รวมชื่อคลาสเฉพาะทั้งหมดที่มีอยู่ในเฉลยเพื่อนำมาใช้เป็นป้ายระบุข้างแกนกราฟ

- `sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes, yticklabels=classes)` วาดแผนภูมิความร้อน (Heatmap) โดยใช้จำนวนครั้งการทายสลับ ตัวเลขระบุชัดเจนด้วยสีกราฟาเดียนต์โทนฟ้า

- `plt.xlabel('Predicted')`

`plt.ylabel('Actual')`

`plt.title('Confusion Matrix (Dynamic Gestures)')`

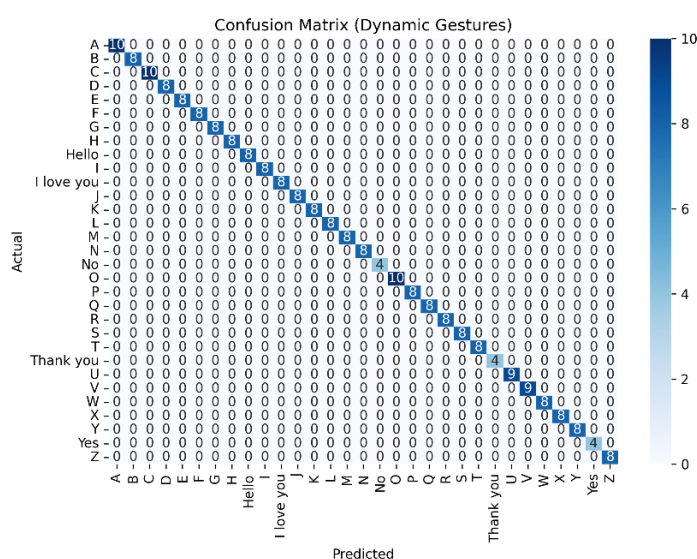
ระบุคำบรรยายในแกนทำนาย (Predicted) แกนจริง (Actual) และกำหนดหัวข้อเรื่องของแผนภาพ

- `plt.savefig('confusion_matrix.png', dpi=300)` เซฟรูปแผนภาพที่วาดเสร็จแล้วออกเป็นไฟล์รูปภาพ `confusion_matrix.png` ด้วยความคมชัดสูง 300 DPI

- `plt.close()` ปิดกราฟของ Matplotlib เพื่อคืนหน่วยความจำเครื่องคอมพิวเตอร์และแสดงผลว่าการเซฟภาพลุล่วง

- `except Exception as plot_err:` ตรวจจับข้อผิดพลาดกรณีไม่สามารถเซฟรูปภาพได้ (เช่น ไม่มีสิทธิ์เขียนไฟล์) ให้แจ้งเตือนและปล่อยโปรแกรมทำงานต่อไปโดยไม่ทำให้ระบบล่ม

หลังจาก Run โค้ดส่วนนี้แล้ว จะได้กราฟที่แสดงการประเมินความถูกต้องที่ไม่เดลทาย ดังรูปที่ 3.23



รูปที่ 3.23 กราฟประเมิน (Confusion Matrix)

```

model_data = {
    "model": model,
    "expected_frames": expected_frames
}
joblib.dump(model_data, MODEL_FILE)
joblib.dump(scaler, SCALER_FILE)

```

รูปที่ 3.24 โค้ดสำหรับการบันทึกโมเดล

จากรูปที่ 3.24 เป็นโค้ดสำหรับการบันทึกโมเดลเพื่อนำมาใช้งานในการเป็นสมองของตัวประมวลผล

```

- model_data = {
    "model": model,
    "expected_frames": expected_frames
}

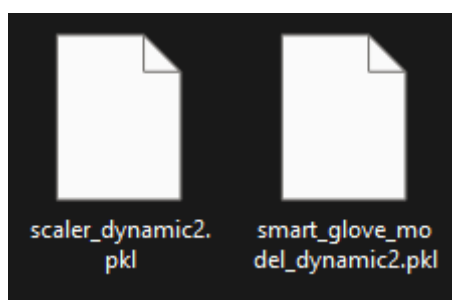
```

รวมโครงข่าย MLP (model) ที่ผ่านการสอนแล้วและข้อมูลจำนวนเฟรมที่คาดหวัง (expected_frames) เข้าด้วยกันในรูปแบบโครงสร้างดิกชันนารี (Dictionary)

- joblib.dump(model_data, MODEL_FILE) เซพวัตถุโมเดลทั้งหมดลงในไฟล์ smart_glove_model_dynamic2.pkl

- joblib.dump(scaler, SCALER_FILE) เซพวัตถุตัวปรับมาตรฐานข้อมูลลงในไฟล์ scaler_dynamic2.pkl

หลังจากทำการรันโค้ดเรียบร้อยแล้วจะได้ไฟล์ออกมา ดังรูปที่ 3.25



รูปที่ 3.25 ไฟล์ที่ได้จากการเทรน

1. smart_glove_model_dynamic2.pkl เป็นไฟล์โมเดลที่ผ่านการเทรนมาเรียบร้อยแล้ว

แล้ว

2. scaler_dynamic2.pkl เป็นไฟล์ scale ที่เอาไว้ปรับสเกลของข้อมูลที่ส่งมาผ่าน ESP32

3.9 การพัฒนา Web Application

เว็บแอปพลิเคชันภายในระบบนี้ทำหน้าที่เป็นตัวกลางเชื่อมโยงระหว่างโมเดลปัญญาประดิษฐ์กับผู้ใช้งาน โดยจะรับผลลัพธ์จากการจำแนกท่าทางที่ประมวลผลโดยโมเดล MLP มาแสดงเป็นข้อความบนหน้าจอ การทำงานในส่วนนี้ช่วยให้บุคคลทั่วไปที่ไม่มีความรู้ด้านภาษามือสามารถเข้าใจความหมายของท่าทางที่ผู้พิการทางการได้ยินสื่อสารออกมาได้อย่างถูกต้อง

3.9.1 เทคโนโลยีที่ใช้ในการพัฒนา

เว็บแอปพลิเคชัน (Web Application) ในโครงงานนี้พัฒนาด้วยชุดเทคโนโลยีที่ทำงานร่วมกัน ดังนี้

1. Flask เป็น Python Micro-framework ที่ทำหน้าที่เป็น Backend Server สำหรับรับคำสั่งจาก Frontend กำหนดเส้นทาง (Route) และเชื่อมต่อกับโมเดล MLP
2. HTML ใช้สำหรับกำหนดโครงสร้างและเนื้อหาของหน้าเว็บ
3. CSS ใช้สำหรับควบคุมรูปแบบการแสดงผล
4. JavaScript ใช้เพิ่มความสามารถในการโต้ตอบฝั่ง Frontend

3.9.2 อธิบายโค้ดส่วนการแสดงผล

```
import json
import socket
import sys
import joblib
import numpy as np
from flask import Flask, jsonify, render_template
```

รูปที่ 3.26 โค้ดสำหรับนำเข้าไลบรารี

จาก รูปที่ 3.26 เป็นการนำเข้าไลบรารี และโมดูลที่ใช้สำหรับการสร้าง Webapplication

- import json นำเข้าโมดูลจัดการข้อมูลประเภท JSON
- import socket นำเข้าโมดูลสำหรับการรับส่งข้อมูลบนระบบเครือข่ายอินเทอร์เน็ต
- import sys นำเข้าโมดูลจัดการระบบการทำงานของภาษา Python

- import joblib นำเข้าโมเดลสำหรับโหลดไฟล์ตัวแบบจำลอง (Model) และตัวปรับระดับสเกลข้อมูล (Scaler)
- import numpy as np นำเข้าคลังจัดการอาเรย์คณิตศาสตร์เพื่อจัดระเบียบเฟรมข้อมูลนำไปทำนายผล
- from flask import Flask, jsonify, render_template นำเข้าคลาสหลักของเว็บเฟรมเวิร์ก Flask รวมถึงฟังก์ชันสำหรับแปลงรูปแบบผลทำนายเป็น JSON (jsonify) และคำสั่งสำหรับส่งหน้าเว็บ HTML ไปแสดงผลบนเบราว์เซอร์

```
MODEL_FILE = 'smart_glove_model_dynamic2.pkl'
SCALER_FILE = 'scaler_dynamic2.pkl'
UDP_IP = "0.0.0.0"
UDP_PORT = 4210
```

รูปที่ 3.27 โค้ดสำหรับกำหนดตัวแปร

จากรูปที่ 3.27 เป็นโค้ดสำหรับกำหนดตัวแปรที่ใช้ในโค้ด

- MODEL_FILE = 'smart_glove_model_dynamic2.pkl'
- SCALER_FILE = 'scaler_dynamic2.pkl'

กำหนดชื่อไฟล์โมเดลปัญญาประดิษฐ์และไฟล์ปรับสเกลข้อมูลที่จะถูกโหลดขึ้นมาใช้

งาน

- UDP_IP = "0.0.0.0" กำหนดไอพีการรับข้อมูลเป็น "0.0.0.0"
- UDP_PORT = 4210 กำหนดเปิดพอร์ตการรับแพ็กเก็ตข้อมูลจากถุงมือไว้ที่

หมายเลขพอร์ต 4210

```
try:
    model_data = joblib.load(MODEL_FILE)
    model = model_data["model"]
    expected_frames = model_data["expected_frames"]
    scaler = joblib.load(SCALER_FILE)
    print(f"Model loaded successfully. Expected frames per gesture: {expected_frames}")
```

รูปที่ 3.28 โค้ดสำหรับการโหลดโมเดล

จากรูปที่ 3.28 เป็นโค้ดสำหรับการโหลดโมเดลที่เทรนไว้ก่อนหน้านี้มาใช้

- try:

```

model_data = joblib.load(MODEL_FILE)
model = model_data["model"]
expected_frames = model_data["expected_frames"]
scaler = joblib.load(SCALER_FILE)

```

โหลดไฟล์โมเดลปัญญาประดิษฐ์ (smart_glove_model_dynamic2.pkl) มาแกะแยก
 คลาสแบบจำลอง (model) และข้อมูลความยาวเฟรมสะสมที่คาดหวัง (expected_frames) พร้อม
 โหลดตัวปรับสเกล (SCALER_FILE) ข้อมูลเข้ามาเก็บในหน่วยความจำ

```

@app.route('/')
def index():
    return render_template('index.html')

```

รูปที่ 3.29 โค้ดสำหรับกำหนดรูปแบบของ Template

จากรูปที่ 3.29 เป็นโค้ดสำหรับการกำหนดรูปแบบของ Template ที่อยู่ใน ไฟล์
 index.html

- @app.route('/')
 def index():
 กำหนดเส้นทางหลักของเว็บไซต์
- return render_template('index.html') ส่งไฟล์รูปแบบเว็บเพจ (index.html) ที่
 จัดเก็บในโฟลเดอร์ templates ออกไปแสดงผลบนหน้าจอผู้เข้าใช้

```

def predict_once():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((UDP_IP, UDP_PORT))
    sock.settimeout(5.0)

```

รูปที่ 3.30 โค้ดสำหรับสร้างช่องทางรับข้อมูลจาก ESP32

จากรูปที่ 3.30 เป็นโค้ดสำหรับการสร้างช่องทางรับข้อมูลที่มาจาก ESP32

- def predict_once(): กำหนดสร้างจุดเชื่อมต่อ API
- sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) ประกาศสร้าง
 พอร์ตเชื่อมต่อเน็ตเวิร์ก (Socket) โดยกำหนดใช้โปรโตคอล IPv4 และใช้ช่องทางการส่งข้อมูลแบบไม่
 ยืนยันผลเชื่อมต่อหรือ

- sock.bind((UDP_IP, UDP_PORT)) สั่งผูกพอร์ตเชื่อมต่อเข้ากับไอพีและพอร์ต 4210
- sock.settimeout(5.0) กำหนดระยะเวลาหน่วงรอคอยข้อมูลสูงสุด (Timeout) ไว้ที่ 5.0 วินาที หากไม่มีการส่งแพ็กเก็ตเกิดใด ๆ จากถูงมือเข้ามาภายในเวลานี้ ให้ระบบยกเลิกการรอคอย

```
try:
    while len(buffer_frames) < expected_frames:
        data, _ = sock.recvfrom(2048)
        try:
            raw_data = json.loads(data.decode('utf-8'))
            frame_vector = raw_data['flex'] + raw_data['acc'] + raw_data['gyro']
            buffer_frames.append(frame_vector)
```

รูปที่ 3.31 โค้ดสำหรับรับและแปลงข้อมูล

จากรูปที่ 3.31 เป็นโค้ดสำหรับรับข้อมูลและปรับข้อมูลให้เหมาะสมสำหรับเข้าโมเดล

- while len(buffer_frames) < expected_frames: วงลูปอ่านข้อมูลทางพอร์ตต่อเนื่อง จนกว่าจะครบ 60 เฟรม
- data, _ = sock.recvfrom(2048) กำหนดความยาวของข้อมูลไม่เกิน 2048 ไบต์
- raw_data = json.loads(data.decode('utf-8')) แปลงข้อความแพ็กเก็ตข้อมูลจาก utf-8 ให้เป็นโครงสร้างข้อมูลประเภทดิกชันนารี
- frame_vector = raw_data['flex'] + raw_data['acc'] + raw_data['gyro'] นำตัวเลขแถว สลิสต์ข้อมูลเซนเซอร์นิ้วมือ 5 นิ้ว (flex), สัญญาณความเร่ง 3 ทิศทาง (acc) และสัญญาณการหมุนข้อมือ 3 ทิศทาง (gyro) มารวมเชื่อมเรียงต่อกันเป็นเวกเตอร์เฟรมข้อมูลมิติเดียว
- buffer_frames.append(frame_vector) นำข้อมูล 11 ตัวเลขที่ได้ไปต่อท้ายเก็บสะสมในอาร์เรย์หลัก buffer_frames

```
X_realtime = np.array(buffer_frames).reshape(1, -1)
X_realtime_scaled = scaler.transform(X_realtime)
prediction = model.predict(X_realtime_scaled)[0]
```

รูปที่ 3.32 โค้ดสำหรับการทำนายผลลัพธ์

จากรูปที่ 3.32 เป็นโค้ดสำหรับการทำนายผลโดยใช้โมเดลที่เทรนมา

- X_realtime = np.array(buffer_frames).reshape(1, -1) นำอาร์เรย์ข้อมูล 60 เฟรม (ขนาด 60×11) มาแปลงเป็น NumPy Array และใช้คำสั่ง .reshape(1, -1) บีบรูปมิติแนวแบนราบให้เหลือมิติขนาด 1 แถว ยาว 660 คอลัมน์ สำหรับให้โมเดลประมวลผล

- `X_realtime_scaled = scaler.transform(X_realtime)` นำเวกเตอร์ขนาด 660 มิตินี้ไปคำนวณปรับสเกลมาตรฐานตัวเลขโดยตัวแปรปรับแต่ง `scaler`
- `prediction = model.predict(X_realtime_scaled) [0]` ป้อนข้อมูลเข้าตัวแบบปัญญาประดิษฐ์เพื่อทำนายผลลัพธ์ และดึงคำตอบลำดับที่ศูนย์ออกมาเก็บในตัวแปร `prediction`

3.10 สรุปท้ายบทที่ 3

ในบทนี้ได้นำเสนอรายละเอียดการวิเคราะห์และออกแบบระบบถุงมือแปลภาษามืออัจฉริยะใน ทุกส่วนอย่างครบถ้วน เริ่มตั้งแต่การจัดทำแผนผังการต่อวงจรและตำแหน่งการกำหนดขาสัญญาณของ ไมโครคอนโทรลเลอร์ ESP32 ร่วมกับเซนเซอร์ตรวจจับการโค้งงอของนิ้วมือ (Flex Sensors) ทั้ง 5 นิ้ว และเซนเซอร์ตรวจจับความเคลื่อนไหวข้อมือ MPU6050 ตลอดจนสถาปัตยกรรมทางโครงสร้าง เครือข่ายและแบบจำลองโครงข่ายประสาทเทียมแบบหลายชั้น (MLP) ที่ใช้คุณลักษณะนำเข้า 11 ตัวแปรเก็บรวมเป็นอนุกรมเวลาจำนวน 60 เฟรม (รวม 660 มิติ) ส่งผ่านทางเครือข่ายไร้สายด้วย โปรโตคอล UDP พอร์ต 4210 เพื่อไปจัดเก็บในรูปแบบโครงสร้าง JSON และเชื่อมต่อไปยังหลังบ้าน ของระบบแสดงผลบนหน้าเว็บเบราว์เซอร์ ซึ่งทั้งหมดนี้ได้กลายเป็นรากฐานและโครงสร้างการ ดำเนินงานสำหรับการนำไปฝึกสอนโมเดลและการทดสอบในบทถัดไป

บทที่ 4

ผลการดำเนินการ

4.1 ผลการดำเนินงาน

การดำเนินงานในระบบถุงมือแปลภาษามือ (ข้างเดียว) ได้รับการออกแบบและพัฒนาให้ครอบคลุมทั้งส่วนของฮาร์ดแวร์ ซอฟต์แวร์ และกระบวนการประมวลผลข้อมูลแบบครบวงจร โดยระบบถูกพัฒนาให้สอดคล้องกับขอบเขตที่กำหนดไว้ทั้ง 6 ข้อ ได้แก่ การใช้บอร์ดไมโครคอนโทรลเลอร์ ESP32 เป็นตัวส่งข้อมูล การติดตั้ง Flex Sensor จำนวน 5 ชิ้นเพื่อวัดค่าการงอของนิ้วมือทั้ง 5 นิ้ว การใช้เซนเซอร์ MPU6050 วัดระนาบและการเคลื่อนไหวของข้อมือ การพัฒนาเว็บแอปพลิเคชันด้วย Flask สำหรับแสดงผลพร้อมปุ่มควบคุมการแปล การรองรับท่าทางมือเดียวครอบคลุมตัวอักษร A-Z และคำศัพท์ Hello, Thank you, I love you, Yes และ No รวม 31 ท่าทาง รวมถึงการทำนายผลท่าทางภาษามือจากค่า Flex Sensor และ MPU6050 ได้จริง องค์ประกอบหลักของระบบแบ่งออกเป็น 3 ส่วน ดังนี้

4.1.1 การเทรนโมเดลและจัดเตรียมโมเดลปัญญาประดิษฐ์

กระบวนการฝึกสอนโมเดลปัญญาประดิษฐ์ดำเนินการโดยใช้โครงข่ายประสาทเทียมแบบ Multi-layer Perceptron Classifier (MLP Classifier) ผ่านไลบรารี Scikit-Learn บนภาษา Python ข้อมูลที่ใช้ในการฝึกสอนได้มาจากการเก็บสัญญาณเซนเซอร์ผ่านถุงมือนับแบบ โดยประกอบด้วยข้อมูลจาก Flex Sensor จำนวน 5 ตัว และ MPU6050 จำนวน 1 ตัว ซึ่งวัดค่าระนาบและการเคลื่อนไหวของข้อมือ การเก็บข้อมูลแต่ละท่าทางดำเนินการในรูปแบบ 60 เฟรมต่อตัวอย่าง ส่งผ่านระบบเครือข่ายไร้สายด้วยโปรโตคอล UDP เพื่อให้ได้ข้อมูลที่มีความหน่วงเวลาต่ำ (Low Latency) โมเดลที่ผ่านการฝึกสอนแล้วจะถูกบันทึกในรูปแบบไฟล์ .pkl เพื่อนำไปใช้งานจริงในกระบวนการจำแนกท่าทางในขั้นตอนถัดไป

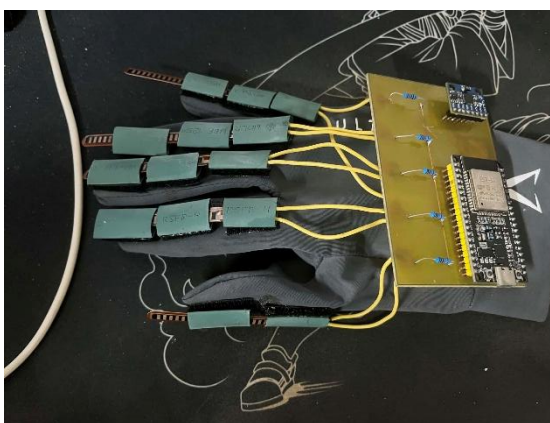
4.1.2 กระบวนการประมวลผลข้อมูลแบบช่วงเวลา

ในขั้นตอนการใช้งานจริง ผู้ใช้งานจะสวมถุงมือและจัดท่าทางภาษามือที่ต้องการแปลความหมาย จากนั้นกดปุ่มแปลบนเว็บแอปพลิเคชัน เพื่อเริ่มต้นกระบวนการเก็บข้อมูล ระบบจะทำการรับสัญญาณจากเซนเซอร์ผ่านไมโครคอนโทรลเลอร์ ESP32 และส่งข้อมูลในรูปแบบ JSON ผ่านโปรโตคอล UDPมายังคอมพิวเตอร์ฝั่งผู้รับอย่างต่อเนื่อง จนกว่าจะครบ 60 เฟรมตามที่กำหนด หากจำนวนเฟรมที่

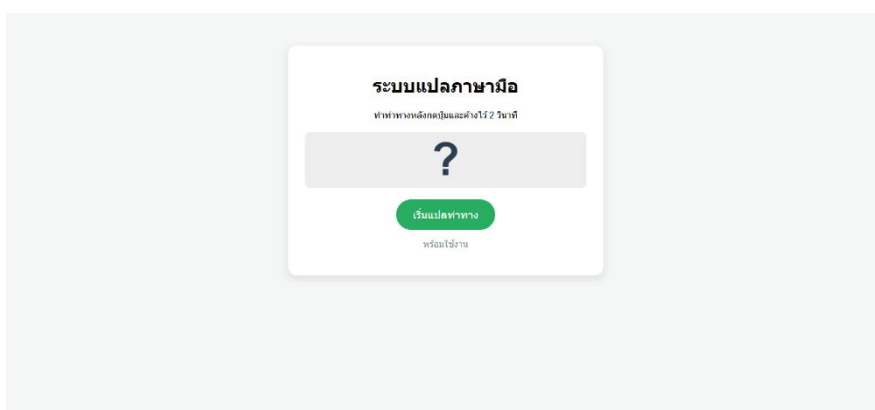
เก็บได้ยังไม่ครบตามเงื่อนไข ระบบจะดำเนินการเก็บข้อมูลต่อเนื่องจนครบจำนวนที่กำหนด เมื่อได้รับข้อมูลครบ 60 เฟรมแล้ว ข้อมูลดังกล่าวจะถูกส่งเข้าสู่โมเดล MLP Classifier เพื่อทำการจำแนกท่าทาง โดยระบบรองรับการจำแนกท่าทางภาษามืออเมริกัน (American Sign Language) ครอบคลุมตัวอักษร A ถึง Z รวมถึงคำศัพท์ที่ใช้บ่อย ได้แก่ Hello, Thank you, I love you, Yes และ No

4.1.3 การแสดงผลผ่านเว็บแอปพลิเคชัน

ผลลัพธ์จากการจำแนกท่าทางจะถูกส่งต่อไปยังเว็บแอปพลิเคชันที่พัฒนาด้วย Flask Framework ซึ่งทำหน้าที่เป็นตัวกลางในการเชื่อมโยงระหว่างโมเดลปัญญาประดิษฐ์และผู้ใช้งาน โดยเว็บแอปพลิเคชันจะแสดงผลการแปลท่าทางภาษามือออกมาในรูปแบบข้อความบนหน้าจอ ซึ่งผู้ใช้งานสามารถอ่านและทำความเข้าใจได้ทันที ระบบการแสดงผลดังกล่าวช่วยลดอุปสรรคด้านการสื่อสารระหว่างผู้พิการทางการได้ยินกับบุคคลทั่วไป และเปิดโอกาสให้ผู้ที่ไม่มีความรู้ด้านภาษามือสามารถเข้าใจความหมายของท่าทางที่แสดงได้อย่างสะดวกและรวดเร็ว



รูปที่ 4.1 ถุงมือต้นแบบที่พัฒนาขึ้น



รูปที่ 4.2 หน้าต่าง Web Application

4.2 ผลการประเมินประสิทธิภาพของโมเดลปัญญาประดิษฐ์

ในการประเมินประสิทธิภาพการใช้งานจริงของระบบถุงมือแปลภาษามือแบบข้างเดียว ทางเราได้ทำการทดสอบจำแนกท่าทางภาษามือทั้งตัวอักษรและคำศัพท์ตามขอบเขตที่กำหนดไว้ทั้งหมด 31 ท่าทาง ผ่านเว็บแอปพลิเคชัน จำนวนทั้งสิ้น 30 ครั้งต่อท่าทาง เพื่อวัดความเสถียรและการตอบสนองของโมเดลโครงข่ายประสาทเทียมต่อการสั่งการจริงของผู้ใช้งาน โดยข้อมูลที่ใช้ทดสอบผ่านการอ่านค่าจาก Flex Sensor (5 นิ้ว) และ MPU6050 (Accelerometer & Gyroscope) ผ่านบอร์ด ESP32 ส่งมาทาง UDP พอร์ต 4210 ครบ 60 เฟรมต่อครั้ง และผ่านการทำนายด้วยโมเดล MLP Classifier ผลการทดสอบและการวิเคราะห์ความถูกต้องปรากฏดังด้านล่าง

ตารางที่ 4.1 ผลการทดสอบการแปลตัวอักษร A

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	A	11	A	21	A
2	C	12	A	22	A
3	A	13	A	23	A
4	A	14	A	24	A
5	A	15	A	25	A
6	A	16	A	26	A
7	A	17	A	27	A
8	A	18	A	28	A
9	A	19	A	29	A
10	A	20	A	30	A

จากตารางที่ 4.1 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 96.67 ผิดเป็นท่า C (1 ครั้ง) เนื่องจากรูปทรงการกำมือพับนิ้วมีความใกล้เคียงกัน ทำให้ค่าสัญญาณจาก Flex Sensor คาบเกี่ยวกันในบางจังหวะ

ตารางที่ 4.2 ผลการทดสอบการแปลตัวอักษร B

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	B	11	B	21	B
2	B	12	B	22	B
3	B	13	B	23	B
4	B	14	B	24	B
5	B	15	B	25	B
6	B	16	B	26	B
7	B	17	B	27	B
8	B	18	B	28	B
9	B	19	B	29	B
10	B	20	B	30	B

จากตารางที่ 4.2 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.3 ผลการทดสอบการแปลตัวอักษร C

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	A	11	C	21	C
2	A	12	C	22	C
3	B	13	C	23	C
4	A	14	C	24	C
5	C	15	C	25	C
6	C	16	C	26	C
7	C	17	C	27	C
8	C	18	C	28	C
9	C	19	C	29	C
10	C	20	C	30	C

จากตารางที่ 4.3 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 86.67 ผิดเป็นท่า A (3 ครั้ง) และท่า B (1 ครั้ง) เนื่องจากลักษณะการงอมือเป็นรูปส่วนโค้งครึ่งวงกลม (C-shape) คาบเกี่ยวกับการกำมือ (A) และการเหยียดตรง (B) เมื่อขยับองศาเล็กน้อย

ตารางที่ 4.4 ผลการทดสอบการแปลตัวอักษร D

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	D	11	D	21	D
2	D	12	D	22	D
3	D	13	D	23	D
4	D	14	D	24	D
5	D	15	D	25	D
6	D	16	D	26	D
7	D	17	D	27	D
8	D	18	D	28	D
9	D	19	D	29	D
10	D	20	D	30	D

จากตารางที่ 4.4 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.5 ผลการทดสอบการแปลตัวอักษร E

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	A	11	E	21	E
2	A	12	E	22	E
3	E	13	E	23	E
4	E	14	E	24	E
5	E	15	E	25	E
6	E	16	E	26	E
7	E	17	E	27	E
8	E	18	E	28	E
9	E	19	E	29	E
10	E	20	E	30	E

จากตารางที่ 4.5 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 93.33 ผิดเป็นท่า A (2 ครั้ง) เนื่องจากรูปทรงการพับนิ้วทั้งสองมาสบกับฝ่ามือคล้ายคลึงกับท่ากำปั้น (A) มาก ทำให้เซนเซอร์อ่านค่าได้ใกล้เคียงกัน

ตารางที่ 4.6 ผลการทดสอบการแปลตัวอักษร F

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	F	11	F	21	F
2	F	12	F	22	F
3	F	13	F	23	F
4	F	14	F	24	F
5	F	15	F	25	F
6	F	16	F	26	F
7	F	17	F	27	F
8	F	18	F	28	F
9	F	19	F	29	F
10	F	20	F	30	F

จากตารางที่ 4.6 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.7 ผลการทดสอบการแปลตัวอักษร G

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	G	11	G	21	G
2	G	12	G	22	G
3	G	13	G	23	G
4	G	14	G	24	G
5	G	15	G	25	G
6	G	16	G	26	G
7	G	17	G	27	G
8	G	18	G	28	G
9	G	19	G	29	G
10	G	20	G	30	G

จากตารางที่ 4.7 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.8 ผลการทดสอบการแปลตัวอักษร H

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	H	11	H	21	H
2	H	12	H	22	H
3	H	13	H	23	H
4	H	14	H	24	H
5	H	15	H	25	H
6	H	16	H	26	H
7	H	17	H	27	H
8	H	18	H	28	H
9	H	19	H	29	H
10	H	20	H	30	H

จากตารางที่ 4.8 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.9 ผลการทดสอบการแปลตัวอักษร I

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	I	11	I	21	I
2	I	12	I	22	I
3	I	13	I	23	I
4	I	14	I	24	I
5	I	15	I	25	I
6	I	16	I	26	I
7	I	17	I	27	I
8	I	18	I	28	I
9	I	19	I	29	I
10	I	20	I	30	I

จากตารางที่ 4.9 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.10 ผลการทดสอบการแปลตัวอักษร J

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	I	11	J	21	J
2	Y	12	J	22	J
3	I	13	J	23	J
4	Y	14	J	24	J
5	J	15	J	25	J
6	J	16	J	26	J
7	J	17	J	27	J
8	J	18	J	28	J
9	J	19	J	29	J
10	J	20	J	30	J

จากตารางที่ 4.10 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 86.67 ผิดเป็นท่า I (2 ครั้ง) และท่า Y (2 ครั้ง) เกิดจากปัจจัยด้านไดนามิก ที่ต้องสับมือวาดรูปตัว J หากขยับมือช้าหรือเซนเซอร์ IMU ตรวจจับจังหวะสับไม่ทัน ระบบจะมองเห็นเพียงนิ้วก้อยตั้งตรงค้างไว้

ตารางที่ 4.11 ผลการทดสอบการแปลตัวอักษร K

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	No	11	K	21	K
2	U	12	K	22	K
3	V	13	K	23	K
4	V	14	K	24	K
5	K	15	K	25	K
6	K	16	K	26	K
7	K	17	K	27	K
8	K	18	K	28	K
9	K	19	K	29	K
10	K	20	K	30	K

จากตารางที่ 4.11 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบความถูกต้องแม่นยำ ร้อยละ 86.67 ผิดเป็นท่า V (2 ครั้ง), No (1 ครั้ง), U (1 ครั้ง) สืบสนในกลุ่มท่าชูสองนิ้ว (ชี้และกลาง) ซึ่งมีค่าสัญญาณพัวพันคล้ายกัน แตกต่างกันเพียงองศาการกางนิ้วในแนวนอนที่เซนเซอร์ Flex วัดไม่ได้โดยตรง

ตารางที่ 4.12 ผลการทดสอบการแปลตัวอักษร L

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	I love you	11	L	21	L
2	I love you	12	L	22	L
3	K	13	L	23	L
4	K	14	L	24	L
5	L	15	L	25	L
6	L	16	L	26	L
7	L	17	L	27	L
8	L	18	L	28	L
9	L	19	L	29	L
10	L	20	L	30	L

จากตารางที่ 4.12 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 86.67 ผิดเป็นท่า I love you (2 ครั้ง) และท่า K (2 ครั้ง) การชูนิ้วชี้และนิ้วโป้งอาจมีความคาบเกี่ยวกับท่าชูสามนิ้วหรือท่าชูสองนิ้วเอียง หากนิ้วนางและนิ้วก้อยพับงอไม่สนิทแน่นพอ

ตารางที่ 4.13 ผลการทดสอบการแปลตัวอักษร M

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	M	11	M	21	M
2	M	12	M	22	M
3	M	13	M	23	M
4	M	14	M	24	M
5	M	15	M	25	M
6	M	16	M	26	M
7	M	17	M	27	M
8	M	18	M	28	M
9	M	19	M	29	M
10	M	20	M	30	M

จากตารางที่ 4.13 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.14 ผลการทดสอบการแปลตัวอักษร N

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	N	11	N	21	N
2	T	12	N	22	N
3	N	13	N	23	N
4	T	14	N	24	N
5	N	15	N	25	N
6	T	16	N	26	N
7	N	17	N	27	N
8	N	18	N	28	N
9	M	19	N	29	N
10	N	20	N	30	N

จากตารางที่ 4.14 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 86.67 ผิดเป็นท่า T (3 ครั้ง) และท่า M (1 ครั้ง) เนื่องจากรูปทรงกำปั้นที่สอดนิ้วโป้งหลบใต้ข้อนิ้วมีความคาบเกี่ยวกันสูงมาก (N สอดใต้ข้อ-กลาง, T สอดระหว่างข้อ-กลาง) ทำให้สัญญาณทับซ้อนกัน

ตารางที่ 4.15 ผลการทดสอบการแปลตัวอักษร O

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	C	11	O	21	O
2	O	12	O	22	O
3	C	13	O	23	O
4	C	14	O	24	O
5	O	15	O	25	O
6	O	16	O	26	O
7	O	17	O	27	O
8	O	18	O	28	O
9	O	19	O	29	O
10	O	20	O	30	O

จากตารางที่ 4.15 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 90.00 ผิดเป็นท่า C (3 ครั้ง) เนื่องจากท่าห่อข้อนิ้วเป็นวงกลม (O) มีลักษณะทางกายภาพโค้งมนใกล้เคียงกับท่ารูปตัวครึ่งวงกลม (C) ส่งผลให้ค่าจากเซนเซอร์งอขึ้นอยู่ในโซนเดียวกัน

ตารางที่ 4.16 ผลการทดสอบการแปลตัวอักษร P

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	P	11	P	21	P
2	P	12	P	22	P
3	P	13	P	23	P
4	P	14	P	24	P
5	P	15	P	25	P
6	P	16	P	26	P
7	P	17	P	27	P
8	P	18	P	28	P
9	P	19	P	29	P
10	P	20	P	30	P

จากตารางที่ 4.16 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.17 ผลการทดสอบการแปลตัวอักษร Q

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	Q	11	Q	21	Q
2	Q	12	Q	22	Q
3	Q	13	Q	23	Q
4	Q	14	Q	24	Q
5	Q	15	Q	25	Q
6	Q	16	Q	26	Q
7	Q	17	Q	27	Q
8	Q	18	Q	28	Q
9	Q	19	Q	29	Q
10	Q	20	Q	30	Q

จากตารางที่ 4.17 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.18 ผลการทดสอบการแปลตัวอักษร R

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	V	11	R	21	R
2	V	12	R	22	R
3	V	13	R	23	R
4	R	14	R	24	R
5	R	15	R	25	R
6	V	16	R	26	R
7	R	17	R	27	R
8	R	18	R	28	R
9	R	19	R	29	R
10	R	20	R	30	R

จากตารางที่ 4.18 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 86.67 ผิดเป็นท่า V (4 ครั้ง) เนื่องจากเซนเซอร์ Flex วัดได้เฉพาะองศาการงอแนวตั้ง ไม่สามารถตรวจจับการกางหรือไขว้นิ้วในแนวนอนได้ ท่าไขว้ นิ้วชี้-กลาง (R) จึงอ่านค่าได้เหมือนท่ากาง นิ้วชี้-กลาง (V)

ตารางที่ 4.19 ผลการทดสอบการแปลตัวอักษร S

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	S	11	S	21	S
2	S	12	S	22	S
3	S	13	S	23	S
4	S	14	S	24	S
5	S	15	S	25	S
6	S	16	S	26	S
7	S	17	S	27	S
8	S	18	S	28	S
9	S	19	S	29	S
10	S	20	S	30	S

จากตารางที่ 4.19 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.20 ผลการทดสอบการแปลตัวอักษร T

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	T	11	T	21	T
2	T	12	T	22	T
3	T	13	T	23	T
4	T	14	T	24	T
5	T	15	T	25	T
6	T	16	T	26	T
7	T	17	T	27	T
8	T	18	T	28	T
9	T	19	T	29	T
10	T	20	T	30	T

จากตารางที่ 4.20 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.21 ผลการทดสอบการแปลตัวอักษร U

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	No	11	U	21	U
2	K	12	U	22	U
3	No	13	U	23	U
4	U	14	U	24	U
5	U	15	U	25	U
6	U	16	U	26	U
7	K	17	U	27	U
8	K	18	U	28	U
9	U	19	U	29	U
10	U	20	U	30	U

จากตารางที่ 4.21 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 83.33 ผิดเป็นท่า K (3 ครั้ง) และท่า No (2 ครั้ง) เกิดจากกลุ่มท่าชูสองนิ้วชิดกัน (U) หากผู้ทดสอบเปลื้องกางนิ้วออกเล็กน้อยหรือขยับข้อมือผิดองศา ระบบจะจำแนกสลับเป็นท่ากางนิ้ว (K) หรือ No ทันที

ตารางที่ 4.22 ผลการทดสอบการแปลตัวอักษร V

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	K	11	V	21	V
2	K	12	V	22	V
3	V	13	V	23	V
4	V	14	V	24	V
5	V	15	V	25	V
6	V	16	V	26	V
7	V	17	V	27	V
8	V	18	V	28	V
9	V	19	V	29	V
10	V	20	V	30	V

จากตารางที่ 4.22 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 93.33 ผิดเป็นท่า K (2 ครั้ง) เนื่องจากความใกล้เคียงกันของกลุ่มท่าชูสองนิ้ว (V คือชูนิ้วกางตรง, K คือชูสองนิ้วเอียงทำมุม) ทำให้องศาข้อมือที่อ่านจาก IMU คาบเกี่ยวกันบางครั้ง

ตารางที่ 4.23 ผลการทดสอบการแปลตัวอักษร W

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	W	11	W	21	W
2	W	12	W	22	W
3	W	13	W	23	W
4	W	14	W	24	W
5	W	15	W	25	W
6	W	16	W	26	W
7	W	17	W	27	W
8	W	18	W	28	W
9	W	19	W	29	W
10	W	20	W	30	W

จากตารางที่ 4.23 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.24 ผลการทดสอบการแปลตัวอักษร X

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	X	11	X	21	X
2	X	12	X	22	X
3	X	13	X	23	X
4	X	14	X	24	X
5	X	15	X	25	X
6	X	16	X	26	X
7	X	17	X	27	X
8	X	18	X	28	X
9	X	19	X	29	X
10	X	20	X	30	X

จากตารางที่ 4.24 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.25 ผลการทดสอบการแปลตัวอักษร Y

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	Y	11	Y	21	Y
2	Y	12	Y	22	Y
3	Y	13	Y	23	Y
4	Y	14	Y	24	Y
5	Y	15	Y	25	Y
6	Y	16	Y	26	Y
7	Y	17	Y	27	Y
8	Y	18	Y	28	Y
9	Y	19	Y	29	Y
10	Y	20	Y	30	Y

จากตารางที่ 4.25 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.26 ผลการทดสอบการแปลตัวอักษร Z

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	Z	11	Z	21	Z
2	Z	12	Z	22	Z
3	Z	13	Z	23	Z
4	Z	14	Z	24	Z
5	Z	15	Z	25	Z
6	Z	16	Z	26	Z
7	Z	17	Z	27	Z
8	Z	18	Z	28	Z
9	Z	19	Z	29	Z
10	Z	20	Z	30	Z

จากตารางที่ 4.26 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.27 ผลการทดสอบการแปลคำว่า Hello

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	Hello	11	Hello	21	Hello
2	Hello	12	Hello	22	Hello
3	Hello	13	Hello	23	Hello
4	Hello	14	Hello	24	Hello
5	Hello	15	Hello	25	Hello
6	Hello	16	Hello	26	Hello
7	Hello	17	Hello	27	Hello
8	Hello	18	Hello	28	Hello
9	Hello	19	Hello	29	Hello
10	Hello	20	Hello	30	Hello

จากตารางที่ 4.27 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.28 ผลการทดสอบการแปลคำว่า Thank you

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	Thank you	11	Thank you	21	Thank you
2	Thank you	12	Thank you	22	Thank you
3	Thank you	13	Thank you	23	Thank you
4	Thank you	14	Thank you	24	Thank you
5	Thank you	15	Thank you	25	Thank you
6	Thank you	16	Thank you	26	Thank you
7	Thank you	17	Thank you	27	Thank you
8	Thank you	18	Thank you	28	Thank you
9	Thank you	19	Thank you	29	Thank you
10	Thank you	20	Thank you	30	Thank you

จากตารางที่ 4.28 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.29 ผลการทดสอบการแปลตัวอักษร I love you

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	I love you	11	I love you	21	I love you
2	I love you	12	I love you	22	I love you
3	I love you	13	I love you	23	I love you
4	I love you	14	I love you	24	I love you
5	I love you	15	I love you	25	I love you
6	I love you	16	I love you	26	I love you
7	I love you	17	I love you	27	I love you
8	I love you	18	I love you	28	I love you
9	I love you	19	I love you	29	I love you
10	I love you	20	I love you	30	I love you

จากตารางที่ 4.29 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.30 ผลการทดสอบการแปลคำว่า Yes

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	Yes	11	Yes	21	Yes
2	Yes	12	Yes	22	Yes
3	Yes	13	Yes	23	Yes
4	Yes	14	Yes	24	Yes
5	Yes	15	Yes	25	Yes
6	Yes	16	Yes	26	Yes
7	Yes	17	Yes	27	Yes
8	Yes	18	Yes	28	Yes
9	Yes	19	Yes	29	Yes
10	Yes	20	Yes	30	Yes

จากตารางที่ 4.30 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

ตารางที่ 4.31 ผลการทดสอบการแปลคำว่า No

ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์	ครั้งที่	ผลลัพธ์
1	No	11	No	21	No
2	No	12	No	22	No
3	No	13	No	23	No
4	No	14	No	24	No
5	No	15	No	25	No
6	No	16	No	26	No
7	No	17	No	27	No
8	No	18	No	28	No
9	No	19	No	29	No
10	No	20	No	30	No

จากตารางที่ 4.31 ผลการทดสอบประสิทธิภาพของระบบจำแนกท่าทางภาษามือจากการทดสอบ ความถูกต้องแม่นยำ ร้อยละ 100.00

จากการทดสอบระบบถูงมือแปลภาษามือ (ข้างเดียว) โดยใช้ Flex Sensor 5 นิ้ว ร่วมกับ MPU6050 ส่งข้อมูลผ่าน ESP32 ทาง UDP ไปยังโมเดล MLP Classifier เพื่อจำแนกท่าทาง รวม 31 ท่าทาง (ตัวอักษร A-Z และคำศัพท์) ทดสอบท่าละ 30 ครั้ง สรุปผลรวมทั้ง 31 ท่าทาง มีค่าความแม่นยำรวม 96.02% คัดจากสมการที่ 11

$$\frac{X}{N} = \text{ค่าเฉลี่ยความแม่นยำรวม} \quad (11)$$

โดยที่ X = ผลรวมที่ได้จากการทดลอง ได้แก่

ท่าทางที่มีความแม่นยำร้อยละ 100 ได้แก่ B, D, F, G, H, I, M, P, Q, S, T, W, X, Y, Z และคำศัพท์ทั้งหมด (Hello, Thank you, I love you, Yes, No)

ท่าทางที่มีความแม่นยำร้อยละ 96.67% ได้แก่ A

ท่าทางที่มีความแม่นยำร้อยละ 93.33% ได้แก่ E, V

ท่าทางที่มีความแม่นยำร้อยละ 90.00% ได้แก่ O

ท่าทางที่มีความแม่นยำร้อยละ 86.67% ได้แก่ C, J, L, N, K, R

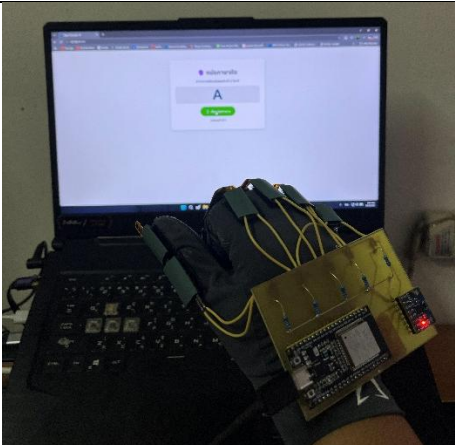
ท่าทางที่มีความแม่นยำร้อยละ 83.33% ได้แก่ U

N = จำนวนท่าทางทั้งหมด

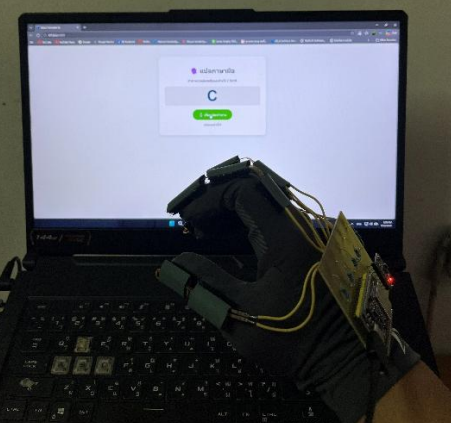
4.3 ผลการแสดงผลการจำแนกท่าทาง

ในการทดสอบประสิทธิภาพของโปรแกรม คณะผู้จัดทำได้ทำการทดสอบจำแนกท่าทางภาษามือ ผ่านการใช้งานจริงร่วมกับเว็บแอปพลิเคชัน เพื่อตรวจสอบความถูกต้องของค่าแปลแต่ละท่าทางตามขอบเขตที่กำหนด โดยทดสอบครอบคลุมท่าทางทั้ง 31 ท่า ได้แก่ ตัวอักษร A-Z (26 ท่า) และคำศัพท์ Hello, Thank you, I love you, Yes, No (5 ท่า) ผลการทดสอบแสดงผลการเปรียบเทียบระหว่างท่าทางของถุงมือและผลลัพธ์บนหน้าจอ ดังตารางที่ 4.32

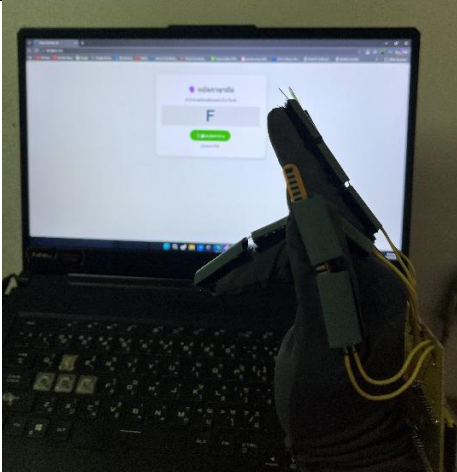
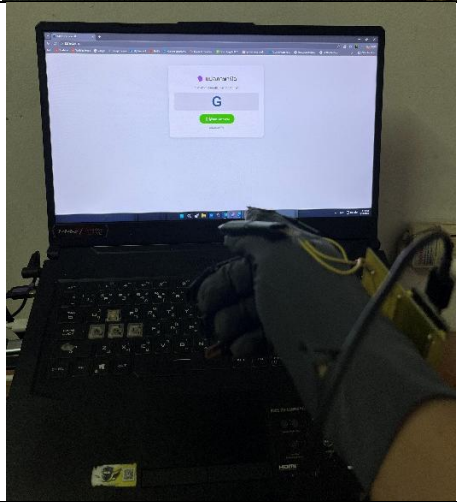
ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง

ตัวอักษรและ คำ	ภาพถ่ายขณะทำและผลลัพธ์	
A		

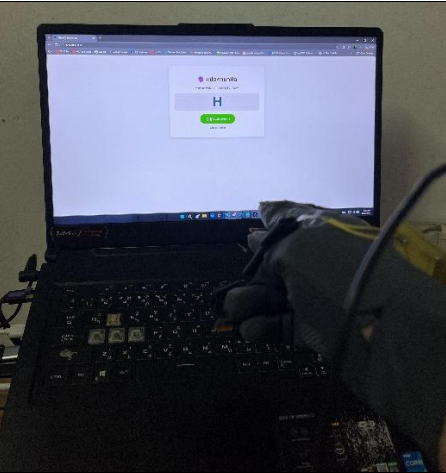
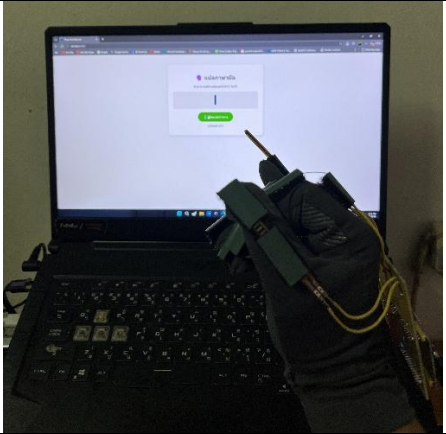
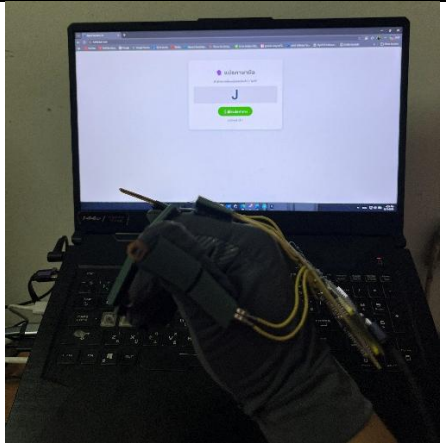
ตารางที่ 4.33 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและ คำ	ภาพถ่ายขณะทำและผลลัพธ์	
B		
C		
D		

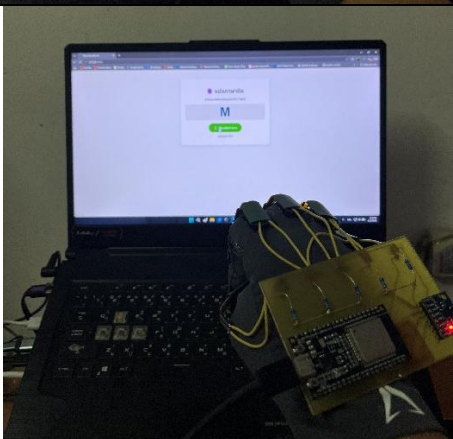
ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและ คำ	ภาพถ่ายขณะทำและผลลัพธ์	
E		
F		
G		

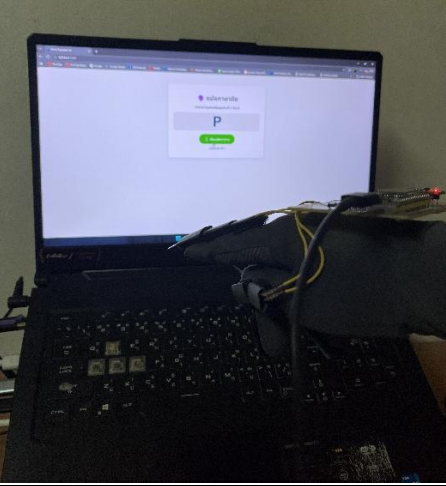
ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและ คำ	ภาพถ่ายขณะทำและผลลัพธ์	
H		
I		
J		

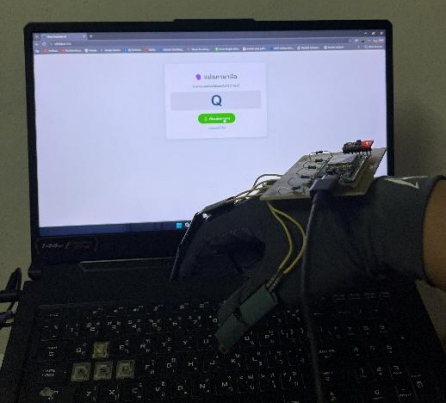
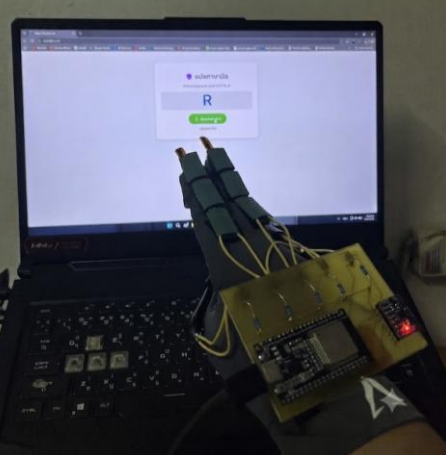
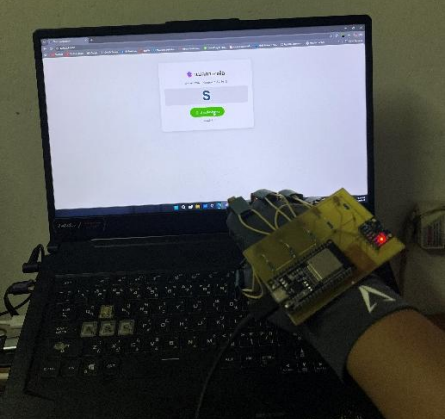
ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและ คำ	ภาพถ่ายขณะทำและผลลัพธ์	
K		
L		
M		

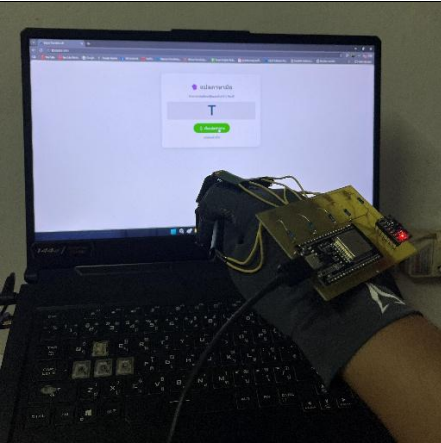
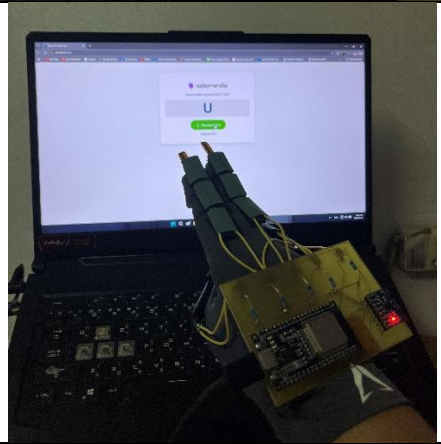
ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและคำ	ภาพถ่ายขณะทำและผลลัพธ์	
N		
O		
P		

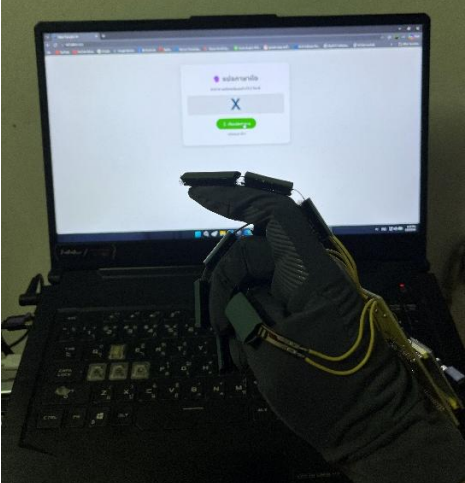
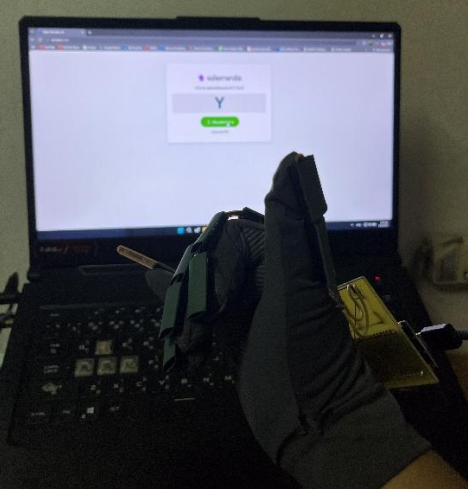
ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและ คำ	ภาพถ่ายขณะทำและผลลัพธ์	
Q		
R		
S		

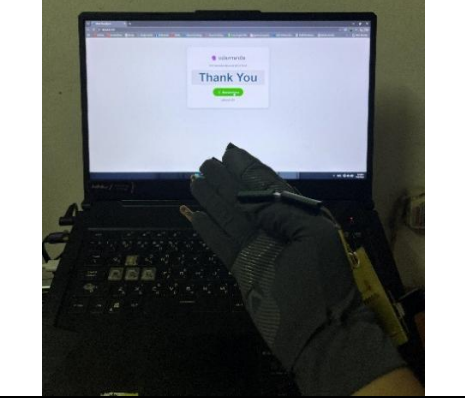
ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและ คำ	ภาพถ่ายขณะทำและผลลัพธ์	
T		
U		
V		

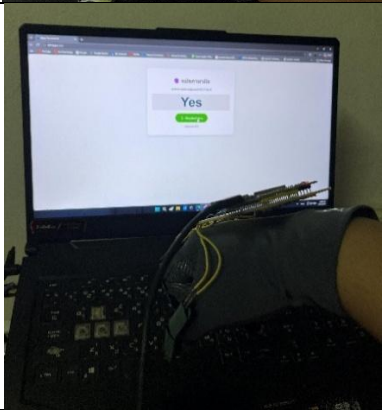
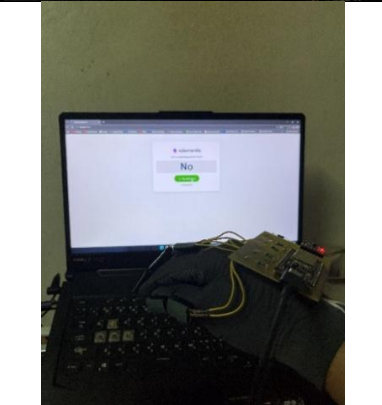
ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและ คำ	ภาพถ่ายขณะทำและผลลัพธ์	
W		
X		
Y		

ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและคำ	ภาพถ่ายขณะทำและผลลัพธ์	
Z		
Hello		
Thank you		

ตารางที่ 4.32 ผลการแสดงผลการจำแนกท่าทาง (ต่อ)

ตัวอักษรและคำ	ภาพถ่ายขณะทำและผลลัพธ์	
I love u		
Yes		
No		

4.4 สรุปผลการทดลอง

จากผลการดำเนินงานและการทดสอบใช้งานระบบถุงมือแปลภาษามืออัจฉริยะร่วมกับเว็บแอปพลิเคชันที่พัฒนาขึ้น สามารถสรุปผลตามขอบเขตที่กำหนดไว้ได้ดังนี้

1. ESP32 เป็นตัวส่งข้อมูล บอร์ดไมโครคอนโทรลเลอร์ ESP32 สามารถรับค่าสัญญาณจาก Flex Sensor และ MPU6050 แล้วส่งข้อมูลผ่าน Wi-Fi โพรโทคอล UDP พอร์ต 4210 ได้อย่างต่อเนื่องและมีความหน่วงต่ำ
2. Flex Sensor 5 ชิ้นวัดการงอของนิ้ว เซนเซอร์ทั้ง 5 ตัวทำงานได้อย่างถูกต้อง ให้ค่าความต้านทานที่สัมพันธ์กับการโค้งงอของนิ้วมือแต่ละนิ้ว ซึ่งแตกต่างกันอย่างชัดเจนตามแต่ละท่าทาง
3. MPU6050 วัดระนาบและการเคลื่อนไหวของข้อมือ โดยเซนเซอร์ให้ค่า Accelerometer และ Gyroscope ทั้ง 6 แกน ซึ่งเป็นข้อมูลสำคัญในการแยกแยะท่าทางที่มีรูปมือนคล้ายกันแต่มีการวางมือต่างกัน และท่าทางที่มีการเคลื่อนไหว เช่น J, Z, Hello
4. Flask แสดงผลพร้อมปุ่มควบคุม เว็บแอปพลิเคชันทำงานได้ตามที่ออกแบบไว้ ผู้ใช้งานสามารถกดปุ่มแปลเพื่อเริ่มเก็บข้อมูล 60 เฟรม และรับผลลัพธ์เป็นข้อความบนหน้าจอได้ทันที
5. รองรับมือเดียว 31 ท่าทาง ระบบรองรับการแปลท่าทางได้ครบตามขอบเขต ทั้งตัวอักษร A-Z และคำศัพท์ Hello, Thank you, I love you, Yes, No โดยโมเดล MLP Classifier มีค่าความแม่นยำรวม 96.02% ที่ได้จากการทดสอบท่าทางแต่ละรูปแบบซ้ำรูปแบบละ 30 ครั้ง
6. ทำนายจาก Flex Sensor และ MPU6050 ได้จริง ระบบสามารถรับค่าเซนเซอร์ 11 พารามิเตอร์ต่อเฟรม รวม 660 มิติต่อการแปล 1 ครั้ง และผ่านโมเดล MLP ได้ผลลัพธ์ที่ถูกต้องและเสถียรตามเกณฑ์ประสิทธิภาพของโครงการ

บทที่ 5

สรุปผลการดำเนินงานและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

จากการดำเนินงานตามขั้นตอนที่วางไว้ คณะผู้จัดทำสามารถพัฒนาและประกอบชุดถุงมือที่สามารถรับค่าสัญญาณการงอนิ้วมือและทิศทางข้อมือส่งผ่านระบบไร้สายไปยังเครื่องประมวลผลได้สำเร็จ ในด้านการพัฒนาปัญญาประดิษฐ์ โมเดลโครงข่ายประสาทเทียมแบบ Multi-layer Perceptron ที่ฝึกสอนด้วยชุดข้อมูลที่รวบรวมขึ้นเองมีประสิทธิภาพในการจำแนกท่าทางภาษามือ (ตัวอักษร A-Z และคำศัพท์พื้นฐาน) ได้อย่างแม่นยำ โดยมีค่าความแม่นยำ 96.02 % จากที่ทำในบทที่ 4.2 และเมื่อนำมาทำงานร่วมกับระบบเว็บแอปพลิเคชันพบว่าระบบสามารถแปลผลภาษามือและแสดงผลได้อย่างเสถียรและถูกต้องตามวัตถุประสงค์ที่ตั้งไว้

5.2 ปัญหาและอุปสรรค

1. ความทนทานของฮาร์ดแวร์ เซนเซอร์ตรวจจับการโค้งงอมีความบอบบาง หากใช้งานด้วยแรงกดที่มากเกินไปหรือพับงอผิดองศาอาจทำให้ค่าสัญญาณคลาดเคลื่อนหรือเซนเซอร์เสียหายได้
2. ความเสถียรของเครือข่ายไร้สาย เนื่องจากการส่งข้อมูลผ่านโปรโตคอล UDP ซึ่งไม่มีการยืนยันการรับส่งข้อมูล หากอยู่ในสภาพแวดล้อมที่มีสัญญาณ Wi-Fi หนาแน่น อาจเกิดการสูญเสียแพ็กเก็ตข้อมูล ได้ในบางช่วง
3. ข้อจำกัดของชุดข้อมูล ชุดข้อมูลที่ใช้สอนโมเดลส่วนใหญ่มาจากผู้ทดสอบคนเดียว ซึ่งอาจส่งผลต่อความแม่นยำเมื่อนำไปใช้กับผู้ใช้คนอื่นที่มีขนาดมือหรือลักษณะการทำท่าทางที่แตกต่างออกไป
4. การออกแบบและต่อวงจรที่มีความซับซ้อนมากอาจทำให้เกิดการต่อวงจรที่ผิดพลาด

5.3 แนวทางในการนำไปพัฒนาต่อ

1. การเพิ่มความหลากหลายของชุดข้อมูล รวบรวมข้อมูลจากกลุ่มตัวอย่างที่มีขนาดมือและสรีระที่แตกต่างกัน รวมถึงการเก็บข้อมูลในสภาพแวดล้อมที่หลากหลาย เพื่อให้โมเดลมีความทนทานและใช้งานได้กับทุกคน

2. การส่งออกแบบเสียง พัฒนาให้ระบบสามารถอ่านออกเสียงคำแปลได้ทันทีผ่านลำโพงของสมาร์ทโฟน เพื่อให้ผู้พิการทางการได้ยินสามารถสื่อสารกับบุคคลทั่วไปได้โดยไม่ต้องจ้องมองหน้าจอเพียงอย่างเดียว
3. พัฒนาแอปพลิเคชันบนมือถือ ย้ายส่วนประมวลผลและหน้าจอแสดงผลไปอยู่บนสมาร์ทโฟนผ่านทางแอปพลิเคชัน เพื่อเพิ่มความคล่องตัวในการพกพาไปใช้งานนอกสถานที่
4. การเชื่อมต่อกับระบบ AI ขนาดใหญ่ นำคำแปลที่ได้ไปประมวลผลต่ด้วย AI อย่าง ChatGPT หรือ Gemini เพื่อช่วยเกลาประโยคที่แปลจากภาษามือให้เป็นภาษาเขียนที่สละสลวยและเข้าใจง่ายขึ้น
5. การประมวลผลภาษามือแบบต่อเนื่อง ต่อยอดจากโมเดล Multi-layer Perceptron ไปสู่โครงสร้างแบบ Long Short-Term Memory หรือ Gated Recurrent Unit เพื่อให้ระบบสามารถแปลภาษามือที่มีการเคลื่อนไหวต่อเนื่องเป็นประโยคยาวๆ ได้

บรรณานุกรม

Arduino. (n.d.). Arduino IDE Software and Documentation.

,สืบค้น จาก <https://www.arduino.cc/en/software>

Espressif Systems. (n.d.). ESP32 Series Datasheet and Technical Reference Manual.

,สืบค้น จาก ESP32 Wi-Fi & Bluetooth SoC | Espressif Systems

Flask. (n.d.). Flask Documentation (User's Guide).

,สืบค้น จาก Welcome to Flask — Flask Documentation (3.1.x)

InvenSense. (n.d.). MPU-6050 Product Specification and Datasheet.

,สืบค้น จาก <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>

Python Software Foundation. (n.d.). Python 3.x Documentation.

,สืบค้น จาก <https://www.python.org/doc/>

Postel, J. (1980). User Datagram Protocol (RFC 768).

,สืบค้น จาก <https://datatracker.ietf.org/doc/html/rfc768>

Scikit-learn. (n.d.). Scikit-learn: Machine Learning in Python - Neural Network Models (Supervised Multi-layer Perceptron).

,สืบค้น จาก https://scikit-learn.org/stable/modules/neural_networks_supervised.html

W3C. (n.d.). JavaScript Object Notation (JSON) Data Interchange Format.

,สืบค้น จาก <https://www.w3.org/TR/json-ld/>

ภาคผนวก ก

โครงการพัฒนาระบบการแปลภาษา

#การนำเข้าไลบรารีที่จำเป็น

```

import os
import json
import joblib
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

```

#นำเข้าชุดข้อมูล

```

DATASET_FILE = 'dynamic_data_60fnew.json'
MODEL_FILE = 'smart_glove_model_dynamic2.pkl'
SCALER_FILE = 'scaler_dynamic2.pkl'

```

#กำหนดโมเดล

```

model = MLPClassifier

```

#เทรนโมเดล

```

model = MLPClassifier(
    hidden_layer_sizes=(256, 128),
    activation='relu',
    solver='adam',
    max_iter=500,
    random_state=42,
    verbose=True,
    early_stopping=False
)

```

#บันทึกโมเดลในรูปแบบ joblib

```

joblib.dump(model_data, MODEL_FILE)
joblib.dump(scaler, SCALER_FILE)

```

ภาคผนวก ข

โค้ดการทำงานของแอปพลิเคชันทำนายผลภาษามือ

#ส่วนนำเข้าไลบรารีที่ใช้

```
import json
import socket
import sys
import joblib
import numpy as np
from flask import Flask, jsonify, render_template
```

#ส่วนใช้โหลดโมเดลและข้อมูลที่จำเป็น

```
MODEL_FILE = 'smart_glove_model_dynamic2.pkl'
SCALER_FILE = 'scaler_dynamic2.pkl'
UDP_IP = "0.0.0.0"
UDP_PORT = 4210
```

ฟังก์ชันการทายผลของโมเดล

```
X_realtime = np.array(buffer_frames).reshape(1, -1)
X_realtime_scaled = scaler.transform(X_realtime)
prediction = model.predict(X_realtime_scaled)[0]
```

ภาคผนวก ค

โค้ดการแทรกการส่งข้อมูล

#ส่วนนำเข้าไลบรารีที่ใช้

```
#include <WiFi.h>
#include <WiFiUdp.h>
#include <Arduino_JSON.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
```

#ส่วนกำหนด IP ที่ใช้

```
const char* ssid    = "BB";
const char* password = "12345678";
const char* udpAddress = "172.20.10.13";
const int udpPort = 4210;
```

#ส่วนกำหนด IP ที่ใช้

```
float ax_off = 0, ay_off = 0, az_off = 0;
float gx_off = 0, gy_off = 0, gz_off = 0;
```

#ฟังก์ชัน Calibrate

```
void calibrateMPU() {
    int samples = 200;
    for (int i = 0; i < samples; i++) {
        sensors_event_t a, g, temp;
        mpu.getEvent(&a, &g, &temp);
        ax_off += a.acceleration.x;
        ay_off += a.acceleration.y;
        az_off += a.acceleration.z - 9.81;
        gx_off += g.gyro.x;
        gy_off += g.gyro.y;
        gz_off += g.gyro.z;
        delay(10);
    }
}
```

```

ax_off /= samples;
ay_off /= samples;
az_off /= samples;
gx_off /= samples;
gy_off /= samples;
gz_off /= samples;
}

```

#ฟังก์ชัน การส่งข้อมูล

```

void sendData() {
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);
    JSONVar payload;
    JSONVar flex;
    flex[0] = analogRead(35);
    flex[1] = analogRead(34);
    flex[2] = analogRead(39);
    flex[3] = analogRead(36);
    flex[4] = analogRead(32);
    payload["flex"] = flex;
    JSONVar acc;
    acc[0] = a.acceleration.x - ax_off;
    acc[1] = a.acceleration.y - ay_off;
    acc[2] = a.acceleration.z - az_off;
    payload["acc"] = acc;
    JSONVar gyro;
    gyro[0] = g.gyro.x - gx_off;
    gyro[1] = g.gyro.y - gy_off;
    gyro[2] = g.gyro.z - gz_off;
    payload["gyro"] = gyro;
    String jsonString = JSON.stringify(payload);
}

```

```
udp.beginPacket(udpAddress, udpPort);  
udp.print(jsonString);  
udp.endPacket();  
}
```

ภาคผนวก ง

โครงการเพรณการเก็บข้อมูล

#นำเข้าไลบรารีที่จำเป็น

```
import os
import json
import socket
import datetime
```

#กำหนดรูปแบบการเซฟข้อมูลและ IP

```
UDP_IP = "0.0.0.0"
UDP_PORT = 4210
FILENAME = "dynamic_data_60fnew.json"
TARGET_FRAMES = 60
```

#ฟังก์ชัน การเก็บข้อมูล

```
if len(sequence_buffer) == TARGET_FRAMES:
    record = {
        "label": current_label,
        "data": sequence_buffer,
        "timestamp": datetime.datetime.now().isoformat()
    }
    data_store.append(record)
    print(" Done!")
    with open(FILENAME, 'w', encoding='utf-8') as f:
        json.dump(data_store, f, indent=4)
```

ประวัติผู้เขียน

ชื่อ - นามสกุล

ศุภโชค ชูใจ

วัน - เดือน - ปีเกิด

เกิดวันที่ 18 ธันวาคม พ.ศ.2546

สถานที่เกิด

จังหวัดกรุงเทพมหานคร

การศึกษา

โรงเรียน พระแม่มาลีสาธิตประดิษฐ์

มัธยมศึกษาชั้นปีที่ 6, 2565

มหาวิทยาลัยรังสิต ปริญญาวิศวกรรมศาสตรบัณฑิต

สาขาวิศวกรรมคอมพิวเตอร์, 2569